# Mystic User Guide

Version W-2024.09, September 2024

SYNOPSYS®

# Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

## Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

## Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

## Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

## Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

## Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

# Contents


Conventions . . . . . 6

Customer Support . . . . . 6

Statement on Inclusivity and Diversity . . . . . 7

**1. Introduction to the Mystic Tool** . . . . . 8

Overview of Mystic Functionality . . . . . 8

Mystic Tool Flow . . . . . 8

Mystic API Documentation . . . . . 9

Sentaurus Workbench API Documentation . . . . . 10

TCAD Sentaurus Tutorial: Simulation Projects . . . . . 10

Installing the Mystic Tool . . . . . 10

Running the Mystic Tool as a Standalone Tool . . . . . 10

Extraction Environment . . . . . 12

**2. Uploading Target Data to Database** . . . . . 15

Basics of Uploading Target Data . . . . . 15

The Data Class . . . . . 15

Updating PLT Files . . . . . 17

Uploading CSV Files . . . . . 18

**3. SPICE Model Extraction Strategy** . . . . . 20

Database . . . . . 20

Target Data . . . . . 23

Multilevel Data Structures . . . . . 25

Filtering . . . . . 25

Figure of Merit Transforms . . . . . 27

Data Masks . . . . . 31

Generic PrimeSim HSPICE .MEASURE Commands . . . . . 32

Bias Extension . . . . . 33

SPICE Models . . . . . 34
Simulator . . . . . 36
Parallel PrimeSim HSPICE Simulations . . . . . 38
Optimization Parameters . . . . . 39
Optimal Parameter Transforms: ExtractionUtils.GetOptimalParameterTransform 41
Parameter Space Exploration . . . . . 41
Sensitivity Analysis: ExtractionUtils.SensitivityAnalysis . . . . . 42
Parameter Sweep: ExtractionUtils.ParamSweep . . . . . 42
Optimization Algorithms . . . . . 42
Bayesian Optimization Algorithm . . . . . 45
Machine Learning in the Mystic Tool . . . . . 46
Data Storage and Retrieval . . . . . 46
Rerunning a Strategy . . . . . 46
Reverse Search Extractions . . . . . 48
ML Classifier Accelerated Extraction . . . . . 50
The Engine Module . . . . . 51
Full Strategy . . . . . 52
Configuring Settings . . . . . 52
Setting Up the Model . . . . . 54
Loading and Filtering Data . . . . . 54
Defining the Optimization Parameters . . . . . 55
Executing the Extraction Strategy . . . . . 56
Writing the Results . . . . . 57

**4. Special Use Cases** . . . . . 58
Transient Simulation Extraction . . . . . 58
MOSFET Reliability (MOSRA) Extraction . . . . . 60
Variability Extraction Using Principal Component Analysis . . . . . 62
Correlated Parameters . . . . . 63
Variability Target Data . . . . . 66
RF Extraction . . . . . 68
Uploading Data . . . . . 68
Loading and Manipulating Data . . . . . 71
Extracting Parameters . . . . . 72
Data Output . . . . . 73
Lookup Table Modeling . . . . . 73

Josephson Junction Extraction . . . . . 75
Extracting Figures of Merit . . . . . 75
Optimization . . . . . 76
Data Output . . . . . 78

**5. Mystic Graphical User Interface** . . . . . 79

Features of the Mystic GUI . . . . . 79
Menus and Toolbars . . . . . 80
Editor Panel . . . . . 81
Inserting Breakpoints in Strategy File . . . . . 82
Syntax Checker Panel . . . . . 82
Console Panel . . . . . 84
Controlling the Execution of the Strategy . . . . . 85
Plot Log Panel . . . . . 86
Doc Lookup Panel . . . . . 87
Variable Explorer Panel . . . . . 88
Plot Area Panel . . . . . 90
Plot Parameters Panel . . . . . 95
Wizards Panel . . . . . 99

Usage Examples . . . . . 100
Running a Strategy Interactively . . . . . 101
Exploring Parameters Interactively . . . . . 104
Parallel Coordinates Plots . . . . . 106
RF Smith Charts and Polar Charts . . . . . 108

References . . . . . 111

**6. Mystic Examples** . . . . . 112

Location of Examples . . . . . 112

14 nm FinFET Example . . . . . 113
First Stage: Low-Drain Extraction . . . . . 113
Second Stage: High-Drain and $I_d$–$V_d$ Extraction . . . . . 115
Third Stage: C–V Extraction . . . . . 116

**Glossary** . . . . . 118

**Index** . . . . . 119

# About This Guide

This document describes the functionality of the Synopsys® Mystic tool.

For additional information, see:

- The TCAD Sentaurus™ release notes, available on the Synopsys SolvNetPlus site (see Accessing SolvNetPlus)
- Documentation available on the SolvNetPlus site

## Conventions

The following conventions are used in Synopsys documentation.

| Convention | Description |
|---|---|
| **Bold text** | Identifies a selectable icon, button, menu, or tab. It also indicates the name of a field or an option. |
| `Courier font` | Identifies text that is displayed on the screen or that the user must enter. It identifies the names of files, directories, paths, parameters, keywords, and variables. |
| *Italicized text* | Used for emphasis, the titles of books and journals, and non-English words. It also identifies components of an equation or a formula, a placeholder, or an identifier. |

## Customer Support

Customer support is available through the SolvNetPlus site.

### Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site:

1. Go to https://solvnetplus.synopsys.com.
2. Enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register.)

### Contacting Synopsys Support

If you have problems, questions, or suggestions, contact Synopsys support in the following ways:

- Go to the Synopsys Global Support Centers site on www.synopsys.com to find email addresses and telephone numbers for Synopsys support centers throughout the world.
- Go to the SolvNetPlus site and open a case (Synopsys user name and password required).

### Contacting Your Local TCAD Support Team Directly

Send an email message to:

- support-tcad-us@synopsys.com from within North America and South America
- support-tcad-eu@synopsys.com from within Europe
- support-tcad-ap@synopsys.com from within Asia Pacific (China, Taiwan, Singapore, Malaysia, India, Australia)
- support-tcad-kr@synopsys.com from Korea
- support-tcad-jp@synopsys.com from Japan

## Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

# 1

# Introduction to the Mystic Tool

*This chapter introduces the Mystic extraction tool.*

## Overview of Mystic Functionality

The Mystic tool is a Python-based framework designed to facilitate the extraction of SPICE compact models, using the Synopsys PrimeSim™ HSPICE® tool. The process of extracting a SPICE model manually involves adjusting the parameter values of a SPICE compact model, running simulations with a SPICE engine, and comparing these simulation results to a set of target data curves, usually coming from measurement or TCAD data. This process is repeated as many times as necessary to obtain an accurate fit with the target data and requires a strong understanding of the underlying equations of the SPICE model and experience in setting up and running SPICE simulations.

The Mystic tool aims to limit barriers to using SPICE modeling by providing a flexible framework to assist with all parts of the procedure, from parsing input SPICE models, to automatically setting up, running, and parsing outputs from SPICE simulations based on provided target data, to optimizing the values of the SPICE model parameters using a range of sophisticated optimization algorithms.

### Mystic Tool Flow

This section looks at the standard Mystic tool flow, including the input requirements, core components, and tool outputs. Figure 1 shows a block-level overview of a standard Mystic tool flow.

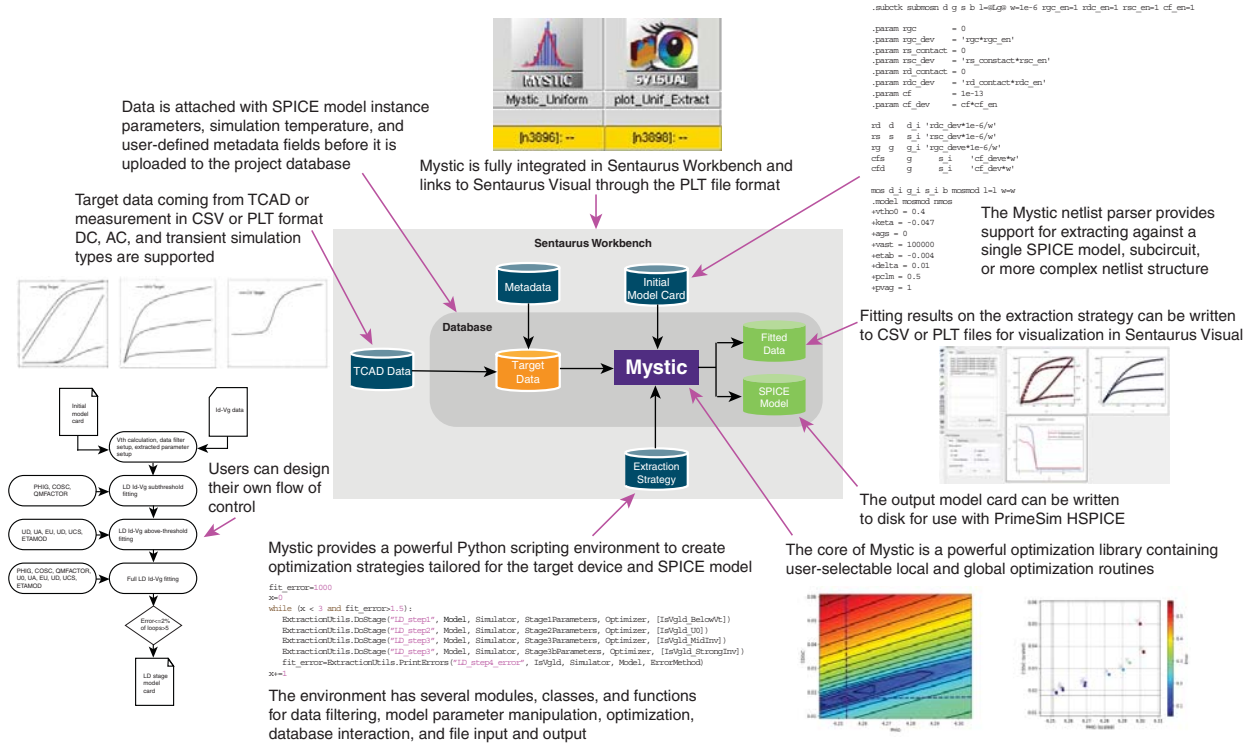
The Mystic tool is contained within Sentaurus Workbench. This integration allows you to easily connect to TCAD Sentaurus projects that can provide target data for SPICE compact model extraction. It also helps with management of splits, such as different temperatures and process conditions, provides access to local compute clusters, and allows you to visualize SPICE model extraction results using Sentaurus Visual.

Underneath Sentaurus Workbench, the Mystic tool provides a local database for consolidating target data and simulation conditions, such as temperature and instance

parameters, and transferring this information between sequential Mystic extraction stages. It can also be used to track SPICE model parameter values across stages and to integrate with the larger TCAD to SPICE tool flow.

This user guide describes each of the main components in the Mystic tool flow and gives you the tools to create your own SPICE model extraction flow in Mystic.

*Figure 1 Overview of Mystic tool flow*




## Mystic API Documentation

The Mystic API documentation provides detailed information about all the Python commands and elements used in the Mystic tool.

You can access the API in the following location:

```
$STROOT/tcad/$STRELEASE/manuals/tcad_spice/mystic_api/index.html
```

The `STROOT` environment variable indicates where the Synopsys TCAD distribution has been installed.

### Sentaurus Workbench API Documentation

From the graphical user interface of Sentaurus Workbench, you can access the Python API documentation by choosing **Help** > **Python API Documentation** > **SWB**.

### TCAD Sentaurus Tutorial: Simulation Projects

The TCAD Sentaurus Tutorial provides projects demonstrating the capabilities of TCAD to SPICE products.

To access the TCAD Sentaurus Tutorial:

1. Open Sentaurus Workbench by entering the following on the command line: `swb`
2. From the menu bar of Sentaurus Workbench, choose **Help** > **Training** or click  on the toolbar.

Alternatively, to access the TCAD Sentaurus Tutorial:

1. Go to the `$STROOT/tcad/current/Sentaurus_Training` directory.

   The `STROOT` environment variable indicates where the Synopsys TCAD distribution has been installed.
2. Open the `index.html` file in your browser.

### Installing the Mystic Tool

For detailed setup and installation instructions, see the *TCAD Installation Notes*.

## Running the Mystic Tool as a Standalone Tool

If you run the Mystic tool through Sentaurus Workbench, then the execution of the Mystic tool and the configuration of command-line options are handled automatically. For instructions on setting up the Mystic tool, see the *TCAD Installation Notes*.

After you have loaded the TCAD to SPICE tools into your environment, you can run Mystic as follows:

```
Mystic <filename> -d <string> -m <string> [other_options]
```

The mandatory file name specifies a Python file containing a structured set of commands that perform the SPICE model extraction. The `-d` option provides an optional path to the TCAD to SPICE database configuration file. If you want to use the database when running Mystic in standalone mode, then it must be configured separately. The `-m` option specifies a

SPICE model compatible with PrimeSim HSPICE. For details, see SPICE Models on page 34.

*Table 1 Command-line options for Mystic*

| Option (short form followed by long form) | Description |
|---|---|
| `-ac`<br>`--auto-close` | Automatically close the GUI window when the strategy execution finished if you are running interactively in the Mystic GUI. |
| `<filename>` | Required. Mystic strategy to execute. |
| `-ar`<br>`--auto-run` | Automatically start the extraction strategy if you are running interactively in the Mystic GUI. |
| `-d <string>`<br>`--db <string>` | Optional. Specifies the location of the configuration file of the TCAD to SPICE database. |
| `-ds <string>`<br>`--dataset <string>` | Optional. Specifies the name of the dataset in the database in which to store the Mystic extraction results. |
| `-g`<br>`--gui` | Optional. Starts the Mystic tool in interactive mode using the Mystic graphical user interface. |
| `-h`<br>`--help` | Optional. Prints usage information and command-line options, and then exits. |
| `-l <string>`<br>`--location <string>` | Optional. Specifies the full path of the PrimeSim HSPICE executable to use. This option overrides the version in the system path. |
| `-m <string>`<br>`--model <string>` | Optional. Specifies the file name of the initial model to be loaded into the extraction strategy. |
| `-o <string>`<br>`--optimizer <string>` | Optional. Specifies the optimization algorithm to use in the strategy (see Optimization Algorithms on page 42). |
| `-v`<br>`--version` | Optional. Prints the Mystic version and then exits. |
| `-x <string>`<br>`--spice-args <string>` | Optional. Specifies additional arguments to pass to the PrimeSim HSPICE backend simulation at runtime. |

When running in interactive mode using the `-g`/`--gui` flag, the Mystic tool can be opened without a strategy filename as follows:

```
Mystic --gui
```

When the GUI opens, you can use the file browser panel to open and run a strategy in the GUI. For more information about the Mystic GUI, see Chapter 5 on page 79.

## Extraction Environment

The Mystic extraction environment is a collection of specific Python modules, classes, and objects that are designed to help with SPICE compact model extraction (see Table 2). You have full freedom to assemble these in a strategy file to suit the specific needs of your extraction.

*Table 2 Python modules, classes, and objects used in Mystic environment*

| Module, class, or object | Description |
|---|---|
| `ExtractionUtils` | This module provides utility functions to help with extraction. These functions usually combine `Model`, `Simulator`, `Optimizer`, and `SimData` objects. |
| `Corrector` | This module provides functionality for applying small analytic corrections on top of a SPICE model to perfectly match target data. This should be used when extreme accuracy is required. |
| `LinkedOptimizationParameter` | This class allows you to link one parameter value to another parameter value to ensure they remain consistent throughout the extraction. |
| `OptimizationParameter` or `OptParam` | This class defines SPICE model parameters to be extracted. These keep track of the current value of parameters throughout the extraction. |
| `mdb` | This object provides access to the TCAD to SPICE database where target data is stored. It can be used to load data into the environment at the start of the strategy and to store fitted models and data at the end. |
| `Model` | This object allows you to retrieve and set SPICE model parameter values inside the strategy. |
| `Optimizer` | This object provides an interface to the Mystic optimization library. It features a range of sophisticated algorithms for optimizing the values of SPICE model parameters. |
| `SimData` | This object provides a container format for target and simulation data. Usually, `SimData` objects can be loaded into the extraction strategy from the database, but they can also be initialized from scratch if necessary. |
| `Simulator` | This object represents the backend PrimeSim HSPICE simulator and can be used to set command-line options, global options, and the simulation temperature. |

Objects that are passed into the environment are dependent on the command-line options passed to the Mystic tool. For example:

- If no initial model card is passed using the `-m` option, then the `Model` object in the environment will default to `None` and must be initialized within the strategy.
- If no database configuration file is passed using the `-d` option, then the `mdb` object will default to `None`.

- When running the Mystic tool through Sentaurus Workbench, the command-line options are handled automatically, so all objects in the extraction environment are configured correctly.

# 2

# Uploading Target Data to Database

*This chapter describes how to upload target data to the database.*

## Basics of Uploading Target Data

As mentioned in Mystic Tool Flow, the Mystic tool is most commonly run within Sentaurus Workbench as part of a larger TCAD to SPICE tool flow. These tool flows provide a local database for propagating information throughout the flow. The most common way of preparing target data for a Mystic extraction is to upload it to the database using a TCAD to SPICE Python tool instance located before the Mystic tool in the flow.

This methodology allows data to be preprocessed into the correct structure and tagged with useful metadata fields that the Mystic tool can utilize, such as simulation temperature and SPICE model instance parameters.

This chapter discusses the different file types that can be used as target data and how to upload these to the database using TCAD to SPICE Python. It also describes the required metadata fields that must be attached to the target data and how the Mystic tool uses these fields.

## The Data Class

The `Data` class is a custom Python class that is inserted into the TCAD to SPICE Python environment at runtime. It provides functionality for reading and writing PLT and CSV files, as well as a link to the database that can be used to save and retrieve simulation data.

In the Mystic environment, the `Data` class is used to represent all target and simulation data. It provides functionality for filtering, resampling, applying transforms, and calculating errors between targets and simulations during optimization. The structure of `Data` objects is used to construct the backend PrimeSim HSPICE netlists that are created and simulated by the Mystic tool during the extraction process. Enough information must be provided through the `metadata` dictionary for the Mystic tool to replicate the simulation conditions of the target in the PrimeSim HSPICE tool. Figure 2 shows a representation of a standard $I_d$–$V_g$ `Data` object.

*Figure 2 Data format*

```
bias : {'drain': 0.05, 'source': 0.0, 'substrate': 0.0}
instances : {'1': 2.5e-08, 'nfin': 1}
nodes : ['drain', 'gate', 'source', 'substrate']
temperature : 27.0
ftin : 8e-09
tspacer : 8e-09
workfn : 4.301
dtype : nMOS
eot : 1e-09
hfin : 4e-09


                    idrain  idrain-fit  errors
vgate
0.000000e+00 -4.654437e-09         nan     nan
2.500000e-02 -1.059546e-08         nan     nan
5.000000e-02 -2.393837e-08         nan     nan
7.500000e-02 -5.346387e-08         nan     nan
1.000000e-01 -1.171774e-07         nan     nan
```

The key–value pair dictionary at the top is the metadata. This contains static bias conditions, SPICE model instance parameters, an ordered list of SPICE model instance terminals, the simulation temperature, and some extra arbitrary fields that help to distinguish between `Data` objects when loading and filtering. These are particularly useful if you have a large design-of-experiments (DoE) of data at different process conditions.

Of these, the bias, temperature, nodes, and instances are required fields. Biases are extracted automatically from the PLT or CSV files if possible, but they can also be supplied explicitly if required (see Updating PLT Files on page 17).

The nodes list must match both the terminals of the model being extracted by the Mystic tool and the names of the biases in the `Data` object. In Figure 2, you can see that drain, source, and substrate are all present in the nodes list and `bias` dictionary (gate voltage does not have to be present in the `bias` dictionary as it is the independent variable being swept in the simulation). The simulation temperature should be entered into the `Data` class in kelvin. The Mystic tool converts this temperature to degree Celsius internally, as required by the SPICE model.

Entries in the instances dictionary must match the instance parameters in the SPICE model. This example comes from a BSIM-CMG SPICE model, which takes `l` and `nfin` instance parameters. This dictionary can remain empty if the default values should be used.

The core of the data is generally a single independent variable sweep (in this case, gate voltage) and a single dependent variable (in this case, drain current). Further dynamic input currents or voltages can be added as stimulus columns. This is mainly used for the transient simulation type and is discussed in Transient Simulation Extraction on page 58. The column headers determine the type of PrimeSim HSPICE simulation that the Mystic tool will produce for the data in question. Table 3 shows the different valid column notations.

*Table 3 Valid column notations*

| Notation | As ivar \| stimulus \| bias | As dvar |
|---|---|---|
| `vcontact` | Voltage source, either stationary or a sweep, applied by using a `.data` statement in the netlist | A contact where a voltage response is measured |
| `icontact` | Current source, either stationary or a sweep, applied by using a `.data` statement in the netlist | A contact from which a current response is measured |
| `c(contact0,contact1)` | n/a | Capacitance, with a small signal applied to `contact0` and measured at `contact1` |
| `time` | Time values for transient simulations | n/a |

When the `Data` object is created, two placeholder columns are added automatically for the PrimeSim HSPICE simulation data and a pointwise error calculation. Mystic populates these columns during the extraction.

For detailed information about the `Data` class, see the Mystic API documentation.

## Updating PLT Files

The most common file format for Mystic extractions is the PLT file. PLT files are produced by Sentaurus Device simulations, which usually provide the target data for TCAD to SPICE flows. The following example demonstrates how to read in a Sentaurus Device PLT file and to upload its contents to the database for use with the Mystic tool:

```
ds = dbi.create_dataset(node_prj, "iv-data", clean=True)

metadata = {'temperature': 300,
            'instances': {'l': 27e-9, 'nfin':1},
            'nodes': ['drain','gate','source','substrate']}

Data.from_plt('idvgld.plt', metadata=metadata, upload_ds=ds)
```

First, a location in the database to store the data is created using `dbi.create_dataset`, where `dbi` is the Python interface to the TCAD to SPICE database. For more information, see the TCAD to SPICE Python API documentation.

Then, you create a metadata dictionary with the required information described in The Data Class on page 15. No bias dictionary is defined here because Sentaurus Device PLT files

contain all of the static and dynamic biases for the simulation, so the bias dictionary is populated automatically when the file is read.

Finally, you use the `Data.from_plt` method to simultaneously read the information from the specified file and to upload it to the database in the created location. The PLT reader automatically recognizes some of the default Sentaurus Device notation and converts it to the format required by the Mystic tool. For example:

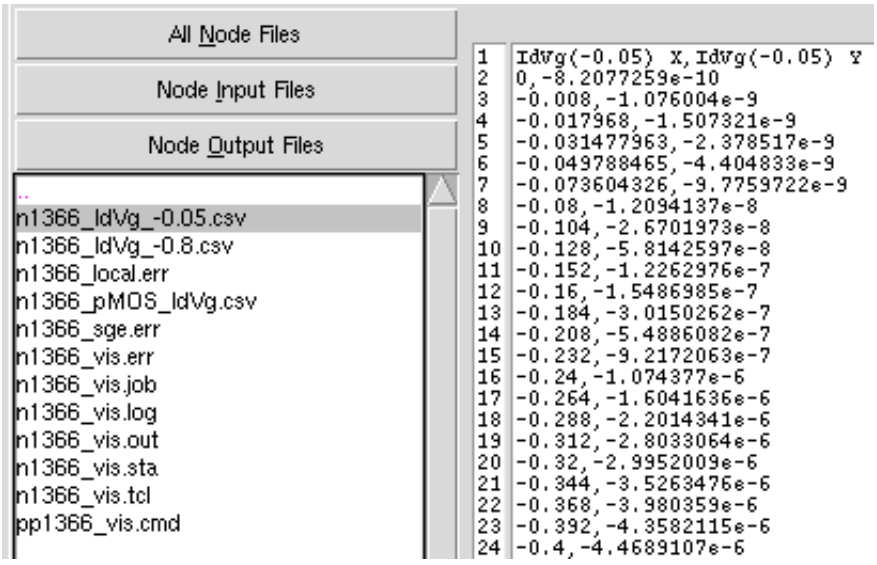
- `"contact OuterVoltage"` is converted to `"vcontact"`.
- `"contact TotalCurrent"` is converted to `"icontact"`.

Capacitances tend to be in the correct format already and do not need to be converted. If the column headers are not in the expected format, then they can also be converted explicitly by using a mapping (see Uploading CSV Files). The independent and dependent variables are detected automatically as the file is read, but they can also be specified explicitly with the ivar and dvar function arguments if required.

## Uploading CSV Files

The CSV file is a very flexible format and allows a wider range of targets to be supported by the Mystic tool. When working with CSV files, you must provide some extra information to ensure the data labels are converted into the format required by the Mystic tool. Consider the example $I_d$–$V_g$ CSV file shown in Figure 3.

*Figure 3 Example of $I_d$–$V_g$ CSV file*



In this case, the columns still contain the gate voltage and drain current, as in the previous example, but the column headers are not in a format that the Mystic tool recognizes. The following example shows how to upload this file:

```
ds = dbi.create_dataset(node_prj, "iv-data", clean=True)

metadata = {'temperature': 300,
            'instances': {'l': 27e-9, 'nfin':1},
            'nodes': ['drain','gate','source','substrate'],
            'bias': {'drain': -0.05,
                     'source': 0.0,
                     'substrate': 0.0}}

column_map = {'IdVg(-0.05) X': 'vgate',
              'IdVg(-0.05) Y': 'idrain'}

Data.from_csv('n1366_IdVg_-0.05.csv', metadata=metadata,
              column_map=column_map, upload_ds=ds)
```

In this example, note the following:

- There is now a `bias` dictionary in the metadata.
- This CSV file contains no static biases, so values for the drain, source, and substrate voltages must be added manually.
- A column mapping dictionary can be defined to instruct the `Data` class how to convert column headers into the correct format. This dictionary is passed into the `Data.from_csv` function as a `column_map` argument. After the data is stored in the database, it will have exactly the same formatting as PLT data and can be used by the Mystic tool to extract a SPICE model.

# 3

# SPICE Model Extraction Strategy

*This chapter describes aspects of the Mystic environment and demonstrates how it contributes to the construction of a SPICE model extraction strategy as a whole.*

## Database

The Mystic tool provides a link to the TCAD to SPICE database for consolidating target data and tracking the values of SPICE model parameters between sequential extraction stages. When running the Mystic tool through Sentaurus Workbench, the database connection is managed automatically with the path to the database configuration file being passed to the Mystic tool on the command line using the `-d` option.

If you run the Mystic tool standalone, then you must configure and manage the database connection manually. For information on how to do this, see Sentaurus Workbench API Documentation on page 10.

In the extraction environment, the `mdb` object is provided to store and retrieve information from the database. A standard Mystic extraction tool flow can contain up to five separate extraction stages, where each performs a subset of the fitting, allowing you to check successful parts of their extraction and to avoid continually rerunning the full strategy during development. The database ensures consistency of data and SPICE model parameter values between these stages as shown in Figure 4 on page 21.

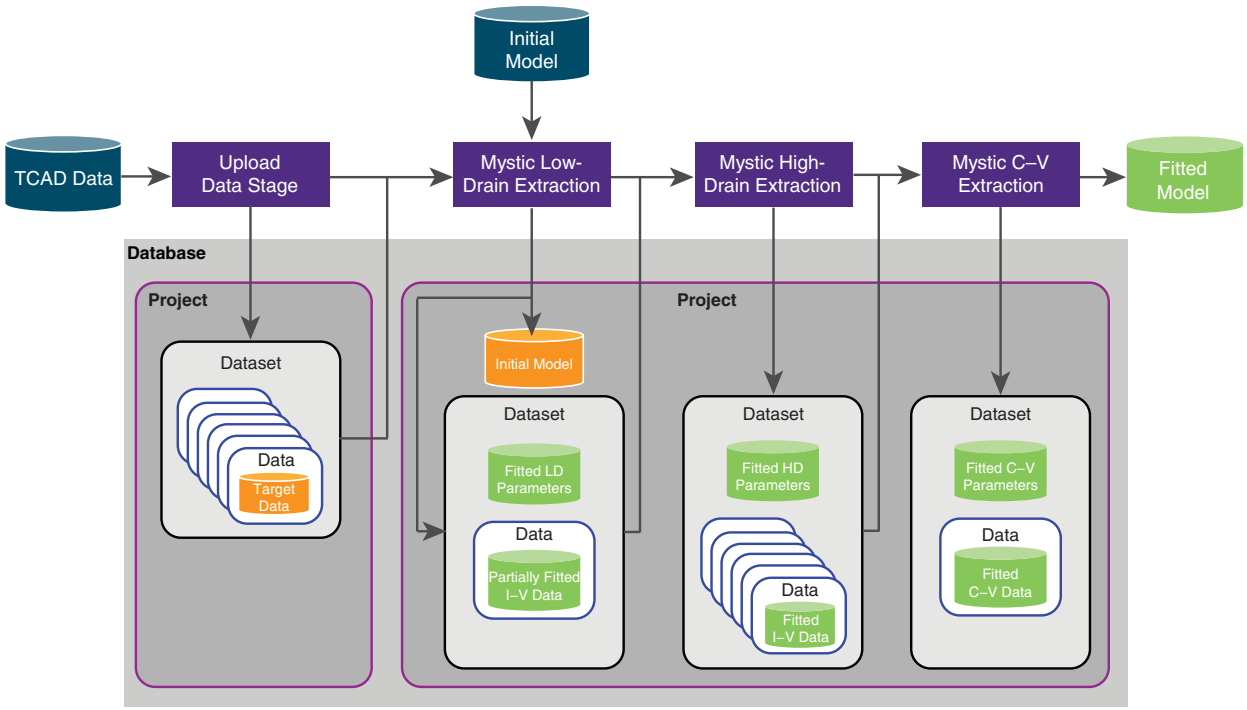
Figure 4 represents a three-stage SPICE model extraction for a MOSFET device with I–V and C–V target data. The upload data stage is a prerequisite for pushing target data into the database before the Mystic extraction. For details on this process, refer to the TCAD to SPICE module of the TCAD Sentaurus Tutorial (see TCAD Sentaurus Tutorial: Simulation Projects on page 10).

The database has a hierarchical structure with three levels: projects, datasets, and data objects. A project represents the full Mystic extraction strategy or one single node in Sentaurus Workbench. The main thing stored at this level is the initial model card, which can be added to the project at the start of the first extraction strategy as follows:

```
mdb.StoreInitialModel(Model)
```

where `Model` is the `Model` object passed in to the environment by the `-m` option. This model is then used as a point of reference in all downstream stages when setting up the `Model` object. Underneath the project is a list of datasets representing each individual extraction stage. Again, these are created by Sentaurus Workbench and passed to the Mystic tool using the `-ds` option.

*Figure 4 Interaction with database in a standard Mystic model extraction*




The datasets provide a container for extracted SPICE model parameters from each stage. Each dataset is linked to the previous one. In downstream stages, this makes it easy to retrieve the model parameters from the previous extraction stage using the `mdb.GetFitModel` method as follows:

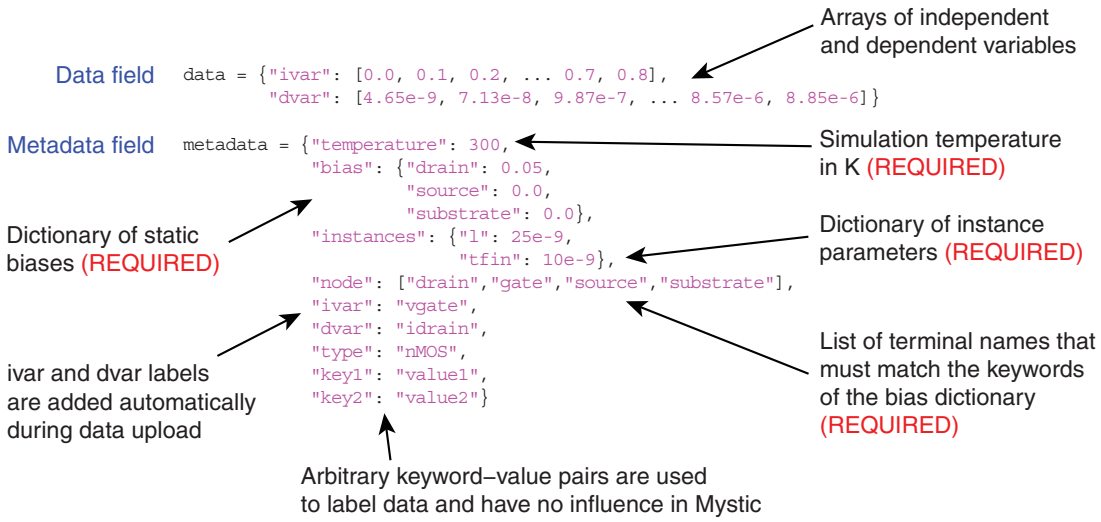
```
Model = mdb.GetFitModel()
```

Datasets also contain a list of data objects, where each object represents one piece of fitted target data. `Data` objects are split into two main fields:

- The *data field* is a dictionary of arrays containing independent and dependent variable data.
- The *metadata field* is a hierarchical dictionary of extra information, which can be either simulation conditions that will be used by the Mystic tool or simply unique tags that can be used to help query the data.

Figure 5 shows the standard Mystic metadata structure.

*Figure 5 Structure of metadata in the database*




It is important to understand the metadata requirements of the Mystic tool, so you can define it correctly when uploading data and know which fields are available when loading data back from the database.

In the Mystic strategy, data is retrieved from the database using the `mdb.Load` and `mdb.GetFitData` methods:

- `mdb.Load` is used to load target data from the upload data project in the first stage of an extraction strategy.
- `mdb.GetFitData` is usually used to load partially fitted data in downstream stages of the Mystic extraction.

Both methods take arbitrary filter arguments that select data from the database based on its project, data, and metadata fields. For example, the low-drain $I_d$–$V_g$ data shown in Figure 5 could be loaded by using:

```
IdVgld = mdb.Load(temperature=300, bias__drain=0.05,
                  instances__l=25e-9, ivar="vgate", dvar="idrain")
```

**Note:**

To match hierarchical fields such as biases and instance parameters, a double underscore is used to separate the levels of hierarchy.

All data entries matching the filter criteria are included in the return. The `mdb.Load` and `mdb.GetFitData` methods return `SimData` objects. For more information about these, see Target Data on page 23.

Fitted data can be stored back in the database at the end of an extraction strategy using the `mdb.StoreFitData` method.

## Target Data

The Mystic tool can accept target data from any PLT-like or CSV-like file format. As described in Chapter 2, Uploading Target Data to Database, this data is uploaded to the Mystic database in a separate upload data tool instance at the start of the TCAD to SPICE flow and is attached with metadata such as simulation temperature and instance parameters. This provides the Mystic tool with enough information to set up backend PrimeSim HSPICE simulations without further input from users.

**Note:**

Only I–V, C–V, transient, and RF scattering parameter data types are supported for extraction.

As previously mentioned, data can be retrieved from the database in the extraction strategy using the `mdb` object. The `mdb.Load` and `mdb.GetFitData` methods both return `SimData` objects, which are the data containers used in the Mystic tool. These consist of one or more datasets that usually have four columns:

- An independent variable
- A target-dependent variable (TCAD or measurement)
- A fitted-dependent variable (PrimeSim HSPICE simulation data)
- A pointwise error

When the Mystic tool creates backend PrimeSim HSPICE simulations, `SimData` objects are used to automatically create biasing, stimulus, and output commands in the PrimeSim HSPICE netlists that match the simulation conditions of the targets as shown in Figure 6.

This ensures consistency between target data and fitted data when calculating errors and removes the burden from users of having to define and manage this information themselves. From an optimization perspective, the most important column of the `SimData` objects is the pointwise error column, which is passed back to the `Optimizer` object after every evaluation and dictates how it adjusts the parameters. Table 4 lists the options of the pointwise error method.

The default option is `rel`, but you can change it at either a global level or on an object-by-object basis as follows:

```
SimData.SetOptions(error_method="abs")
```

*Figure 6 Example of how the Mystic tool transforms SimData objects into PrimeSim HSPICE netlist commands*

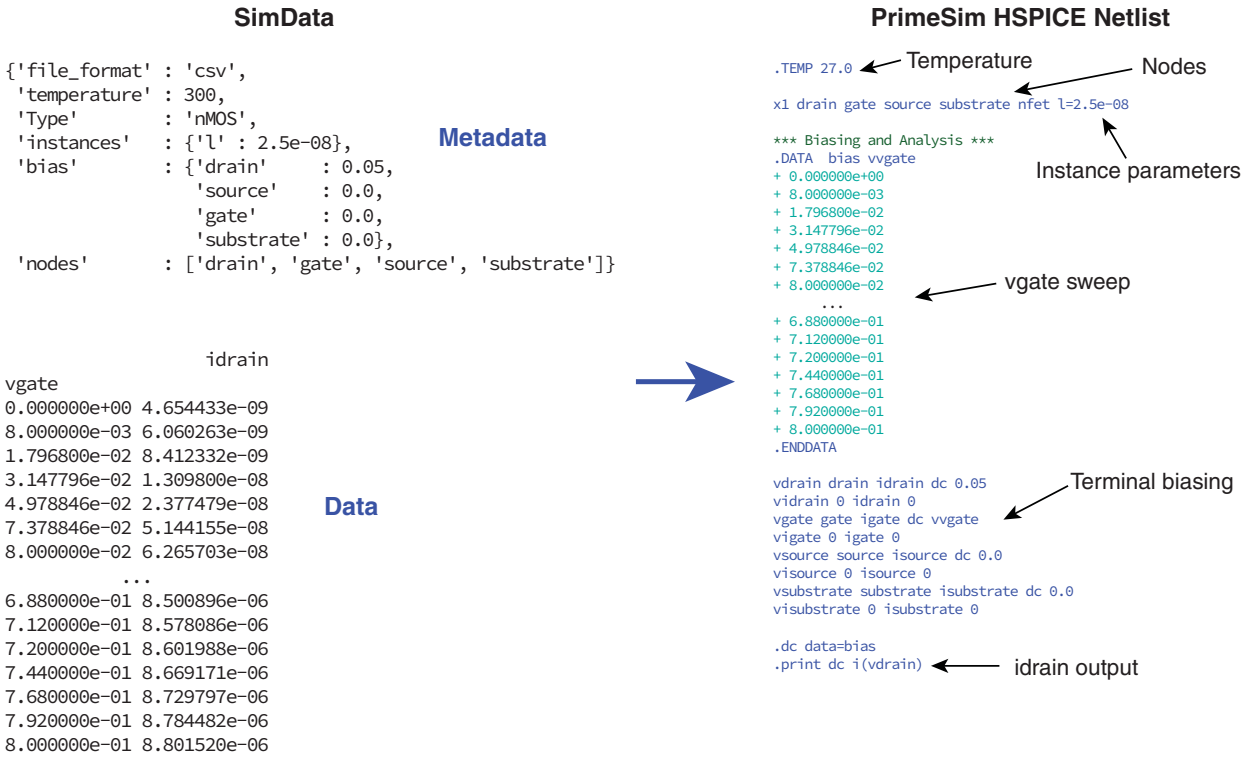



*Table 4 Options of pointwise error method*

| Option | Description | Equation (f=fit, t=target) |
|---|---|---|
| `rel` | (Default) Relative | $\left\|\frac{f-t}{t}\right\|$ |
| `abs` | Absolute | $\|f-t\|$ |
| `log_abs` | Absolute to the log10 data | $\left\|\log_t\left(\frac{f}{t}\right)\right\|$ |
| `log_rel` | Relative to the log10 data | $\left\|\log_{10}\left(\frac{f}{t}\right)\right\|$ |
| `square` | Square | $(f-t)^2$ |

## Multilevel Data Structures

When loading data from the database, multiple datasets of the same simulation type but varying metadata fields can be combined to significantly reduce the runtime of a Mystic extraction. This can be done by adding `combine=True` to any `mdb.Load` command. Consider the following example where there are two target $I_d$–$V_g$ simulations with drain biases of 0.05 V and 0.8 V, respectively:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName)
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName, combine=True)
```

The first line returns one `SimData` object with two underlying datasets separated by drain bias. The second line returns one `SimData` object containing one underlying dataset with a two-level index of drain bias and gate bias. At runtime, the combined dataset will produce only one backend PrimeSim HSPICE netlist per evaluation instead of two, which results in a large runtime reduction.

When the loaded data has more than one varying bias field, such as bulk device simulation data where there are multiple substrate biases and multiple drain biases, that data is combined based on the first varying field that is discovered. Alternatively, a combination field can be specified explicitly using the `extra_index` argument as follows:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName, combine=True,
                extra_index="vdrain")

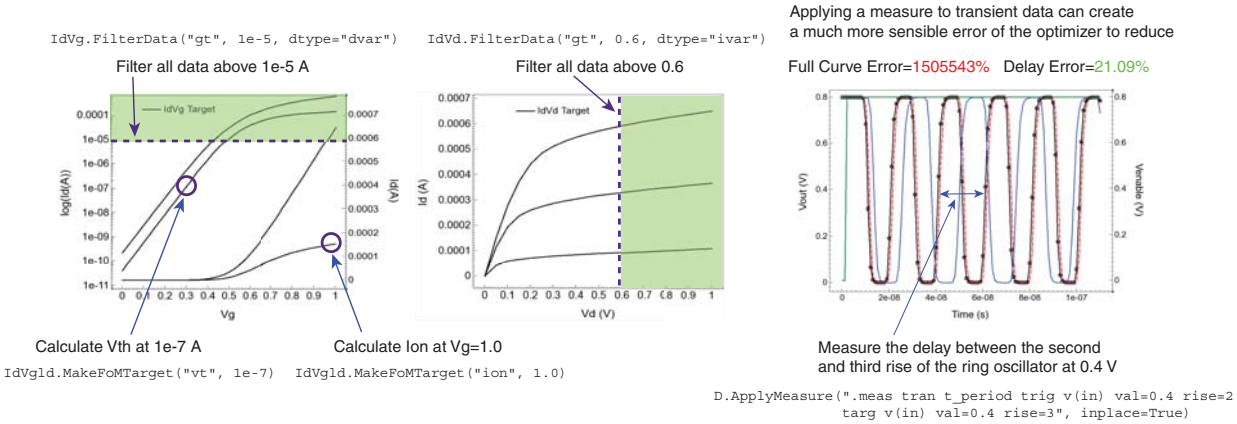
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName, combine=True,
                extra_index="vsubstrate")
```

For best performance increase, data can be combined based on all changing conditions by supplying `extra_index="all"`. This combination covers different biases, temperatures, and instance parameters. It can be the most suitable option when extracting multiple temperatures or geometries simultaneously.

## Filtering

In SPICE model extraction, it is often necessary to target the fitting of a parameter or set of parameters to a specific region of the device-operating characteristics. For example, when fitting the workfunction, only the low-drain subthreshold characteristics are relevant. Fitting parameters against too broad a range of target data can overcomplicate the optimization problem and result in poor optimization performance. To help with this, `SimData` provides a range of Python methods for filtering, resampling, measuring, and calculating device figures of merit as shown in Figure 7.

*Figure 7 Common data types supported by the Mystic tool*




Filters can be applied to any column of the data structure or any bias metadata value using the `SimData.FilterData` method. Any of the standard mathematical operators shown in Table 5 can be applied.

*Table 5 Standard mathematical operators*

| Operator | Description |
|---|---|
| `ge` | Greater than or equal to |
| `gt` | Greater than |
| `lt` | Less than |
| `le` | Less than or equal to |
| `eq` | Equal to |
| `neq` | Not equal to |

The following example illustrates how to filter data in a set of two $I_d$–$V_g$ curves into manageable segments:

```
# Load data from database.
IdVg = mdb.Load(ivar="vgate", dvar="idrain", project="upload-data")

# Split the data by drain bias.
IdVgld = IdVg.FilterData("eq", 0.05, dtype="bias.drain")
IdVghd = IdVg.FilterData("eq", 0.05, dtype="bias.drain")

# Split the low-drain data into below and above Vt.
IdVgld_BelowVt = IdVgld.FilterData("le", 0.3, dtype="ivar")
```

```
IdVgld_AboveVt = IdVgld.FilterData("ge", 0.4, dtype="ivar")

# Split the high-drain data into below and above Vt.
IdVghd_BelowVt = IdVghd.FilterData("le", 0.3, dtype="ivar")
IdVghd_AboveVt = IdVghd.FilterData("ge", 0.4, dtype="ivar")
```

For RF (S-parameter) data, where there might be multiple target columns, you might also want to filter by column name to extract parameters against a single S-parameter value. For example, an S11 target can be created as follows:

```
Y22 = Y_f28.FilterData("eq", ["y(22)re","y(22)i"], dtype="column",
                       mask=True)
```

Filters can also be applied to figure of merit values for each curve. For example, a curve can be filtered below $V_t$ as follows:

```
IdVgld_BelowVt = IdVgld.FilterData("le", "vtgmmax", dtype="ivar)
```

For details, see Figure of Merit Transforms.

## Figure of Merit Transforms

Each of these segments then has its own role in the extraction, matched to SPICE model parameters that influence the behavior in that region.

The `SimData` object also provides a set of figures of merit calculation methods specific to the behavior of MOSFET devices (see Table 6).

*Table 6 Figures of merit calculation methods*

| Method | Description |
|---|---|
| `Derivative` | Derivative of the data. |
| `DIBL` | Drain-induced barrier lowering of the device calculated between the low and high drain IdVg data. The data must contain only two target data curves with different drain biases for the calculation to work. |
| `Gm` | Transconductance of the device at all bias points. |
| `GmMax` | Maximum transconductance of the device. |
| `FMax` | Calculates `FMax` of the device. This figure of merit can be applied only to Y-parameter data or maximum unilateral gain data. |
| `ft` | Calculates the unity gain frequency of the device's short circuit current gain (`ft`). This figure of merit can be applied only to H-parameter data. |

*Table 6 Figures of merit calculation methods (Continued)*

| Method | Description |
|---|---|
| `Ioff` | Off-current of the device, which is calculated at $V_g$ = 0.0 by default. |
| `Ion` | On-current of the device, which is calculated at the maximum $V_g$ point of the data by default. |
| `Rout` | Output resistance of the data. |
| `SubSlope` | Subthreshold slope of the device measured in mV/decade. The default minimum current is the minimum current of the data. The default maximum current is 1 decade above the minimum current. |
| `Vt` | Threshold voltage of device, which should be calculated on low-drain $I_d$–$V_g$ data. The default current criteria is 1e-7 A. |
| `Sum` | Sums the target values across multiple targets. Targets should have the same independent variables, for example, $V_g$, but do not necessarily need to have matching dependent variables. For example, $C_{gd}$ + $C_{gs}$ is possible. |
| `VtGmMax` | Threshold voltage based on the maximum transconductance of the device. The calculated value is the x-intercept of the gradient of the maximum transconductance. |

Josephson junction I–V targets have a set of custom figures of merit specific to those devices described in Figure 8 and Table 7.

*Figure 8 Typical Josephson junction I–V curve shape (red) with key figures of merit: P1–P5, rnorm, and rsg*

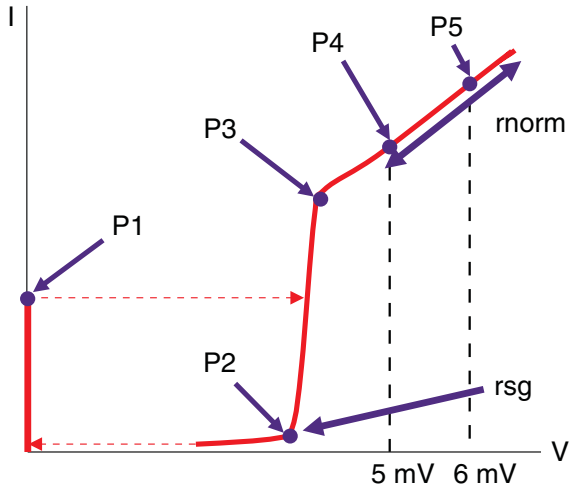



*Table 7 Methods for Josephson junctions*

| Method | Description |
|---|---|
| P1 | Critical current of the device. |
| P2 | Lower voltage in the gap voltage slope. The gap voltage can be calculated by adding P2 and P3 and dividing by 2. |
| P3 | Upper voltage in the gap voltage slope. |
| P4 | Junction voltage at vmax – 1 mV. It can be used to manually calculate the normal resistance. |
| P5 | Highest junction voltage present in the data, vmax. |
| rnorm | Normal resistance of the device. |
| rsg | Subgap resistance of the device, calculated at the bottom of the gap voltage slope. |

For details about Josephson junction SPICE model extraction, see Josephson Junction Extraction on page 75.

**Note:**

This functionality is not included in the main Mystic license and requires an add-on license to use. For more information, contact your local TCAD support team.

Each figure of merit (FoM) has its own method. For example:

```
Vt = IdVgld.VtGmmax()
```

In this case, `Vt` is a single float that can be used as a threshold for filtering. Figures of merit that require an argument, for example, $V_t$ current criteria, are initialized with sensible defaults. These can be changed using the `SimData.SetFoMCriteria` method. It is also possible to create an extraction target from a figure of merit using the `SimData.MakeFoMTarget` method as follows:

```
Vt = IdVgld.MakeFoMTarget("vt", 1e-7)
```

In this case, `Vt` is a new `SimData` object that can be used as usual in extraction steps. The data of the root object (`IdVgld`) is used to create and run the PrimeSim HSPICE simulation, then the FoM transform is applied to both the target and fit columns as a postprocessing task. When used in an extraction step, the single error between the target and fitted FoM values will be used instead of the pointwise error between the curves.

In addition, you can define your own FoM transforms and apply them to any `SimData` object. The transform can be applied to all data points, such as a log transform or multiplier, or it can aggregate the data to a single value, such as threshold voltage.

The following example shows how to apply a log10 transform to all data:

```
import numpy as np

IdVg = mdb.Load(ivar="vgate", dvar="idrain")

def log10(data):
     return np.log10(data)

IdVg = IdVg.MakeFoMTarget(log10)

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain")
IdVgld_BelowVt = IdVgld("lt", 0.3, "ivar")
IdVgld_AboveVt = IdVgld("gt", 0.4, "ivar")

IdVghd = IdVg.FilterData("eq", 0.05, "vdrain")
IdVghd_BelowVt = IdVghd("lt", 0.3, "ivar")
IdVghd_AboveVt = IdVghd("gt", 0.4, "ivar")
```

Your function should accept a data argument that is the internal `Data` object to which the transform is applied. For a detailed description of this class, see Sentaurus Workbench API Documentation on page 10. As the transform is applied to the root $I_d$–$V_g$ object, all objects derived from that using `SimData.FilterData` will also have the log transform applied to them.

The following example shows how to apply a mean aggregation transform:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

def mean(data, col="target"):
```

```
    if col == "target":
       return data._target_values.mean()
    elif col == "fit":
       return data._fit_values.mean()

IdVg_mean = IdVg.MakeFoMTarget(mean, add_as_fom=True)
```

The `add_as_fom` argument signals that the function is an aggregation that should be added to the `Data` object API. In this scenario, the function must accept a column argument that selects whether the aggregation is applied to the target data or fit data.

FoM transforms can also be chained together where appropriate. Consider this example that extracts against the log10 transform of the device transconductance:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain")
Gmld = IdVgld.MakeFoMTarget("gm")

def log10(data):
   return np.log10(data)

Gmld_log = Gmld.MakeFoMTarget(log)
```

## Data Masks

When filtering a `SimData` object in the Mystic tool using the `SimData.FilterData` method, a copy of the object is created with the filter criteria applied. The copy can be replaced with a mask that retains a reference to the root object. When a filtered object is used in a `ExtractionUtils.PrintErrors` or `ExtractionUtils.DoStage` call, the root object is simulated instead of the filtered object. This means that when two or more `SimData` objects with the same root object are passed in to a `ExtractionUtils.DoStage` optimization, only one PrimeSim HSPICE simulation is required per evaluation. This significantly reduces the runtime of a Mystic extraction.

Data masks can be activated globally through the `Data` class as follows:

```
from libfromage.data import Data
Data.set_options(data_mask=True)
```

They can also be activated on a case-by-case basis using the `mask` argument of `FilterData`:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain", mask=True)
IdVgld_BelowVt = IdVgld.FilterData("lt", 0.3, "ivar", mask=True)
IdVgld_AboveVt = IdVgld.FilterData("gt", 0.4, "ivar", mask=True)

IdVghd = IdVg.FilterData("eq", 0.05, "vdrain", mask=True)
```

```
IdVghd_BelowVt = IdVghd.FilterData("lt", 0.3, "ivar", mask=True)
IdVghd_AboveVt = IdVghd.FilterData("gt", 0.4, "ivar", mask=True)
```

Masks can also be applied when creating FoM targets using `MakeFoMTarget`. As with filters, the root object is always simulated, meaning multiple FoMs can be evaluated in a single PrimeSim HSPICE simulation.

Consider the following example:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain")

IdVgld = IdVg.FilterData("eq", 0.05, "vdrain")
IdVgld_AboveVt = IdVgld.FilterData("gt", 0.4, "ivar", mask=True)

Ioff       = IdVgld.MakeFoMTarget("ioff")
VtGmMax    = IdVgld.MakeFoMTarget("vtgmmax")
Gm_AboveVt = IdVgld_AboveVt.MakeFoMTarget("gm")
Ion        = IdVgld.MakeFoMTarget("ion")

phig = OptParam('PHIG', 4.4,   4.2,   4.6,   1.0)
cdsc = OptParam('CDSC', 0.001, 0.0,   10.0,  1.0)
u0   = OptParam('U0',   0.02,  0.004, 1.0,   1.0)
ua   = OptParam('UA',   0.3,   0.0,   100.0, 1.0)
eu   = OptParam('EU',   2.5,   0.1,   10.0,  1.0)

params = [phig,cdsc,u0,ua,eu]

ExtractionUtils.DoStage("FOM", Model, Simulator, params, Optimizer,
                        [Ioff,VtGmMax,Gm_AboveVt,Ion])
```

This example is trying to extract against four FoMs that all come from the same root target data. Previously, four PrimeSim HSPICE simulations would have been required per evaluation. Using masks, only one simulation is required.

## Generic PrimeSim HSPICE .MEASURE Commands

More generic transforms can also be applied to `SimData` objects using the PrimeSim HSPICE `.MEASURE` syntax. PrimeSim HSPICE measures can be used in the same way as figures of merit to calculate the electrical characteristics of a simulation, but they give you more freedom to define and apply your own transforms. In the Mystic tool, these can be applied using the `SimData.ApplyMeasure` method. When the data is used in a parameter optimization, the `.MEASURE` command is first post-applied to the target data and then added to each runtime netlist to obtain a fitted value from the PrimeSim HSPICE simulation. The root mean square (RMS) error of all measured values is then minimized by the optimizer.

The following is an example of calculating the $V_t$ and $I_{on}$ of an $I_d$–$V_g$ target using `.MEASURE` commands:

```
# Create a Vt target.
```

```
Vt = IdVg.ApplyMeasure('.MEASURE DC vt FIND v(g) when i(vd)=-1e-7')

# Create an Ion target.
Ion = IdVg.ApplyMeasure('.MEASURE DC ion MAX i(vd)')
```

As with `MakeFoMTarget`, `SimData` objects are returned from `SimData.ApplyMeasure` and can be used as usual throughout the extraction. At any point in the extraction, the contents of a `SimData` object can be written to disk as either a PLT or CSV file, using the `SimData.WritePLTFile` and `SimData.WriteCSVFile` methods. The created files can then be plotted using Sentaurus Visual.

All bias terminal names referenced in the `.MEASURE` command must match terminal names attached to the data. For example, if a data object has the terminal names `drain`, `gate`, `source`, and `substrate`, then the following `.MEASURE` command will fail at runtime because the terminal name `d` does not exist:

```
Ioff = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vd)")
```

The following would correctly measure $I_{off}$ for the low-drain $I_d$–$V_g$ curve:

```
Ioff = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vdrain)")
```

You can test `.MEASURE` commands before using them in an optimization by utilizing the `TestMeasure()` method as follows:

```
Ioff = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vdrain)")
Ioff.TestMeasure(Simulator)
```

Multiple `.MEASURE` commands can also be applied to the same `SimData` object using the `inplace` argument. For example, $I_{off}$ and $V_t$ of a low-drain $I_d$–$V_g$ simulation can be optimized simultaneously by specifying:

```
IoffVt = IdVgld.ApplyMeasure(".MEAS DC ioff MIN i(vdrain)")
IoffVt.ApplyMeasure(".MEAS DC vt FIND v(gate) WHEN i(vdrain)=1e-7",
                    inplace=True)
```

The error in both measured quantities will be passed to the optimizer during optimization. You can apply `.MEASURE` commands to DC, AC, or transient data, provided they are valid based on the simulation conditions of the data. For more information about how to construct a `.MEASURE` command, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

## Bias Extension

After model extraction, you might want to simulate your model at bias conditions outside of the extraction range. This can be done using `SimData.ExtendBias`.

Consider an extraction where the target data consists of two $I_d$–$V_g$ curves at $V_d$=0.05 V and 0.8 V, respectively, with a range of $V_g$=0.0 V – 0.8 V, and three $I_d$–$V_d$ curves at $V_g$=0.4 V, 0.6 V, and 0.8 V, with a range of $V_d$=0.0 V – 0.8 V:

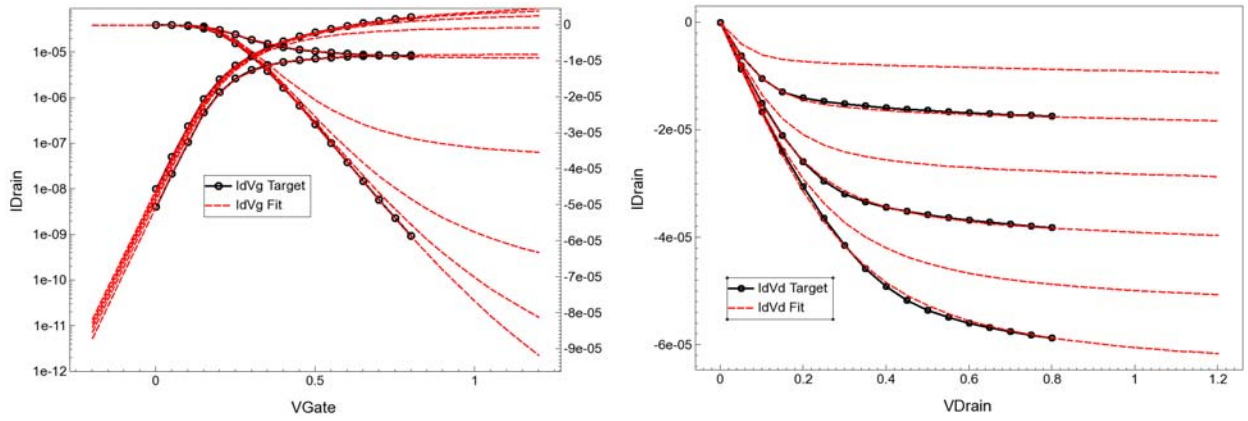
```
# Extend the bias ranges of the data
IdVg_ext = IdVg.ExtendBias(ivar_min=-0.2, ivar_max=1.2, spacing=0.05,
                           vdrain=[0.2,0.4,0.6])
IdVd_ext = IdVd.ExtendBias(ivar_max=1.2, spacing=0.05,
                           vgate=[0.3,0.5,0.7])

# Simulate the model at the extended ranges
ExtractionUtils.PrintErrors("idvg-ext", IdVg_ext, Simulator, Model,
                            "rmsd")
ExtractionUtils.PrintErrors("idvd-ext", IdVd_ext, Simulator, Model,
                            "rmsd")

# Write the extended data to a PLT file
IdVg_ext.WritePLTFile('idvg-ext')
IdVd_ext.WritePLTFile('idvg-ext')
```

As can be seen in Figure 9, the data range has extended to the bounds specified in `SimData.ExtendBias` and new data curves have been simulated for the intermediate drain and gate biases.

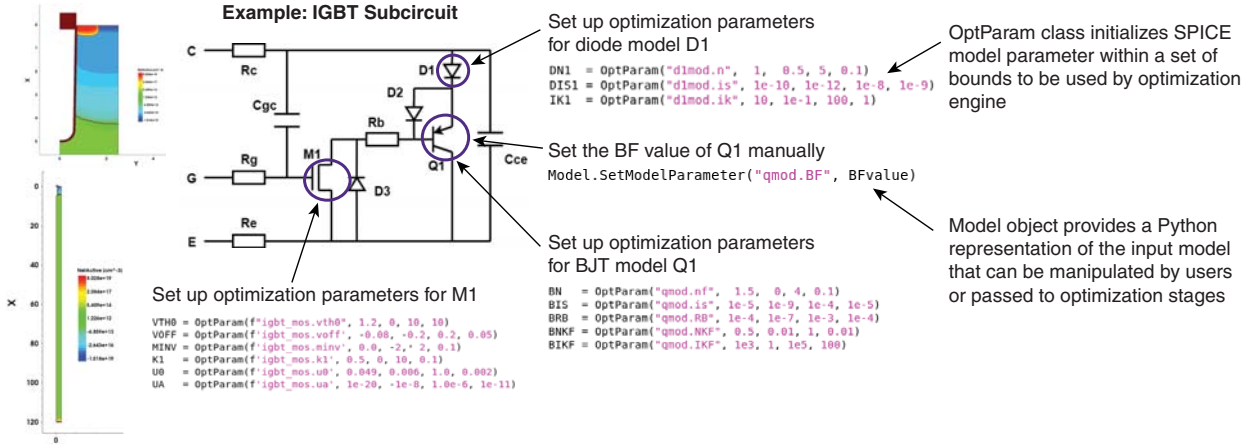
*Figure 9 Target and fitted I–V data after the bias extension*




## SPICE Models

The Mystic tool supports any netlist that is compatible with the PrimeSim HSPICE tool for extraction. This can be anything from a single model card to a complex configuration of circuit components, including subcircuits and `.INCLUDE` commands. Figure 10 shows an example of a subcircuit model used for the extraction of an IGBT power device.

*Figure 10 Example of an IGBT subcircuit model*




When a model card is supplied on the command line using the `-m` option, it is parsed by the Mystic tool and injected into the extraction environment as a `Model` object. The `Model` object contains a hierarchical representation of the model definition in Python and has an API for manipulating the SPICE model parameter values at any level of the netlist structure using a dot-separated syntax. For example, in the netlist shown in Figure 10, if you want to set the Beta factor (BF) for the BJT model (QMOD) to 100, then you can add the following line to your Mystic strategy:

```
Model.SetModelParameter("QMOD.BF", 100)
```

Conversely, the current value of any model parameter can be retrieved from the netlist using:

```
BF = Model.GetModelParameter("QMOD.BF")
```

The primary objective of a Mystic extraction strategy is to find the set of SPICE model parameter values that ensure the best fit between the provided target data and the output of a PrimeSim HSPICE simulation under matching simulation conditions. To achieve this, the Mystic tool can control the model parameter values during optimization, which is done through the `OptParam` class. For more information, see Optimization Parameters on page 39.

For more information about `OptParam`, see Optimization Algorithms on page 42. At any point in the strategy, the current state of `Model` can be written to disk using `Model.write` or pushed to the database using `mdb.StoreFitParameters` or `mdb.StoreFitData`.

The model can also be checkpointed at any point in the strategy using the `set_checkpoint()` method as follows:

```
Model.set_checkpoint("Stage1")
```

The supplied argument is a label that you can use to retrieve the state of the model at a later time. This functionality is most frequently used in interactive mode when developing the extraction methodology.

Previously checkpointed states of the model can be recovered using the `get_checkpoint()` method as follows:

```
Model.get_checkpoint("Stage1", inplace=True)
```

If `inplace=True`, then the model state is applied to the existing model object. If `inplace=False`, then a copy of the model state is returned from the method.

When running an extraction strategy, automated checkpoints are also created whenever there is a call to `ExtractionUtils.DoStage`. The label used for these checkpoints is the label passed as the first argument in `ExtractionUtils.DoStage`. This effectively allows you to undo the action of one or more optimization stages.

The following example returns the model to the state it had before executing `Stage1`:

```
ExtractionUtils.DoStage("Stage1", Model, Simulator,
                        [PHIG,CDSC,QMFACTOR], Optimizer,
                        [IdVgld_BelowVt])

Model.get_checkpoint("Stage1", inplace=True)
```

## Simulator

The Mystic tool uses the PrimeSim HSPICE tool as a backend simulation engine. Before running the Mystic tool, you must ensure you have a valid PrimeSim HSPICE binary in your environment path or supply the full path to one using the `-l` option.

The Mystic `Simulator` object provides functionality for writing netlists based on the supplied `Model` and `SimData` objects, running PrimeSim HSPICE simulations, parsing the outputs of those simulations, and comparing the results to the target data. In a standard extraction strategy, these operations are all handled automatically by the Mystic tool inside the utility functions provided by `ExtractionUtils` so your only interaction with the simulator will be to set command-line options and to configure the global simulation options. The `Simulator` object is configured with the default PrimeSim HSPICE options listed in Table 8.

*Table 8 PrimeSim HSPICE options and default values*

| Option | Default | Description |
|---|---|---|
| ABSTOL | 1e-15 | Absolute error tolerance for branch currents in DC and transient analysis. |
| EXT_OP | 1 | Allows additional information output in the PrimeSim HSPICE tool. |

*Table 8 PrimeSim HSPICE options and default values (Continued)*

| Option | Default | Description |
|---|---|---|
| GMINDC | 1e-18 | Specifies minimum transconductance in parallel for p-n junctions and MOSFET nodes in DC analysis. |
| INGOLD | 2 | Controls whether the PrimeSim HSPICE tool prints output in exponential form. Default is exponential. |
| MEASFORM | 1 | Allows writing of measurement output files to Excel or HSIM formats as well as traditional PrimeSim HSPICE formats. |
| MEASDGT | 8 | Controls the value precision in measure files. |
| NOMOD | 1 | Suppresses the printing of model parameters in the output. |
| NUMDGT | 8 | Controls the listing printout accuracy. |

These options are added to all backend netlists as `.OPTION` commands and can be altered using the `Simulator.set_options` method.

From a PrimeSim HSPICE licensing perspective, the Mystic tool can run simulations in different modes:

- In *normal mode*, a PrimeSim HSPICE license is checked out, utilized, and then checked back in for every simulation. Depending on the license checkout time and the number of simulations required in an extraction, this might be very inefficient.
- In *server mode*, the Mystic tool checks out and holds a PrimeSim HSPICE server license at the beginning of the extraction and then continually uses it throughout. To activate the PrimeSim HSPICE server mode, add a `-C` option to `Simulator.spiceargs` as follows:

```
Simulator.spiceargs.append('-C')
```

A single PrimeSim HSPICE simulation can be run from any `SimData` object using the `SimData.GenerateData` method as follows:

```
IdVg.GenerateData(Simulator, Model)
```

The `Simulator` and `Model` are passed in as arguments to construct the netlist and run the backend PrimeSim HSPICE simulation. On completion of the simulation, the fit column of $I_d$–$V_g$ is updated with the result of the simulation, and the pointwise error column is updated with the pointwise error between the target and fit curves. When running inside an optimization, this error vector is passed back to the optimization algorithm.

In a standard extraction strategy, the `Simulator` object is most commonly used as an argument to the utility functions `ExtractionUtils.PrintErrors` and

`ExtractionUtils.DoStage`, and is rarely used standalone. For more information about these functions, see Full Strategy on page 52.

## Parallel PrimeSim HSPICE Simulations

The Mystic tool extracts the values of SPICE model parameters by continually running PrimeSim HSPICE simulations and comparing their results with target data coming from TCAD or measurement. For targets that have more than one dataset, for example, low-drain and high-drain $I_d$–$V_g$ data, multiple PrimeSim HSPICE simulations are required to complete one evaluation.

The Mystic PrimeSim HSPICE library allows these simulations to be executed in parallel. This can significantly reduce the runtime of complex extraction strategies.

The maximum number of parallel PrimeSim HSPICE simulations is controlled by the property `Simulator.max_sims` as follows:

```
Simulator.max_sims = 10
```

The default value for this property is 1, that is, serial simulation. The objective function of the `Optimizer` object has also been parallelized where possible, particularly in the Jacobian calculation of the commonly used bounded trust region algorithm. The Jacobian evaluation makes small perturbations in each of the parameters being optimized and uses the results to determine a gradient of best descent for the problem.

Consider an optimization problem with five parameters and 2 bits of target data. The number of PrimeSim HSPICE simulations required for a Jacobian evaluation is 5 parameters × 2 samples per parameter × 2 targets = 20 simulations. As these simulations have no dependence on each other, they can all be performed in parallel. To activate this feature, pass `parallel=True` to any call to `ExtractionUtils.DoStage` as follows:

```
ExtractionUtils.DoStage("BelowVt", Model, Simulator,
                        [PHIG,CDSC,QMFACTOR], Optimizer,
                        [IdVgld_BelowVt], parallel=True)
```

This can also be set at a global level using the `ExtractionUtils.settings.parallel` flag.

**Note:**

Running PrimeSim HSPICE simulations in parallel requires multiple PrimeSim HSPICE licenses. If you are limited by PrimeSim HSPICE licenses, then you should run PrimeSim HSPICE in server mode to avoid license failures in Mystic. For example:

```
Simulator.spiceargs = ["-C"]
```

With the PrimeSim HSPICE server mode and parallel simulations activated, a queue of PrimeSim HSPICE server licenses is created dynamically when needed, then retained for

the duration of the extraction. This is a more reliable way to limit the number of licenses required when running in parallel to the exact number specified by `Simulator.max_sims`.

## Optimization Parameters

The core of a successful SPICE model extraction strategy is a set of parameters that can be varied to fit the data. Mystic provides the `OptParam` class for defining and constraining these parameter values. `OptParam` allows you to set an initials value (if the parameter does not already exist in the model), upper and lower bounds, scaling factors and transforms for the set of parameters you want to optimize. The naming conventions here are the same as for `Model.GetModelParameter` references previously, with a dot separating the levels of hierarchy in the netlist. For example, to initialize the Beta factor (BF) of the BJT (QMOD) for optimization in the IGBT device, use:

```
# name, default, lower bound, upper bound, scale factor BF =
OptParam("QMOD.BF", 100.0, 1.0, 1000.0, 1.0)
```

The scaling factor can be used to help control the influence the parameter has in the optimization when using gradient descent-based algorithms. Before calculating the parameter step for the Jacobian calculation, the current value of the parameter is divided by the scaling factor. This can help normalize the parameters and balance the optimization. It is particularly useful for parameters with very small values, for example, capacitance parameters in the order of 1e-16, or parameters which are initialized far below their expected final value. Here is an example of a capacitance parameter using an appropriate scaling factor:

```
cfs = OptParam(“mosmod.cfs”, 1e-11, 1e-15, 1e-8, 1e-11)
```

Each SPICE model parameter has a different impact on the data against which it is being optimized. Some parameters have a linear response, whereas others can be highly nonlinear. The performance of gradient descent-based optimization routines is best when parameters vary close to linearly against the fit error. To help with this, transforms can be applied to each of the `OptParam` parameters in place of the scaling factor. For example:

```
cfs = OptParam(“mosmod.cfs”, 1e-11, 1e-15, 1e-8, “decimal-log”)
```

Instead of scaling, the parameter is now be transformed before the steps are calculated in the Jacobian calculation, then inverse transformed to run the simulations. In this case:

$$x0 = 1e^{-11}$$

$$x1 = 1e^{-11} + (1e^{-11} \times 1e^{-3})$$

$$x1 = 1.001e^{-11}$$

Become:

$$x0 = 1e^{-11}$$

$$x0t = np\log 10(1e^{-11})$$

$$x0t = -11$$

$$x1t = -11 + (-11 \times 1e^{-3})$$

$$x1t = -11.011$$

$$x1t = 10^{-11.011}$$

$$x1 = 9.75e^{-12}$$

The available transforms are listed in following table.

| Transform | Description |
|---|---|
| `'scale'` | Applies a scalar division to the parameter value. Equivalent to using a scaling factor directly. |
| `'shift'` | Adds or subtracts from the parameter value. Useful to move parameters away from a zero starting value, which is often difficult for an optimization routine to converge from. |
| `'log'` | Applies a natural logarithm transform to the parameter value. |
| `'decimal-log'` | Applies a log 10 transform to the parameter value. |
| `'log-exp'` | Applies a logarithmic-exponential transform to the parameter from below. It maps `[lower, inf)` to `(-inf, inf)`. |
| `'exp'` | Applies an exponential transform to the parameter value. |

| Transform | Description |
|---|---|
| `'decimal-exp'` | Applies a decimal exponential transform to the parameter value, that is, 10 ^ value. |
| `'dual-log1p'` | Applies a log transform to parameters with positive and negative values. Parameters around zero are approximately linearly transformed. |
| `'square'` | Applies a square transform to the parameter value. |
| `'square-root'` | Applies a square root transform to the parameter value. |
| `'power'` | Applies a power transform to the parameter value. Square and square root are examples of specific power transforms, for example, 2 and 0.5. |

## Optimal Parameter Transforms: ExtractionUtils.GetOptimalParameterTransform

In the `ExtractionUtils` model, there is also a helper function to calculate the best transforms for a given set of SPICE model parameters. The best transform is defined as the transform which provides the most linear response against the data. The function can be used as follows:

```
PHIG = OptParam('PHIG', 4.3, 4.25, 4.35, 1.0)
CDSC = OptParam('CDSC', 7e-3, 1e-5, 1.0, 1.0)
QMFACTOR = OptParam('QMFACTOR', 1.0, 1e-5, 100, 1.0)

tphig,tcdsc,tqmfactor = ExtractionUtils.GetOptimalTransforms(
 [PHIG,CDSC,QMFACTOR], IsVgld_BelowVt,
 Simulator, Model, step=0.01
)

PHIG = OptParam('PHIG', 4.3, 4.25, 4.35, tphig)
CDSC = OptParam('CDSC', 7e-3, 1e-5, 1.0, tcdsc)
QMFACTOR = OptParam('QMFACTOR', 1.0, 1e-5, 100, tqmfactor)
```

## Parameter Space Exploration

Before extracting a SPICE model parameter set in the Mystic tool, it is important to understand how each parameter in your chosen model affects the target data curves against which you are extracting. To help with this, Mystic provides the following utility functions to explore the parameter space.

The Mystic graphical user interface also provides functionality to explore the parameter space interactively. See Exploring Parameters Interactively on page 104 for details.

### Sensitivity Analysis: ExtractionUtils.SensitivityAnalysis

One-factor-at-a-time sensitivity analysis aims at understanding the response of the underlying model as a function of small changes to a single parameter, with all other parameters fixed. It reveals only the local gradient of the response surface of the model with respect to a given parameter.

This local sensitivity analysis can be performed in the Mystic tool by using the `ExtractionUtils.SensitivityAnalysis` utility function as shown here:

```
IdVg_belowvt = IdVg.FilterData("le", 0.4)
IdVg_belowvt = IdVg.FilterData("gt", 0.4)

PHIG     = OptParam("PHIG", 4.5, 4.2, 4.7, 1.0)
CDSC     = OptParam("CDSC", 0.1, 0.0, 10.0, 1.0)
U0       = OptParam("U0",0.02,0.006,1.0,0.02)
UA       = OptParam("UA",0.3,0.0,100.0,1.0)

sense_df = SensitivityAnalysis([IdVg_belowvt, IdVg_abovevt],
                               Simulator, Model, [PHIG,CDSC,U0,UA])
```

The method can take any number of targets and parameters, and displays the sensitivity for each target individually and the combined root mean square deviation (RMSD) of the targets.

### Parameter Sweep: ExtractionUtils.ParamSweep

You can perform a parameter sweep of a single parameter against a single target by using the `ExtractionUtils.ParamSweep` utility function, which is a simpler version of the utility function `ExtractionUtils.SensitivityAnalysis` and produces one PLT file for each evaluation of the target at a particular parameter value. The following example shows a workfunction parameter sweep using the BSIM-CMG parameter `PHIG`:

```
ExtractionUtils.ParamSweep("idvg-ld", IdVgld, Simulator, Model,
                           PHIG, "rmsd")
```

The default number of parameters in the sweep is 11, and the default upper and lower bounds are taken from the PHIG `OptParam` definition. These can be overwritten using the `num`, `pmin`, and `pmax` function arguments, respectively.

## Optimization Algorithms

The core of the Mystic tool is a powerful optimization library with several algorithms to help optimize SPICE model parameter values to fit the target data provided. The `Optimizer`

object is injected into the environment at runtime and provides an interface to the underlying library.

*Table 9 Supported optimization algorithms*

| Algorithm | Description |
|---|---|
| `BOUNDED_TRUST_REGION` | Use Intel MKL bounded trust region least squares solver. |
| `TRUST_REGION` | Use Intel MKL trust region least squares solver. |
| `least_squares` | Use least squares solvers. Available internal methods are trust region, dogbox, and Levenberg–Marquardt. |
| `minimize` | Use minimizers. A cost function to aggregate the residuals must be given in this case (root mean square error by default). Available minimizers include trust region, BFGS (default), COBYLA, and L-BFGS-B. |
| `differential_evolution` | Use a differential evolution algorithm to find the global minimum of the multivariate function. |
| `doe` | Provides access to the Bayesian search-based optimization method. |

The initial algorithm of the `Optimizer` is dictated by the `-o` option. If not supplied, then it defaults to `BOUNDED_TRUST_REGION`, which is used most frequently throughout Mystic extractions. Inside the strategy, the algorithm can be changed using `Optimizer.SetMethod` as follows:

```
Optimizer.SetMethod("least_squares")
```

Each algorithm has a set of parameters that can be used to adjust tolerances, the method, the initial conditions, and so on. A description of these parameters for the available algorithms can be found in the Optimizer/Optimization Parameter section of the API documentation. These parameters can be set using the `Optimizer.SetOptimizationParameter` method. The following code snippet illustrates some recommended settings for the `BOUNDED_TRUST_REGION` algorithm:

```
Optimizer.SetMethod("BOUNDED_TRUST_REGION")
Optimizer.SetOptimizationParameter("eps1", 1e-3)
Optimizer.SetOptimizationParameter("eps2", 1e-3)
Optimizer.SetOptimizationParameter("eps3", 1e-3)
Optimizer.SetOptimizationParameter("eps4", 1e-3)
Optimizer.SetOptimizationParameter("eps5", 1e-3)
Optimizer.SetOptimizationParameter("eps6", 1e-3)
Optimizer.SetOptimizationParameter("jac", True)
Optimizer.SetOptimizationParameter("kwargs", {"eps": 1e-3,
                                              "step_type": "abs",
                                              "step_method": "cd",
                                              "eps_fac": 1e-6})
```

The recommended settings for the `least_squares` algorithm are as follows:

```
Optimizer.SetMethod(“least_squares”)
Optimizer.SetOptimizationParameter("method", "trf")
Optimizer.SetOptimizationParameter("ftol", 1e-4)
Optimizer.SetOptimizationParameter("xtol", 1e-6)
Optimizer.SetOptimizationParameter("gtol", 1e-4)
Optimizer.SetOptimizationParameter("diff_step", 1e-3)
```

For more information on the settings that are available for specific algorithms, refer to the Mystic API documentation.

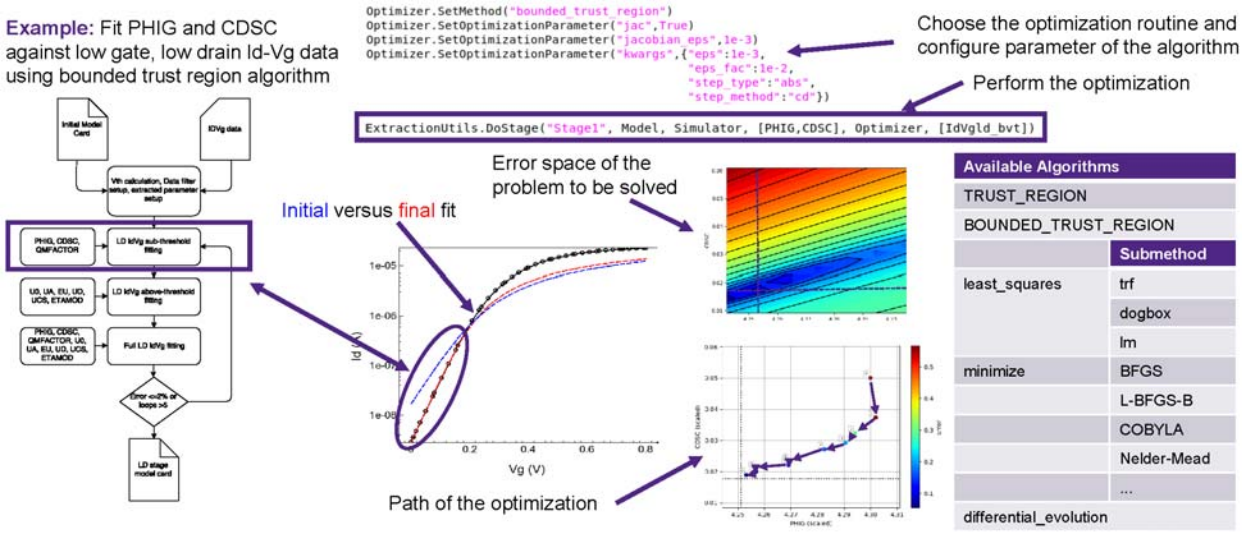
As mentioned in SPICE Models, `OptParam` objects are created for all parameters that will be optimized. How these are used varies depending on the optimization algorithm. For example, if `BOUNDED_TRUST_REGION` is used, then the parameter value will remain strictly within the bounds defined by the `OptParam` object during optimization. However, if `TRUST_REGION` is used, then these bounds will be ignored.

The last argument in the `OptParam` definition is a scale factor that can play a very important role in the convergence of an optimization, particularly with gradient descent–based methods. The idea of the scale factor is to balance the sensitivities of a set of parameters being used in the optimization to prevent one parameter dominating another parameter. A sensible default value for the scale factor is the initial parameter value. It can then be increased to add more sensitivity or decreased to reduce the sensitivity. To better understand the sensitivity of a set of parameters against an extraction target, you can use the utility function `ExtractionUtils.SensitivityAnalysis`.

As with the `Simulator`, the `Optimizer` has limited use in the extraction strategy as a standalone object beyond algorithm selection and parameter setup. It is most commonly

used in combination with other objects in the `ExtractionUtils.DoStage` utility function, as shown in Figure 11.

*Figure 11 Overview of Mystic optimization engine*




The `ExtractionUtils.DoStage` utility function takes the `Model`, the `Simulator`, a list of `OptParam`, the `Optimizer`, and a list of `SimData` objects, and combines them to perform a single optimization step within a strategy. After a stage completes, the optimization result is written to screen or to the Mystic log file, the optimal parameter values are retained in the `Model` object, and the optimal simulation results are stored in the `SimData` objects.

## Bayesian Optimization Algorithm

The Mystic optimization library provides a Bayesian optimization algorithm. This algorithm performs an adaptive search of the parameter space guided by current available knowledge from all the previously performed simulations. It starts with Latin hypercube sampling to construct an initial Gaussian process response surface model. In each iteration, an acquisition function (expected improvement) is evaluated, and new points are selected that could potentially improve on the current best solution.

To use the Bayesian optimization algorithm in Mystic, add the following to your extraction strategy:

```
Optimizer.set_method("doe")
Optimizer.set_optimization_parameters(method="bayesian_optimizer")
```

A list of other options for the Bayesian optimizer can be found in the Optimizer/Optimization Parameter section of the API documentation (see Mystic API Documentation on page 9).

## Machine Learning in the Mystic Tool

Machine Learning (ML) is set to play an important role in improving the efficiency and performance of SPICE model extraction in the future. This section describes some of the ML and ML-adjacent techniques leveraged in the Mystic tool to improve the SPICE model extraction process.

## Data Storage and Retrieval

When the Mystic tool executes, it is continually evaluating PrimeSim HSPICE simulations at a particular state of the user's target model. The results of these simulations contain valuable information about how the behavior of the model responds to changes in parameter values. In the normal execution mode, these simulation results are used to determine the convergence of the optimization algorithm and immediately discarded. The base of any good machine learning model is a broad training set. The Mystic tool provides the capability to store all its simulation results in its database and organize them by the SPICE base model, parameter values, and data type. This data can form the training set for complex ML based models that can accelerate repeating SPICE model extractions for a given technology.

Data storage can be activated when storing the initial model in the database for the first time through `mdb.StoreInitialModel` using the `spice_ml_name` flag:

```
mdb.StoreInitialModel(Model, spice_ml_name="<my-nmos>")
```

where `<my-nmos>` is a label supplied by the user.

After the base model has been stored, all evaluations executed by the Mystic tool are stored in the database along with the parameter values that differ from the base model configuration.

In addition to providing a data set for training ML models, the data storage framework can provide a few advantages over traditional extraction methodologies, described in the next sub-section.

## Rerunning a Strategy

When developing a strategy, it is common to build the optimization steps iteratively, making small adjustments to parameter sets and rerunning the strategy regularly until it gives the desired results. During reruns, the strategy must move through all steps sequentially, even if these steps have been run many times earlier. This can be time consuming, especially for more complex strategies with many steps. When ML data storage is activated, simulation results that have been executed previously can be retrieved directly from the database instead of rerunning the simulation. This can result in steps that have been executed previously under identical conditions to run in a fraction of the time taken for the initial

execution and can help to significantly speed up the strategy development process. The following example illustrates a simple re-executed strategy:

```
ExtractionUtils.DoStage(
     'Step1', Model, Simulator, [PHIG,QMFACTOR,CDSC],
     Optimiser, [IsVgld_BelowVt]
)
ExtractionUtils.DoStage(
     'Step2', Model, Simulator,
     [U0,UA,EU,UD,UCS,ETAMOB],
     Optimiser, [IsVgld_AboveVt]
)
```

Results of the first execution of the strategy are as follows:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.2822, QMFACTOR : 4.3788e-06,
             CDSC : 0.26308
iterations = 7
fit error  = 16.4240
converged  = 1
method     = bounded_trust_region
wallclock  = 5.88s
message    = 2-norm of trial step < eps[3]

Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.01898, UA : 0.4321, EU : 1.4205,
             UD : 0.01552, UCS : 1.0577, ETAMOB : 4.9028
iterations = 17
fit error  = 1.5499
converged  = 1
method     = bounded_trust_region
wallclock  = 24.11s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 < eps[4]
```

The total run time of the extraction is ~30 s. If the strategy is then rerun from scratch in a new Mystic process, the results are as follows:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.2822, QMFACTOR : 4.3788e-06,
             CDSC : 0.26308
iterations = 7
fit error  = 16.4240
converged  = 1
method     = bounded_trust_region
wallclock  = 0.26s
message    = 2-norm of trial step < eps[3]
```

```
Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.01898, UA : 0.4321, EU : 1.4205,
             UD : 0.01552, UCS : 1.0577, ETAMOB : 4.9028
iterations = 17
fit error  = 1.5499
converged  = 1
method     = bounded_trust_region
wallclock  = 1.15s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 <eps[4]
```

The previous simulation results are now being retrieved from the database and the total run time reduces to ~1.4 s while the results remain identical. It is important to understand that if changes are made to any extraction steps (for example, adding a new parameter to the set or changing the target data), then the convergence path of the optimization changes, and all downstream steps generates new results instead of retrieving results from the database. For example, removing `ETAMOB` from Step 2 in the previous example results in the following:

```
Fitting Result Data
-------------------
name       = Step1
parameters = PHIG : 4.2822, QMFACTOR : 4.3788e-06,
             CDSC : 0.26308
iterations = 7
fit error  = 16.42402671579762
converged  = 1
method     = bounded_trust_region
wallclock  = 0.26s
message    = 2-norm of trial step < eps[3]

Fitting Result Data
-------------------
name       = Step2
parameters = U0 : 0.01932, UA : 1.4596, EU : 1.4824,
             UD : 0.00064739, UCS : 2.4998e-07
iterations = 8
fit error  = 1.6507
converged  = 1
method     = bounded_trust_region
wallclock  = 7.36s
message    = ||F(x)||_2 - ||F(x) - J(x)s||_2 <eps[4]
```

The run time for the first step remains low because the change has been made in the second step, which now takes ~7 s.

## Reverse Search Extractions

When running multiple Mystic extractions across multiple conditions in the same, for example different geometries, device flavors, or when rerunning a project regularly with

small changes in target data from new TCAD/measurement results, it may also be useful to retrieve the best existing parameter set in the database using a reverse search algorithm. You can do this using the Mystic tool's `ExtractionUtils.ReverseSearch` function. The following example demonstrates this capability by first executing a multi-step extraction, adjusting the target data values by 5%, finding the current best parameter set using a reverse search, and then using the `optimize` flag to perform an optimization to find the true best parameter set for the new target data.

```
ExtractionUtils.DoStage(
    'Step1', Model, Simulator, [PHIG,QMFACTOR,CDSC],
    Optimiser, [IsVgld_BelowVt]
)
ExtractionUtils.DoStage(
    'Step2', Model, Simulator, [U0,UA,EU,UD,UCS,ETAMOB],
    Optimiser, [IsVgld_AboveVt]
)
ExtractionUtils.DoStage(
    'Step3', Model, Simulator,
    [PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,UCS,ETAMOB],
    Optimiser, [IsVgld]
)

# Scale the target by 5%
IsVgld.scale(1.05)

# Perform a reverse search against the new target and find the
# best existing parameter set in the database
ExtractionUtils.ReverseSearch(
    mdb, IsVgld, Model, optimize=True,
    parameters=[PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,UCS,ETAMOB]
)
```

The output of the `ExtractionUtils.ReverseSearch` command is:

```
Best existing parameter set:
+----------------+----------------------+
|   Parameter    |        Value         |
+----------------+----------------------+
|   nmos1.cdsc   | 0.14238785666429357  |
|   nmos1.cgsl   |        1e-20         |
|  nmos1.dvtp1   |         1.0          |
|   nmos1.eta0   |         0.0          |
|  nmos1.etamob  |  7.135360337604873   |
|    nmos1.eu    |  1.200097982796041   |
|   nmos1.hfin   |        4e-08         |
|   nmos1.phig   |  4.2778232413144535  |
| nmos1.qmfactor | 0.04298636993612226  |
|   nmos1.tfin   |        8e-09         |
|    nmos1.u0    | 0.024312196905548547 |
|    nmos1.ua    |  0.5051960473845137  |
|   nmos1.ucs    |  1.0955676204214733  |
|    nmos1.ud    |  0.1005523392201767  |
```

```
+----------------+----------------------+
|     error      | 0.03559017295353863  |
+----------------+----------------------+

Optimizing best parameter set...

Fitting Result Data
-------------------
name       = ReverseSearch
parameters = PHIG : 4.2776, QMFACTOR : 0.04177, CDSC : 0.1346,
             U0 : 0.02574, UA : 0.51327, EU : 1.2103,
             UD : 0.09876, ETAMOB : 7.2624, UCS : 1.0624
iterations = 14
fit error  = 0.7521253104902021
converged  = True
method     = least_squares
wallclock  = 9.13s
message    = `xtol` termination condition is satisfied.
```

From the output, you can see that the first three steps take a total of ~38 s, with a final error of 0.88%.

## ML Classifier Accelerated Extraction

Population based optimization algorithms such as evolutionary algorithms and DoE algorithms explore a parameter space iteratively and converge to the global optimal solution of the space. For higher dimensional problems, these algorithms can require hundreds of thousands of evaluations and run times can come prohibitively expensive. With the Mystic tool, you can enhance the performance of these algorithms by inserting a classifier between the algorithm and the PrimeSim HSPICE objective function, which classifies candidates in the population based on their likelihood to have a low error versus the target data. High potential candidates are simulated while low potential candidates are discarded. This can help to reduce the overall number of evaluations required for convergence and reduce the runtime, while still achieving successful convergence. The following example demonstrates this capability on a low-drain IdVg optimization problem.

```
Optimiser.set_method("differential_evolution")
Optimiser.SetOptimisationParameter("popsize", 30)
Optimiser.SetOptimisationParameter("maxiter", 50)
Optimiser.SetOptimisationParameter("ml_classifier", True)
Optimiser.SetOptimisationParameter("ml_kwargs", {
    "min_percentile": 80,
    "fill_multiplier": 10,
    "score_threshold": 0.8
})

Simulator.max_sims = 10

ExtractionUtils.DoStage(
    "Stage1", Model, Simulator, [PHIG,QMFACTOR,CDSC,U0,UA,EU],
```

```
        Optimiser, [IsVgld], parallel=True
    )
```

The `ml_classifier` and `ml_kwargs` options can be applied to any population-based algorithm to apply the classifier. The output of the above strategy snippet is as follows:

```
Fitting Result Data
-------------------
name       = 14nm-FinFET-Mystic-nmos-global
parameters = PHIG : 4.2635, QMFACTOR : 0.19948, CDSC : 0.09921,
             U0 : 0.02073, UA : 1.7408, EU : 1.3239
iterations = 9187
fit error  = 2.2325
converged  = False
method     = differential_evolution
wallclock  = 933.4s
message    = Maximum number of iterations has been exceeded.
score      = 0.80398
threshold  = 0.24717
run_ratio  = 62.8061
```

The important number to take note of in the output is that of `run_ratio`, which shows the percentage of evaluations which have been executed versus discarded. In the example above, almost 40% of total evaluations have been discarded, significantly reducing the run time.

For more information on applying the ML classifier, see the Machine Learning Acceleration section in the Mystic API documentation.

## The Engine Module

The `Engine` module is designed to be an object-oriented replacement for the `ExtractionUtils` module. It groups together the `Model`, `Simulator`, and `Optimizer` objects and provides functionality for defining the flow of an extraction strategy. A full description of the `Engine` module can be found in the Mystic API documentation.

A SPICE model extraction strategy is defined as a series of sequential optimization stages that fit sets of SPICE model parameters to a section of target data. The `Engine` module allows you to calculate parameter sensitivity, to define extraction steps, and to extract SPICE model parameters. Multiple stages can also be grouped together in a loop with a specified exit criterion.

The following code extract is from an example of a low-drain $I_d$–$V_g$ strategy:

```
Engine.add_extraction_step(IsVgld_BelowVt, [PHIG,QMFACTOR,CDSC])
Engine.add_extraction_step(IsVgld_AboveVt, [U0,UA,EU,UD,UCS,ETAMOB])
Engine.add_extraction_step(IsVgld_BelowVt + IsVgld_AboveVt,
                           [PHIG,QMFACTOR,CDSC,U0
                            UA,EU,UD,ETAMOB,UCS])
```

```
fit_error = Engine.extract(error_threshold=2.0, max_loops=3)
```

Individual extraction stages are defined using the `add_extraction_step` method based on a list of SPICE model parameters and a subset of the target data.

The `extract` method is then used to perform the extraction. This method moves through the added stages sequentially, then calculates a single RMS error at the end across a combination of all the data in the stages. If the `error_threshold` is met, then the extraction concludes successfully, and it returns to the start of the loop and reruns the stages. If the exit criterion is not met after the maximum number of loops, then the extraction concludes in any case.

# Full Strategy

This section describes how all of the individual components of a Mystic SPICE model extraction strategy work together to achieve an accurate SPICE model fitting of a template 14 nm FinFET device at low-drain bias conditions.

**Note:**

This section covers only the content of the extraction strategy. To run this example, see Chapter 6 on page 112.

In general, a standard extraction strategy can usually be split into the following sections:

- Configuring Settings
- Setting Up the Model
- Loading and Filtering Data
- Defining the Optimization Parameters
- Executing the Extraction Strategy
- Writing the Results

## Configuring Settings

This section controls top-level options for the data, simulator, and optimization algorithm. The following code snippet shows the settings applied to the low-drain $I_d$–$V_g$ extraction of a 14 nm FinFET device. In general, this section remains consistent across most extraction strategies.

```
tfin            = 8e-09         # Physical fin thickness [m]
hfin            = 4e-08         # Physical fin height [m]
wf0             = 4.301         # Workfunction
```

```
modstr         = "mosmod."

# Data settings
SimData.SetOptions(data_mask=True)

# Simulator settings
Simulator.spiceargs = ['-C']
Simulator.set_options(gmin=1e-18)

# Optimizer settings
eps=1e-3
Optimizer.SetMethod("BOUNDED_TRUST_REGION")
Optimizer.SetOptimizationParameter("jac", True)
Optimizer.SetOptimizationParameter("eps1",eps)
Optimizer.SetOptimizationParameter("eps2",eps)
Optimizer.SetOptimizationParameter("eps3",eps)
Optimizer.SetOptimizationParameter("eps4",eps)
Optimizer.SetOptimizationParameter("eps5",eps)
Optimizer.SetOptimizationParameter("eps6",eps)
Optimizer.SetOptimizationParameter("jacobian_eps",eps)
Optimizer.SetOptimizationParameter("kwargs",{"eps":eps,
                                             "step_type":"abs",
                                             "step_method":"cd"})

ExtractionUtils.settings.parallel = False
ExtractionUtils.settings.verbose = False
ExtractionUtils.settings.error_method = 'rmsd'
```

As the core of the Mystic tool is a Python interpreter, any number of custom variables can be defined through the strategy. The example defines some physical characteristics of the device:

- `tfin` is the physical fin thickness.
- `hfin` is the physical fin height.
- `wf0` is the work function of the gate stack.

Target data options are configured through the `SimData.SetOptions` method. Here, `data_mask=True`. This argument handles data more efficiently when filtering and reduces extraction runtime. For details, see Data Masks on page 31.

Simulator settings can be grouped into two categories:

- Command-line arguments to pass to PrimeSim HSPICE, controlled with `spiceargs`
- Options to add to the backend netlists as `.OPTION` commands, controlled by `Simulator.set_options`

Here, you use the PrimeSim HSPICE server mode (see Simulator on page 36) and set the minimum conductance in PrimeSim HSPICE. The `Optimizer` object is configured with the

`BOUNDED_TRUST_REGION` algorithm, which is the default. This can be changed using `Optimizer.SetMethod`.

`ExtractionUtils` settings can be defined through the `settings` attribute. The following are the available options.

- `parallel` is a flag to run the extraction stages in parallel based on the `Simulator.max_sims` attribute.
- `verbose` is a flag to enable debug mode and print all optimizer function evaluations in a table.
- `error_method` is the single error calculation method.

The `eps1` to `eps6` arguments control the exit criteria of the algorithm. The value 1e-3 is usually a sensible value for these, although it might need to be reduced or increased for better accuracy or turnaround time.

`kwargs` is a dictionary of options to pass to the Jacobian calculation of the `BOUNDED_TRUST_REGION` algorithm (see Optimization Algorithms on page 42).

## Setting Up the Model

Next, the initial `Model` object must be prepared for extraction by setting some default values. Here, you set the fin thickness, fin height, and workfunction based on the quantities defined earlier in the strategy. You also set the source and drain resistance of the model to zero to match the conditions of the target simulations. The specific parameter set used for a BSIM-CMG model extraction is beyond the scope of this user guide. For a more in-depth look at the model, refer to the TCAD to SPICE module of the TCAD Sentaurus Tutorial (see TCAD Sentaurus Tutorial: Simulation Projects on page 10).

```
Model.SetModelParameter(f'{modstr}TFIN',tfin)
Model.SetModelParameter(f'{modstr}HFIN',hfin)
Model.SetModelParameter(f'{modstr}PHIG',wf0)
Model.SetModelParameter('r_sc',0)
Model.SetModelParameter('r_dc',0)

Model.SetModelParameter(f'{modstr}ETA0', 0.0)
Model.SetModelParameter(f'{modstr}DVTP1',1.0)
Model.SetModelParameter(f'{modstr}CGSL', 1e-20)
```

## Loading and Filtering Data

Now, you load in the target data. When running the Mystic tool through Sentaurus Workbench, data is usually loaded in from the database using `mdb.Load` and `mdb.GetFitData`. This example is standalone, meaning you do not have a database connection and all data is file based. Here, you load the data in from PLT files using the

`SimData.FromPLT` method. The target PLT files come from a Sentaurus Device simulation of a 14 nm FinFET template device.

As can be seen in the following code snippet, the target data is attached with metadata containing the simulation temperature and a nodes list. The nodes list instructs the Mystic tool about the terminal order when it constructs netlists and must match the terminal naming in the PLT file.

```
metadata = {"temperature": 300,
            "nodes": ["drain","gate","source","substrate"]}

IdVgld = SimData.FromPLT("targets/IdVg_ld.plt", ivar="vgate",
                         dvar="idrain", metadata=metadata)

vt_gm_val = float(IdVgld.VtGmMaxFinal())

IdVgld         = IdVgld.FilterData('lt', -1e-13, dtype="dvar",
                                   mask=False)
IdVgld_BelowVt = IdVgld.FilterData('le', vt_gm_val-0.05, dtype="ivar")
IdVgld_AboveVt = IdVgld.FilterData('ge', vt_gm_val+0.05, dtype="ivar")

IdVgld._name = "IdVgld"
IdVgld.interactive = True
```

The `IdVgld SimData` object can then be filtered around `VtGmMax` in preparation for the extraction.

## Defining the Optimization Parameters

Now, you must initialize an `OptParam` of each SPICE model parameter to be extracted. For some parameters, such as the workfunction, it might be sensible to restrict the bounds based on its current value. In this case, the workfunction is obtained from the TCAD simulation, so you do not want it to vary significantly from that value.

```
phigval = Model.GetModelParameter(f'{modstr}PHIG')
PHIG    = OptParam(f'{modstr}PHIG',   phigval, phigval-0.5,
                   phigval+0.5, 1.0)
CDSC    = OptParam(f'{modstr}CDSC',   0.001,   0.0,   10.0,  1.0)
U0      = OptParam(f'{modstr}U0',     0.02,    0.006, 1.0,   0.02)
UA      = OptParam(f'{modstr}UA',     0.3,     0.0,   100.0, 1.0)
ETAMOB  = OptParam(f'{modstr}ETAMOB', 2.0,     0.0,   10.0,  2.0)
EU      = OptParam(f'{modstr}EU',     2.5,     0.1,   10.0,  2.0)
UD      = OptParam(f'{modstr}UD',     0.1,     0.0,   1.0,   0.1)
UCS     = OptParam(f'{modstr}UCS',    1.0,     0.0,   10.0,  1.0)

Stage1Parameters = [PHIG,CDSC]
Stage2Parameters = [U0,UA,EU,UD,UCS,ETAMOB]
Stage3Parameters = [PHIG,CDSC,U0,UA,EU,UD,ETAMOB,UCS]
```

The parameters are assembled into lists that will control which segments of data they are optimized against:

- `Stage1Parameters` will be fitted against below $V_t$ data.
- `Stage2Parameters` will be fitted against above $V_t$ data.
- `Stage3Parameters` will be fitted against all the low-drain $I_d$–$V_g$ data.

Coefficients of parameters presented as equations in the model file can also be defined and optimized using an `OptParam`. Consider the following model example:

```
.subckt nsub d g s b l=25n

.param phig_add = 0.0
.param cdsc_add = 0.0

m1 d g s b nmos1 l=l

.model nmos1 nmos

+ DEVTYPE=1
+ PHIG = '4.5 + phig_add*1.0'
+ CDSC = '0.1 + cdsc_add*1.0'
```

In this case, both PHIG and CDSC are defined by equations with a base value and an additional component defined as a `.param` statement above, which is a common occurrence in SPICE modeling.

The coefficients of PHIG and CDSC can be optimized as follows:

```
# Base model values
PHIG0 = OptParam("nmos1.phig", 4.5, 4.2, 4.8, 1.0, index=0)
CDSC1 = OptParam("nmos1.cdsc", 1.0, 0.0, 2.0, 1.0, index=1)

# Multipliers to the additive components
PHIG1 = OptParam("nmos1.phig", 1.0, 0.0, 2.0, 1.0, index=1)
CDSC1 = OptParam("nmos1.cdsc", 1.0, 0.0, 2.0, 1.0, index=1)
```

The `index` argument controls which numeric coefficient you are optimizing on from left to right in the equation definition, as shown by the highlighting.

## Executing the Extraction Strategy

Now everything is set up to begin the extraction process. As the environment is Python, you can use loops and value comparisons to control the flow of the extraction. For low-drain $I_d$–$V_g$ data, the extraction strategy is very simple: calculate the single-value fit error of the model against the `IdVgld` target, and continue executing the extraction loop until the error has dropped below 2% or you have reached the maximum of five loops.

In the core of the loop, first you fit the below-$V_t$ data, then the above-$V_t$ data, and then all the data together.

```
 fit_error=ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator,
Model)
 x=0

 while (x < 5 and fit_error>2.0):
    ExtractionUtils.DoStage("Step1", Model, Simulator,
       Stage1Parameters, Optimizer, [IdVgld_BelowVt])
    ExtractionUtils.DoStage("Step2", Model, Simulator,
       Stage2Parameters, Optimizer, [IdVgld_AboveVt])
    ExtractionUtils.DoStage("Step3", Model, Simulator,
       Stage3Parameters, Optimizer, [IdVgld_BelowVt,IdVgld_AboveVt],
       [1,2])
    fit_error=ExtractionUtils.PrintErrors("IdVgld", IdVgld,
       Simulator, Model)
    x+=1

 ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator, Model)
```

When using gradient descent–based algorithms such as bounded trust region, it is unlikely that fitting all the parameters against all the data, at the same time, will find a sensible minimum. The problem space is too big and the relationship between the SPICE model parameters is too nonlinear. It is possible to achieve a good result in a single extraction step by using a global optimization routine such as differential evolution, but this can be extremely time-consuming, taking thousands of iterations. Usually, an iterative approach as shown here can find a balance between the final error and the extraction time. After the extraction has finished, the final error is printed to screen.

## Writing the Results

All that remains is to write out the results of the extraction. If you run the Mystic tool through Sentaurus Workbench, results will be stored in the database there. Data is written to a PLT file for visualization in Sentaurus Visual, and the `Model` is written as a `.mod` file. This will be used as the initial model for the next extraction strategy.

The evaluation history of the `Optimizer` can be written to a CSV file. This can be useful as it shows the single error and parameter value progression across the entire extraction strategy, which can help to identify problems or inefficiencies.

```
 IdVgld.WritePLTFile("./results/IdVg-ld")
 Model.write("./models/nmos-ld.mod")
 Optimizer.write_parameter_csv("./results/IdVg-ld-parameters")
```

# 4

# Special Use Cases

*This chapter presents specific use cases in the Mystic tool.*

## Transient Simulation Extraction

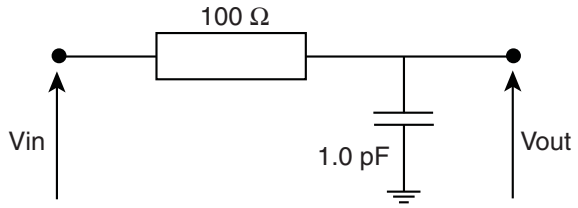
The Mystic tool can handle and extract model parameters against transient simulation data coming from Sentaurus Device as any CSV-like data. This section describes how to upload transient simulation data and use it to extract SPICE parameters based on a very simple RC time constant example shown in Figure 12.

```
.subckt rc in out

r in out 100.0
c out 0 1e-12

.ends
```

*Figure 12 Simple RC netlist example schematic*




The subcircuit `rc` will become the initial model for the extraction. Synthetic target data has been generated by PrimeSim HSPICE at R=100 and C=3e-12, with a square wave input stimulus. This data has been written to a CSV file as shown in Figure 13.

Before performing a parameter extraction, the target data must be uploaded to the TCAD to SPICE database as follows:

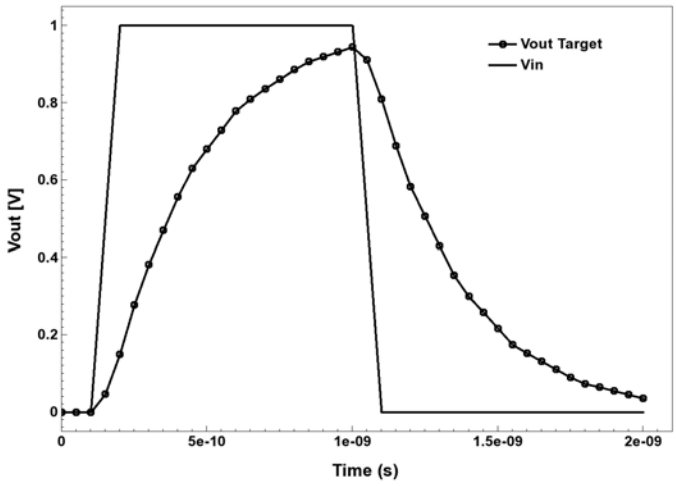
```
Data.from_csv("rc.csv", ivar="time", stimulus="vin",
              metadata={"nodes":["in","out"], "t":300},
              upload_ds="rc-target-data")
```

For more details about uploading target data, see Sentaurus Workbench API Documentation on page 10.

**Note:**

The `ivar` label for transient data must be `time` because the Mystic tool uses this in the backend to determine the simulation type and to set up the appropriate backend SPICE simulation.

*Figure 13 Synthetic target I–V data generated by PrimeSim HSPICE tool*




| time | vout | vout-fit | errors | vin |
|---|---|---|---|---|
| 0.000000e+00 | 0.000000e+00 | nan | nan | 0.000000e+00 |
| 5.000000e-11 | 0.000000e+00 | nan | nan | 0.000000e+00 |
| 1.000000e-10 | 0.000000e+00 | nan | nan | 0.000000e+00 |
| 1.500000e-10 | 4.620000e-02 | nan | nan | 5.000000e-01 |
| 2.000000e-10 | 1.500000e-01 | nan | nan | 1.000000e+00 |
| 2.500000e-10 | 2.780000e-01 | nan | nan | 1.000000e+00 |
| 3.000000e-10 | 3.830000e-01 | nan | nan | 1.000000e+00 |
| 3.500000e-10 | 4.700000e-01 | nan | nan | 1.000000e+00 |
| 4.000000e-10 | 5.570000e-01 | nan | nan | 1.000000e+00 |
| 4.500000e-10 | 6.300000e-01 | nan | nan | 1.000000e+00 |
| 5.000000e-10 | 6.800000e-01 | nan | nan | 1.000000e+00 |
| 5.500000e-10 | 7.290000e-01 | nan | nan | 1.000000e+00 |
| 6.000000e-10 | 7.790000e-01 | nan | nan | 1.000000e+00 |
| 6.500000e-10 | 8.100000e-01 | nan | nan | 1.000000e+00 |
| 7.000000e-10 | 8.360000e-01 | nan | nan | 1.000000e+00 |
| 7.500000e-10 | 8.620000e-01 | nan | nan | 1.000000e+00 |
| 8.000000e-10 | 8.870000e-01 | nan | nan | 1.000000e+00 |
| 8.500000e-10 | 9.070000e-01 | nan | nan | 1.000000e+00 |
| 9.000000e-10 | 9.200000e-01 | nan | nan | 1.000000e+00 |
| 9.500000e-10 | 9.330000e-01 | nan | nan | 1.000000e+00 |
| 1.000000e-09 | 9.450000e-01 | nan | nan | 1.000000e+00 |
| 1.050000e-09 | 9.110000e-01 | nan | nan | 5.000000e-01 |
| 1.100000e-09 | 8.100000e-01 | nan | nan | 2.040000e-15 |
| 1.150000e-09 | 6.890000e-01 | nan | nan | 0.000000e+00 |
| 1.200000e-09 | 5.830000e-01 | nan | nan | 0.000000e+00 |
| 1.250000e-09 | 5.060000e-01 | nan | nan | 0.000000e+00 |
| 1.300000e-09 | 4.300000e-01 | nan | nan | 0.000000e+00 |
| 1.350000e-09 | 3.540000e-01 | nan | nan | 0.000000e+00 |
| 1.400000e-09 | 3.000000e-01 | nan | nan | 0.000000e+00 |
| 1.450000e-09 | 2.590000e-01 | nan | nan | 0.000000e+00 |
| 1.500000e-09 | 2.170000e-01 | nan | nan | 0.000000e+00 |
| 1.550000e-09 | 1.750000e-01 | nan | nan | 0.000000e+00 |
| 1.600000e-09 | 1.530000e-01 | nan | nan | 0.000000e+00 |
| 1.650000e-09 | 1.320000e-01 | nan | nan | 0.000000e+00 |
| 1.700000e-09 | 1.110000e-01 | nan | nan | 0.000000e+00 |
| 1.750000e-09 | 9.050000e-02 | nan | nan | 0.000000e+00 |
| 1.800000e-09 | 7.390000e-02 | nan | nan | 0.000000e+00 |
| 1.850000e-09 | 6.440000e-02 | nan | nan | 0.000000e+00 |
| 1.900000e-09 | 5.500000e-02 | nan | nan | 0.000000e+00 |
| 1.950000e-09 | 4.550000e-02 | nan | nan | 0.000000e+00 |
| 2.000000e-09 | 3.610000e-02 | nan | nan | 0.000000e+00 |

The Mystic strategy for extracting against transient data is essentially the same as a standard DC or AC extraction:

```
# Load the target data from the database
RC = mdb.Load(ivar="time", dvar="vout")

# Calculate an initial error and write a PLT file
ExtractionUtils.PrintErrors("RC-trans", D, Simulator, Model, "rmsd")
RC.WritePLTFile("RC-trans-init", bias_info=False)

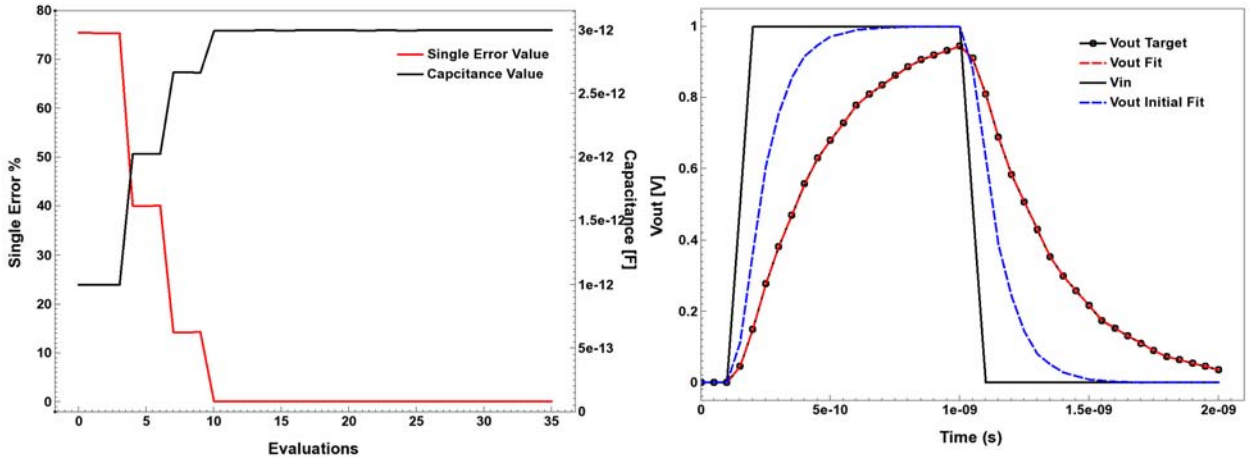
# Extract the capacitance
C = OptParam("C", 1e-12, 1e-13, 1e-11, 1e-12)
```

```
ExtractionUtils.DoStage("RC-trans", Model, Simulator, [C],
                        Optimizer, [D])

# Write the final PLT file and the parameter history CSV file
RC.WritePLTFile("RC-trans", bias_info=False)
```

The stimulus columns of the target data (`vin` in this case) are converted into piecewise linear (PWL) functions in the backend SPICE netlist to ensure the simulation receives the same triggers used to generate the target data. Figure 14 shows the parameter value history of the `Optimizer` and the initial and final fits of the capacitance.

*Figure 14 (Left) Parameter history and (right) final RC data fit for the capacitance extraction*




# MOSFET Reliability (MOSRA) Extraction

As CMOS technology scales down, reliability requirements become more challenging and more important. Hot-carrier injection and bias temperature instability can change the characteristics of a device over time. PrimeSim HSPICE MOSRA analysis can be used to model these effects, allowing circuit designers to predict the reliability of their design. Mystic provides the capability to extract PrimeSim HSPICE MOSRA parameters against $\Delta V_{th}$ and $\Delta I_{ds}$ targets.

The target data for a MOSRA extraction should be presented in a CSV file with the following layout:

```
temperature,stresstime,l,vdrain,vgate,delvth0,dids
85,3000,2.8e-08,1.1,1.1,0.0601,0.0206
85,3000,2.8e-08,1.3,1.1,0.0918,0.1460
85,3000,2.8e-08,1.5,1.1,0.1261,0.9991
85,3000,2.8e-08,1.1,1.3,0.1815,0.0165
85,3000,2.8e-08,1.3,1.3,0.2693,0.0742
85,3000,2.8e-08,1.5,1.3,0.4189,0.5421
```

```
85,3000,2.8e-08,1.1,1.5,0.5309,0.0071
85,3000,2.8e-08,1.3,1.5,0.7941,0.0466
85,3000,2.8e-08,1.5,1.5,1.1795,0.3020
```

The only required labels are `temperature`, `stresstime`, and at least one target, for example, `delvth0` or `dids`. The target labels must match the fields output to the `radeg0` stressing file produced by PrimeSim HSPICE. Other optional column labels are instance parameters, for example, `l`, and terminal biases, for example, `vdrain` and `vgate`.

Unlike conventional data that must be uploaded to the database, MOSRA target data is loaded directly into the strategy using the `CreateMOSRATarget()` method as shown here. In this case, you are only interested in the `delvth0` degradation type:

```
dvth = SimData.CreateMOSRATarget("mosra_target.csv",
                                 target=["delvth0"],
                                 instances=["l"],

metadata={"nodes":["drain","gate","source","substrate"],
          "mosra":{"bias":{"source":0.0,"substrate":0.0},
          "mosra_model":"nfet_ra",
          "base_model":"nfet"}})
```

Lists of instance parameters and target labels must be passed to the function to allow Mystic to identify the appropriate columns in the CSV file. Extra metadata can also be passed in during target creation to pass additional information that is not included in the CSV file such as static biases.

As with any standard Mystic target, the terminals present in the metadata `bias` dictionary must match those in the metadata nodes list. Note that the `mosra_model` and `base_model` keywords, which identify the MOSRA and base model definitions in the initial model card, are required entries in the `mosra` metadata dictionary if the built-in Mystic equations are used. These allow the Mystic tool to identify the MOSRA and base model definitions in the initial model card.

After the targets have been created, they can be treated as standard Mystic target data objects. When passed into an optimization stage, Mystic inherently knows how to set up the backend PrimeSim HSPICE MOSRA aging simulation and to parse the outputs accordingly.

When extracting against multiple MOSRA targets simultaneously, the Mystic tool must run one PrimeSim HSPICE simulation per target, which can cause a significant increase in extraction time. To combat this, the PrimeSim HSPICE MOSRA equations have been built in to the Mystic tool and can be evaluated directly without calling PrimeSim HSPICE. To select this mode of operation, set the `UseMOSRAEquation` attribute at the top of the Mystic extraction strategy as follows:

```
Simulator.UseMOSRAEquation = True
```

When running in this mode of operation, it is necessary to run a validation PrimeSim HSPICE simulation after the model parameters have been extracted to ensure the RMS error is consistent.

To correctly model physical reliability effects for a device, you might want to customize the model equations in PrimeSim HSPICE. Doing so will create a mismatch between the outputs of PrimeSim HSPICE and the outputs of the MOSRA equations built in to the Mystic tool. To prevent this, the Mystic tool allows you to change the built-in equations to match those that are being used in PrimeSim HSPICE using the `SwitchMOSRAEquation` method as shown here:

```
SwitchMOSRAEquation(new_equations, Model, delvth0,
                    error_threshold=1.0)
```

Here, `new_equations` is a Python callable with the function signature shown next, that is, it must accept model and data arguments. Inside the function, the `Model` and `Data` objects can be used to aid the construction of the equation:

```
def new_equations(model, data):
   # Metadata and model parameters can be retrieved from the model
   # and data objects are used within the custom equations
   L = data.metadata["mosra"]["instances"]["l"]
   hciap = model.get_model_parameter("nfet_ra.hciap")
   btiap = model.get_model_parameter("nfet_ra.btiap")

   # Define custom equations and perform calculations. Note that
   # the equations shown here are for illustration only
   dvth0 = 1e-3 * btiap * L
   dids = hciap * L

   # Return a dictionary of values calculated by the custom equations
   return {"delvth0":  dvth0, "dids": dids}
```

The return of the function is a Python dictionary, the keys of which must match the degradation labels from the target CSV file.

When the equations are changed, Mystic will run a validation simulation in PrimeSim HSPICE to ensure that the outputs of the new equations match the outputs of PrimeSim HSPICE within a tolerance dictated by the `error_threshold` argument. If this is not the case, then any parameters extracted against the Mystic equations will give incorrect or unexpected degradation effects when run in PrimeSim HSPICE.

After the extraction has concluded, data can be output to a CSV file by using the `WriteMOSRAFile()` method as shown here. The target and fit column headers will be appended with either `_target` or `_fit`, for example, `delvth0_target`, `delvth0_fit`.

```
delvth0.WriteMOSRAFile("mosra_output.csv")
```

## Variability Extraction Using Principal Component Analysis

Adding variation to a SPICE compact model can be achieved in different ways. In TCAD to SPICE tool flows, variation models are created through a combination of variability TCAD simulations, Mystic SPICE model extractions, and RandomSpice simulations.

In these flows, the Mystic tool produces individual model cards without variation information, which are amalgamated into a variation library by the RandomSpice tool.

This library accurately captures the variability trends shown by the input TCAD data and can be used to run simple circuit benchmarks for the target technology. The Mystic tool can also add simple variations to SPICE models themselves, without the need for the RandomSpice tool. This section discusses how to do this using the `CorrelatedParameters` class.

## Correlated Parameters

When running PrimeSim HSPICE Monte Carlo simulations, you can add a Gaussian distribution to a single parameter using the `agauss` statement. For example, the following model applies a Gaussian distribution to `PHIG`, causing a $V_t$ shift:

```
.param phig_std = 0.001
.param pvar = agauss(0,1,1)

.model nmos1 nmos
+ level = 72
+ devtype = 1
+ phig = '4.3 + phig_std*pvar'
```

The arguments passed to the `agauss` statement are the mean, the standard deviation, and the number of standard deviations. It is common practice in SPICE modeling to create zero-centered distributions with a standard deviation of one, and then shift them afterwards. In this example, `PHIG` will be centered around 4.3 with a single standard deviation of 0.001.

The simplest form of variation SPICE model is to apply these single distributions to multiple parameters. In reality, the variation of SPICE model parameters is likely to be highly correlated and this oversimplified approach produces unrealistic output distribution. For example, as $V_t$ decreases, the subthreshold slope also starts to decrease, so variation parameters for the workfunction and subthreshold slope are likely to be highly correlated. Applying two uncorrelated Gaussian distributions to these parameters is likely to underestimate the overall variation and give combinations of a high $V_t$ and steep subthreshold slope that would likely not occur.

The `CorrelatedParameters` class can help to add correlations to variation parameters. It uses a principal component analysis (PCA) technique to automatically calculate variation terms for each parameter based on correlation and standard deviation information provided by users, and then it applies these terms to the model. Consider the following example:

```
PHIG = OptParam("PHIG", 4.5, 4.2, 4.7, 1.0)
CDSC = OptParam("CDSC", 0.1, 0.0, 10.0, 1.0)
U0 = OptParam("U0",0.02,0.006,1.0,1.0)
UA = OptParam("UA",0.3,0.0,100.0,1.0)

CP = CorrelatedParameters([PHIG,CDSC,U0,UA])
```

```
CP.setup_model(Model)
CP.update_model(Model)
print(Model)

# Set parameter standard deviations
CP.set_sigma("PHIG",0.01)
CP.set_sigma("CDSC",0.001)
CP.set_sigma("U0",0.005)
CP.set_sigma("UA",0.002)

# Set some correlation between parameters. These are for demonstration
# purposes only and do not represent anything real
CP.set_correlation("PHIG","CDSC", 0.7)
CP.set_correlation("U0","UA", 0.8)

CP.update_model(Model, components=2)
print(Model)
```

Here, you start from a BSIM-CMG model that has been prefitted to some nominal performance target data. The example does not consider variability target data as there is no need for optimization (which is covered in Variability Target Data on page 66). Four parameters are selected to apply variation to, covering the workfunction, subthreshold slope, and mobility. These parameters are first initialized as `OptParams` as in any other standard Mystic extraction and are used to create the `CorrelatedParameters` object. Before adding variation, the model is set up with the correct parameter structure using `CorrelatedParameters.setup_model` and `CorrelatedParameters.update_model`. The first printing of the `Model` object should look something like this:

```
.model nmos1 nmos
+DEVTYPE=1
+LEVEL=72
+PHIG='4.5 + PHIG_var'
+UA='0.001 + UA_var'
+CDSC='0.1 + CDSC_var'
+U0='0.01 + U0_var'

.param PHIG_var = '0.0*pca0 + 0.0*pca1'
.param CDSC_var = '0.0*pca0 + 0.0*pca1'
.param U0_var = '0.0*pca0 + 0.0*pca1'
.param UA_var = '0.0*pca0 + 0.0*pca1'
.param pca0_tmp = agauss(0,1,1)
.param pca0 = pca0_tmp
.param pca1_tmp = agauss(0,1,1)
.param pca1 = pca1_tmp
.param pca2_tmp = agauss(0,1,1)
.param pca2 = pca2_tmp
.param pca3_tmp = agauss(0,1,1)
.param pca3 = pca3_tmp
```

The parameter definitions in the base model definition have been replaced with `'<value> + <param>_var'`. The variation parameters are then defined below the model as a combination of multiple `agauss` statements.

**Note:**

Before you apply any standard deviations, all of the coefficients are zero, that is, there is no variation.

There are as many `agauss` statements as there are variation parameters. Each `agauss` statement represents one eigenvector of the multivariate parameter distribution, and they are ordered from largest influence to smallest influence, that is, the coefficients of `pca0` represent the largest contribution to the overall variation and `pca3`, the lowest. You can also see that only `pca0` and `pca1` are applied to the parameters. This is controlled by the `components` argument that is passed to `CorrelatedParameters.update_model` method. Depending on the complexity of the variation, the terms of least influence can be omitted without significantly changing the shape of the overall parameter distribution, that is, you can use the `components` argument to reduce the dimensionality of the variation.

Each parameter then has a standard deviation applied to it by using the method `CorrelatedParameters.set_sigma`, and some correlations are applied between parameters using `CorrelatedParameters.set_correlation`. (The standard deviations are solely for demonstration purposes and do not represent any real technology.) After variation has been applied, you can call `CorrelatedParameters.update_model` again to apply these changes to the model. The final model printout is shown here:

```
.model nmos1 nmos
+DEVTYPE=1
+LEVEL=72
+PHIG='4.5 + PHIG_var'
+UA='0.001 + UA_var'
+CDSC='0.1 + CDSC_var'
+U0='0.01 + U0_var'

.param PHIG_var = '-0.0050724244*pca0 + 0.0076987343*pca1'
.param CDSC_var = '-0.00050724244*pca0 + 0.00076987343*pca1'
.param U0_var = '-0.0041036776*pca0 + -0.0023790397*pca1'
.param UA_var = '-0.0016414710*pca0 + -0.00095161589*pca1'
.param pca0_tmp = agauss(0,1,1)
.param pca0 = pca0_tmp
.param pca1_tmp = agauss(0,1,1)
.param pca1 = pca1_tmp
.param pca2_tmp = agauss(0,1,1)
.param pca2 = pca2_tmp
.param pca3_tmp = agauss(0,1,1)
.param pca3 = pca3_tmp
```

The variation coefficients have now been updated to reflect the combined variation of all the parameters. The model can now written to a file and used in a PrimeSim HSPICE Monte Carlo simulation.

## Variability Target Data

The variations produced by a Mystic PCA-based variability model can be compared against variability TCAD simulations or measurement target data. This section presents an example of this based on simulation data from the Synopsys statistical variability TCAD simulator, Garand VE. As the Garand VE tool is part of the TCAD to SPICE tool suite, all of its simulation results are uploaded to the TCAD to SPICE database, allowing the Mystic tool to seamlessly load and use the results as target data.

As with any other target in the Mystic tool, all of the information required to set up and run PrimeSim HSPICE simulations under the same conditions as the target is stored in the structure and the metadata of the target itself. Garand VE data can be loaded in to the Mystic environment as usual by using the `mdb.Load` method as follows:

```
IdVg = mdb.Load(ivar='vgate', dvar='idrain', combine=True)
```

In this scenario, the `combine` argument is essential to ensure that the Garand VE data is kept together in a single `Data` object. The second-level independent variable of the data is an increasing integer field labeled `ensemble_id`. The label here informs the Mystic tool that this is variation data and the Mystic tool should set up a PrimeSim HSPICE Monte Carlo netlist when running against this simulation data. It is assumed, at this point, that the `Model` object in the environment has already had some sensible variations applied to it. The variation data can then be passed as usual into `ExtractionUtils.PrintErrors`:

```
ExtractionUtils.PrintErrors('Variation', IdVg, Simulator, Model,
                            'rmsd')
```

When the simulation is finished, the fit column of the $I_d$–$V_g$ `Data` object is populated with the result of the PrimeSim HSPICE Monte Carlo simulation.

At this point, the error values between the curves are meaningless because each dataset in the target and fit simulations has different random seeds that do not align. To calculate a more sensible error, the results can be transformed, first into single figures of merit and then into the moment of the figure of merit distributions as follows:

```
VtIon = IdVg.MakeFoMTarget(['vt','ion'], [5e-8,None])
MeanStd = VtIon.MakeFoMTarget(['mean','std'])
```

A printout of `MeanStd` would look something like this:

```
               mean          std      mean-fit      std-fit        errors
index
Vt    2.143298e-01 1.986727e-02  2.143137e-01 2.483426e-02  1.767829e-01
Ion  -9.023801e-07 2.105117e-08 -9.017812e-07 2.633861e-08  1.776053e-01
```

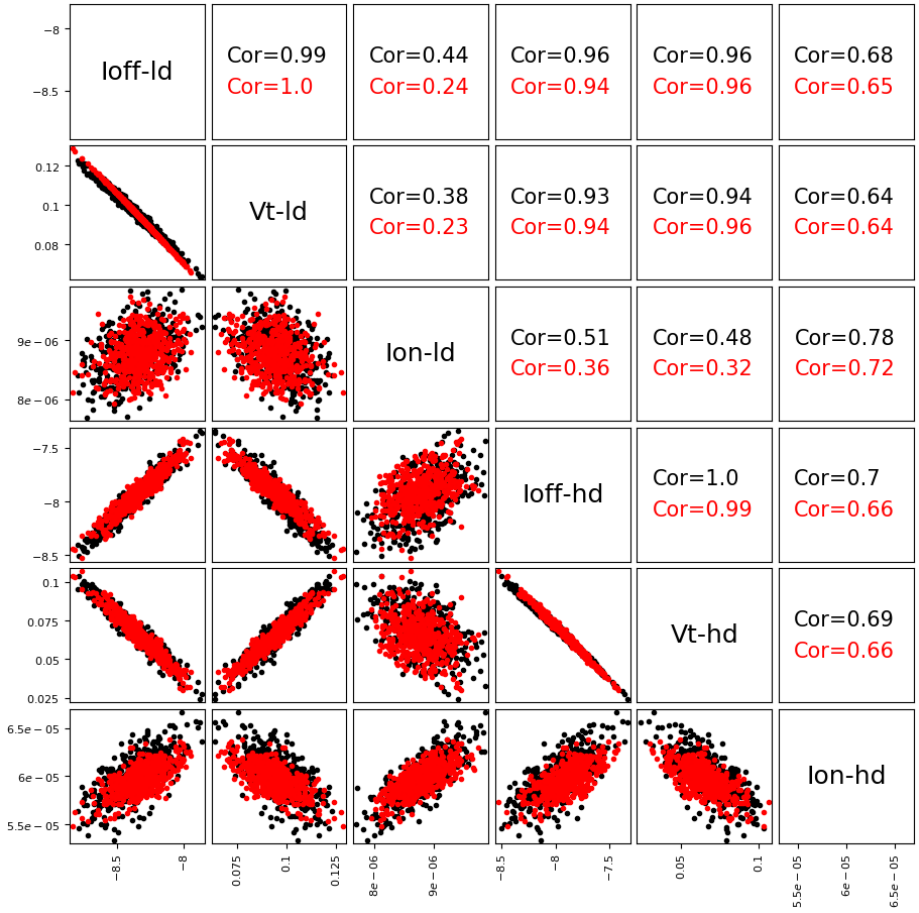
The error column of the transformed data should now contain much more manageable values. Any of these data objects can be written to file using `SimData.WritePLTFile` and plotted externally to the Mystic tool using Sentaurus Visual or the `matplotlib` Python plotting library provided with TCAD to SPICE Python.

Figure 15 shows an example of figure of merit distributions generated by the Garand VE tool being matched by a PCA-based variation SPICE model generated by the Mystic tool.

The transformed targets can also be used to optimize the parameters of the `CorrelatedParameters` class to give a variation model that better matches the Garand VE target distributions, but this is not covered in this document.

For more information as well as access to demonstration PCA projects, contact your local TCAD support team.

*Figure 15 Statistical variability figure of merit distributions generated by the Garand VE tool (black) and PCA variation model created by the Mystic tool (red)*



# RF Extraction

The general procedure of SPICE model parameter extraction for RF applications is slightly more complex than for standard CMOS applications. The main challenges in RF SPICE modeling are:

- There is a very high accuracy requirement on the SPICE models, because the admittance (Y) and scattering (S) parameters of an RF device are determined by the derivatives of I–V characteristics (transconductance and output resistance). To achieve good Y- and S-parameter accuracy, at the device active region, the RMS error of the SPICE model I–V must be under 1%.
- For very high frequency applications required by an RF SPICE model, distributed parasitic networks, which might not be important for general CMOS logic applications, must be considered.

To help address these challenges, the Mystic tool supports the fitting of SPICE model parameters against Y- and S-parameter targets and the extraction of RF-related parasitic networks.

This section describes how to interact with RF-related target data within the Mystic framework but it does not describe in any detail the parameter extraction procedure. For a full demonstration flow of RF SPICE model extraction, contact your local TCAD support team.

## Uploading Data

As with I–V, C–V, and transient targets, the Mystic tool expects scattering parameter data to be formatted in a certain way and to provide enough information about the simulation to allow the Mystic tool to reconstruct the simulation conditions in the backend PrimeSim HSPICE netlists it creates and runs.

This section describes how to upload PLT target data from a Sentaurus Device small-signal AC simulation to the TCAD to SPICE database for use in a Mystic extraction.

Figure 16 shows the PLT file containing admittance data versus frequency for a two-port network representing the device being modeled. The admittance data can be extracted against directly in the Mystic tool or converted to an S-parameter matrix internally before extracting.

*Figure 16 Sentaurus Device AC small-signal PLT output file*

```
DF-ISE text

Info {
  version   = 1.0
  type      = xyplot
  datasets  = [
    "frequency"
    "v(1)" "v(2)" "a(1,1)" "c(1,1)" "a(1,2)"
    "c(1,2)" "a(2,1)" "c(2,1)" "a(2,2)" "c(2,2)" ]
  functions = [
    Frequency
    Voltage Voltage Admittance Capacitance Admittance
    Capacitance Admittance Capacitance Admittance Capacitance ]
}

Data {
     7.00000000000000E+09
     4.00000000000000E-01   1.00000000000000E+00   2.05746555054810E-05   2.97051216276209E-14  -5.35215470262388E-06
    -8.78627105607206E-15   5.60590721073175E-02  -3.60398335248718E-14   2.66178221353005E-03   1.68118975683240E-14
     1.40000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   8.22337630452501E-05   2.96824687541098E-14  -2.13926407327530E-05
    -8.78061866298883E-15   5.59809725014359E-02  -3.60108012253218E-14   2.68220851905252E-03   1.68047545845361E-14
     2.10000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   1.84783263306495E-04   2.96447939893905E-14  -4.80736504747715E-05
    -8.77121762734341E-15   5.58510841968250E-02  -3.59625173262611E-14   2.71618445011621E-03   1.67928748365051E-14
     2.80000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   3.27901441671843E-04   2.95922166995640E-14  -8.53159247504598E-05
    -8.75809721449307E-15   5.56698214209221E-02  -3.58951358603892E-14   2.76360870954171E-03   1.67762959513041E-14
     3.50000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   5.11141477823370E-04   2.95249026662063E-14  -1.33009376531383E-04
    -8.74129807073404E-15   5.54377594668781E-02  -3.58088707536249E-14   2.82434060985314E-03   1.67550701858389E-14
     4.20000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   7.33934962097791E-04   2.94430627791226E-14  -1.91013951199770E-04
    -8.72087190386197E-15   5.51556301508956E-02  -3.57039941339137E-14   2.89820118265606E-03   1.67292640153693E-14
     4.90000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   9.55596327434367E-04   2.93469513890680E-14  -2.59160710773220E-04
    -8.69688108056986E-15   5.48243160860411E-02  -3.55808342043607E-14   2.98497457757609E-03   1.66989576135024E-14
     5.60000000000000E+10
     4.00000000000000E-01   1.00000000000000E+00   1.29532813201212E-03   2.92368643435425E-14  -3.37253126445924E-04
    -8.66939814654730E-15   5.44448438591123E-02  -3.54397727002367E-14   3.08440972763483E-03   1.66642442323733E-14
}
```

As shown in Updating PLT Files, `Data.from_plt` can be used to simultaneously read in and upload the PLT data to the TCAD to SPICE database. For admittance data, some extra options and metadata fields are required. The following example demonstrates an admittance file upload:

```
ds = dbi.create_dataset(node_prj, "rf-data", clean=True)

metadata = {"temperature": 298,
            "instances": {"l":3e-8, "w":5e-5},
            "ports": {"1": "gate", "2": "drain"},
```

```
            "nodes: ["drain","gate","source","substrate"],
            "bias": {"source": 0.0, "substrate": 0.0}}

Data.from_plt("RFQ_ac_des.plot", ivar="frequency",
              metadata=metadata, upload_ds=ds,
              rf_data=True, combine=True)
```

The `metadata` dictionary contains the same temperature, instances, nodes, and bias fields that are required for DC target data. There is an extra `ports` field that maps port numbers in the network to terminals of the SPICE model. In general, for standard MOSFETs represented as two-port networks, this is always the same: port 1 is mapped to the gate contact and port 2 is mapped to the drain contact.

For admittance data, two extra arguments must be passed to `Data.from_plt`. The `rf_data` argument instructs the `Data` class to produce an `RF_Data` object. This is a derivative of the `Data` class described in The Data Class with some extra functionality for matrix conversion and extracting the real and imaginary parts of the individual data components. The `combine` argument indicates that all of the individual admittance components should be combined horizontally into a single structure as shown here:

```
                    (11)re       y(11)i       y(12)re        y(12)i ...
frequency                                                          ...
2.795000e+10 1.784044e-04 3.542094e-03 -3.196667e-05 -9.527018e-04 ...
2.797500e+10 1.787211e-04 3.545245e-03 -3.202406e-05 -9.535513e-04 ...
2.800000e+10 1.790381e-04 3.548396e-03 -3.208150e-05 -9.544008e-04 ...
2.802500e+10 1.793553e-04 3.551547e-03 -3.213899e-05 -9.552503e-04 ...
2.805000e+10 1.796729e-04 3.554698e-03 -3.219653e-05 -9.560998e-04 ...
```

As with I–V and C–V data, the Mystic tool expects the column headers to be in a specific format that it uses to infer the simulation type when constructing backend PrimeSim HSPICE netlists. For Y- and S-parameter data, Table 10 shows the required header formatting.

*Table 10 Header notation*

| Notation | As ivar | As dvar |
|---|---|---|
| `y(<port1><port2>)<real>` | n/a | The admittance between port 1 and port 2 of the network, where `real` denotes whether it is the real or imaginary part |
| `s(<port1><port2>)<real>` | n/a | The scattering parameter between port 1 and port 2 of the network |
| `frequency` | AC frequency sweep. This is the required independent variable for admittance or S-parameter data. | n/a |

As mentioned in Updating PLT Files, there is a standard mapping to convert Sentaurus Device columns to the expected Mystic format. The default mapping for small-signal AC data is as follows:

```
"a(<port>,<port>)" is converted to "y(<port><port>)re"

"c(<port>,<port>)" is converted to "y(<port><port>)i"
```

If the data is not coming from the output of a Sentaurus Device simulation, then you can use `column_map` to convert and upload any CSV-like data format. See Uploading CSV Files on page 18.

## Loading and Manipulating Data

When the Y-parameter target data has been uploaded to the TCAD to SPICE database, it can be loaded into the Mystic extraction strategy as usual by using the `mdb.Load` method (see Chapter 2 on page 15). The following example shows how to do this for RF-specific data:

```
Y_data = mdb.Load(ivar='frequency',
                  dvar=['y(11)re','y(11)i',
                  'y(12)re','y(12)i',
                  'y(21)re','y(21)i',
                  'y(22)re','y(22)i'])
```

Here, you load the reflection and gain from both ports, giving you the full matrix. This is required if the data should be converted to S-parameter data as the conversion function requires all components to be present. After the data is in the Mystic environment, it can be filtered and resampled like any other data object:

```
Y_f28 = Y_data.FilterData('eq', 28e9, dtype='ivar')

Y22 = Y_f28.FilterData("eq", ["y(22)re","y(22)i"],
                       dtype="column", mask=True)
Y12 = Y_f28.FilterData("eq", ["y(12)re","y(12)i"],
                       dtype="column", mask=True)
Y21 = Y_f28.FilterData("eq", ["y(21)re","y(21)i"],
                       dtype="column", mask=True)
```

The first filter selects the first harmonic of the frequency sweep, and the subsequent filters use `dtype='column'` to isolate individual Y-components, so they can be extracted separately.

**Note:**

In these filters, `mask=True`. This is essential so a reference to all components is retained in order to construct the PrimeSim HSPICE simulation netlist. It also helps to speed up the extraction when multiple components are being extracted against simultaneously as only a single simulation of the root object is required. Only the RMS error will be calculated on the filtered components.

The `RF_Data` objects have a specific function for performing matrix conversions between Y-parameters and S-parameters. The following example shows how to create an S-parameter target from Y-parameter data using `SimData.MakeFoMTarget`, which is described in Figure of Merit Transforms on page 27:

```
S_data = Y_data.MakeFoMTarget("convert", "s")
```

SPICE model parameters can be extracted against either Y-parameters or S-parameters depending on the required effect.

## Extracting Parameters

In general, RF data can be treated in the same as any other type of data when it is used to extract SPICE model parameters.

Individual data objects can be passed to `ExtractionUtils.PrintErrors` to calculate single RMSD errors and to `ExtractionUtils.DoStage` to optimize parameter values. The following example shows a two-parameter fringing capacitance extraction against some Y-parameter target data:

```
cfs = OptParam("submosn.cfringes", 1e-16, 1e-17, 1e-14, 1e-18)
cfd = OptParam("submosn.cfringed", 1e-16, 1e-17, 1e-14, 1e-17)

#Y12_data1 : vg=0.4 vd=1.0
#Y12_data2 : vg=0.4 vd=1.2
#Y12_data3 : vg=0.6 vd=1.2

ExtractionUtils.DoStage("Step1", Model, Simulator,
                        [cfd,cds], Optimizer,
                        [Y12_data1,Y12_data2,Y12_data3],
                        [2,2,1])
```

The target here is the Y12 gain component of the matrix. One of the complexities of RF parameter extraction is that the device often must perform over a range of frequencies and biases. In this example, the full set of Y-parameter data includes three biases denoted in the commented lines.

While you want the fit to be as accurate as possible across all data, it sometimes makes sense to apply weighting factors to a section of the data to prioritize accuracy in that area. In this case, you double the influence of the `vg=0.4` targets and deprioritize the `vg=0.6`

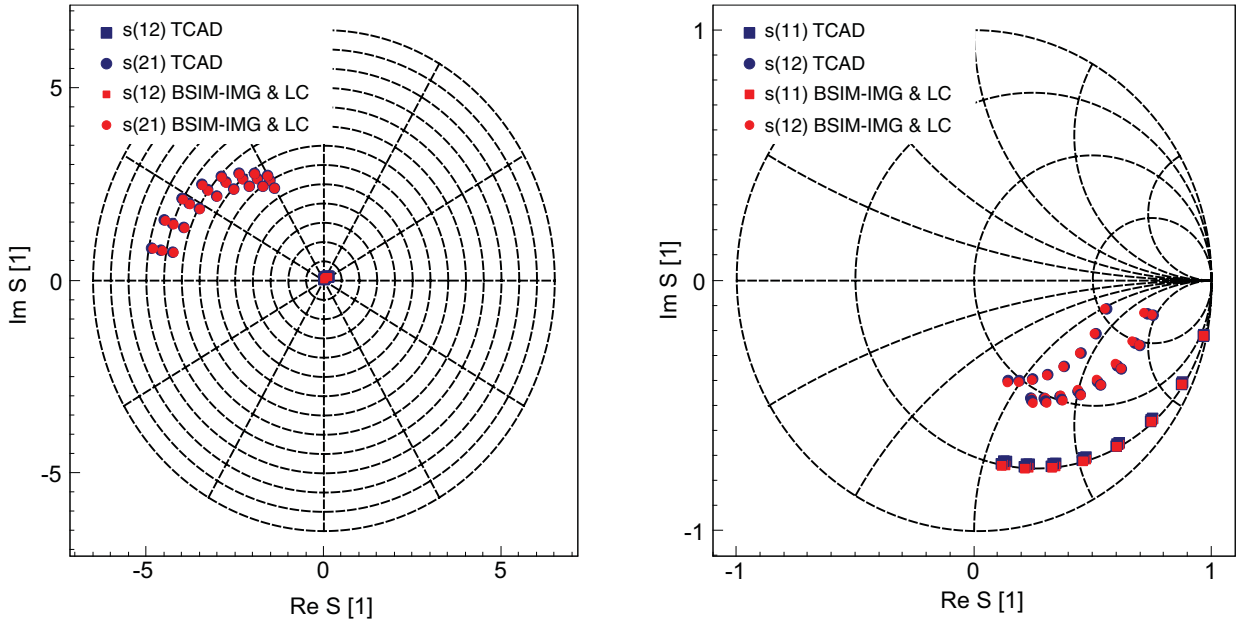
target. This gives extra flexibility where extreme accuracy cannot be achieved across the full performance range. Note that this is only a single extraction step in a complex multi-stage strategy.

## Data Output

RF simulation targets can be written to disk using the `WritePLTFile` or `WriteCSVFile` method. For details about these methods, see the Mystic API documentation.

After the data is written, it can be plotted externally using Sentaurus Visual (see Figure 17). The plotted data covers a range of bias conditions and frequencies, and shows an accurate fitting for all components between the BSIM-IMG SPICE model and TCAD data.

*Figure 17 Scattering parameter fitting for BSIM-IMG SPICE model versus TCAD*




# Lookup Table Modeling

Lookup table (LUT) models are analytic behavioral models that are used to perform fast analysis of a new technology before a reliable SPICE model or extraction strategy has been established. The Mystic tool can generate LUT models based on grids of target I–V and Q–V information using the Synopsys PrimeSim™ tool.

Target data for an LUT extraction should be uploaded to the TCAD to SPICE database in the usual way as described in Chapter 2, Uploading Target Data to Database. The target data

should include biases and currents for all the terminals of the device, that is, drain, gate, source, and substrate voltages and currents. It should also have length (L), width (W), and temperature metadata fields. These are required by the PrimeSim LUT model.

After the data has been loaded from the database into the environment using `mdb.Load`, it should be resampled using the `SimData.resampleNd` method to give an evenly spaced, dense grid of points as shown here:

```
IV = mdb.Load(project=targetDataProject, ivar='vgate',
              combine=True, extra_index='all')

IVr = IV.resampleNd(points={'vdrain':51, 'vgate':51, 'vsubstrate':11},
                    overwrite=False)
```

**Note:**

A denser target grid gives a smoother model response in the PrimeSim tool but results in slower simulation times. You should evaluate some different options and choose a reasonable grid density for your application.

To allow grid resampling, the `combine=True` and `extra_index='all'` parameters should be passed to the `mdb.Load` method to condense all the data into a single object. The `resampleNd` method should be passed as either a `points` or a `step` dictionary that determines the density of the data for all terminal biases.

After the dense data has been created, the LUT can be written using the `Simulator.write_LUT` method as shown here:

```
column_map = {'vdrain':     'Vds',
              'vgate':      'Vgs',
              'vsubstrate': 'Vbs',
              'idrain':     'Id',
              'igate':      'Ig',
              'isource':    'Is',
              'qdrain':     'Qd',
              'qgate':      'Qg',
              'qsource':    'Qs'}

Simulator.write_LUT(IVr[0], 'nmosmod', path='LUT_MODELS',
                    column_map=column_map)
```

The PrimeSim LUT model requires specific formatting of voltages, currents, and charges. Therefore, you must supply a `column_map` dictionary to convert the naming conventions in the target data to the expected format of the PrimeSim tool. The LUT model is written to the directory specified by the `path` argument using the `'nmosmod'` model name.

In the previous example, the output model will be written to `LUT_MODELS/nmosmod_W1u_L32n_T27.dat`, where the device width is 1 µm, the gate length is 32 nm, and the temperature is 27 K. The PrimeSim tool uses this model file when running SPICE simulations.

When the LUT model has been written, you can run a validation simulation using the `ExtractionUtils.PrintErrors` utility as follows:

```
Simulator.executable = 'primesim'

ExtractionUtils.PrintErrors("IV", IVr, Simulator, Model, "rmsd")
```

Before running the simulation, ensure a PrimeSim binary is available is your `$PATH` and switch to using the PrimeSim tool in the Mystic tool by setting `Simulator.executable`.

If the LUT model is operating correctly, then the error printed to screen should be close to zero as the data generated by the PrimeSim simulation should match the target data perfectly.

---

## Josephson Junction Extraction

The Josephson effect is a phenomenon that occurs when two superconducting electrodes are separated by an insulating (nonsuperconducting barrier). When cooled below the critical temperature of the material, $T_C$, a current, known as a supercurrent, can flow continuously across the barrier without any voltage applied. A device that takes advantage of this effect is known as a Josephson junction.

Josephson junctions can be used in a range of quantum-mechanical superconducting circuits, such as rapid single flux quantum (RSFQ) digital logic, superconducting quantum interference devices (SQUIDs), and superconducting qubits.

The PrimeSim HSPICE tool contains a SPICE model representing a Josephson junction. It can be used to simulate the behavior of these devices in a range of circuit applications.

The Mystic tool can extract SPICE model parameters for these devices against quasistatic I–V target data coming from either TCAD or measurement.

### Extracting Figures of Merit

The I–V target for a Josephson junction is a transient simulation where a stimulus current is applied across the device, eliciting a voltage response across the junction.

As with most transient targets, the I–V curve for a Josephson junction can be noisy and is a difficult shape to fit holistically. The model response also oscillates, making a direct extraction against the full I–V near impossible. To overcome this, the Mystic tool provides some specific transforms and figures or merit to split the full target into a smaller set of key characteristics. Figure 8 shows the parts of the curve that should be captured accurately.

First, you apply a time-averaged transform to dampen the oscillations of the SPICE model response:

```
iv = mdb.Load(ivar="time", temperature=4.2)
iv_smooth = iv.ApplyMeasure("jj-avg")
iv_smooth.TestMeasure(Simulator)
```

This ensures that both the target and fitted data is smooth and continuous. Then, you extract points P1 to P5 as well as the normal resistance (rnorm) and the subgap resistance (rsg) as follows (see Figure 8):

```
p1 = iv_smooth.P1()[0]
p2 = iv_smooth.P2()[0]
p3 = iv_smooth.P3()[0]
p4 = iv_smooth.P4()[0]
p5 = iv_smooth.P5()[0]

rnorm = iv_smooth.rnorm()[0]
rsub = iv_smooth.rsg()
```

The Josephson junction SPICE model available in the PrimeSim HSPICE tool is behavioral, meaning that many of the parameter values can be set directly based on the extracted electrical FoMs without the need for optimization. For example, the critical current parameter `ics` can be set using `p1`:

```
Model.SetModelParameter('ics', p1)
Model.SetModelParameter('rnorm', rnorm)
Model.SetModelParameter('rsub', rsub)
Model.SetModelParameter('vgap', (p2+p3)/2)
Model.SetmodelParameter('delv', p3-p2)
```

## Optimization

Although most parameters can be set behaviorally, some optimization might be required to obtain a more accurate fit or to extract parameters that cannot be set in this way, for example, the temperature dependence of the device.

As the simulation temperature increases, the gap voltage of the junction deceases. To model this effect, you can create FoM targets based on `p2` and `p3` and extract the two available temperature dependence parameters `tc` and `tcfct`.

```
Optimizer.set_method("doe")
Optimizer.set_optimization_parameters(method="bayesian_optimizer",
                                      Ns=10, do_plots=True)

tcfct = OptParam("jj_u.tcfct", 1.5,  1.5, 2.5, 100.0)
tc = OptParam("jj_u.tc", 9.28,  7.1, 9.5, 10.0)

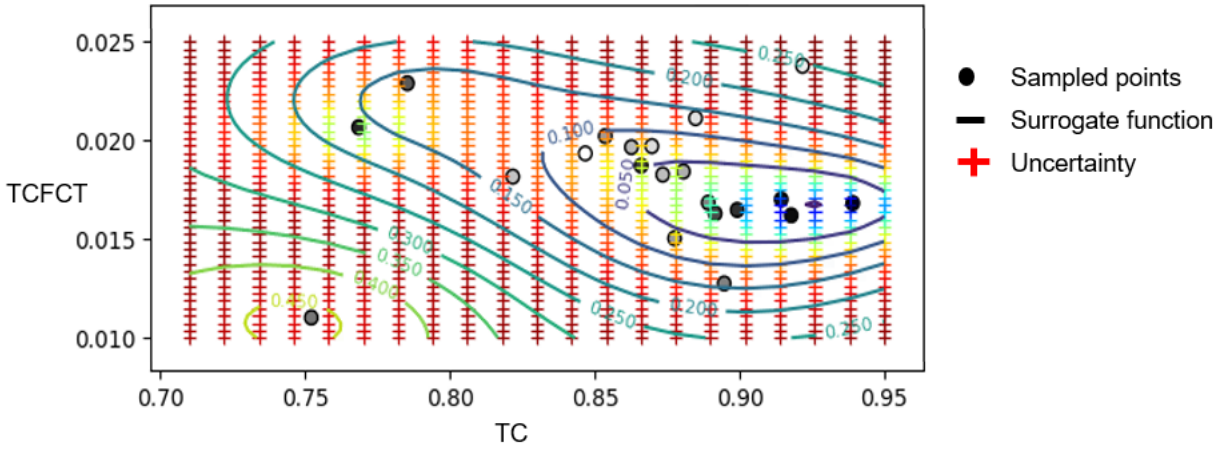
p2 = iv_smooth.MakeFoMTarget("p2", add_as_fom=True,
                             apply_to_measure=True)
p3 = iv_smooth.MakeFoMTarget("p3", add_as_fom=True,
                             apply_to_measure=True)
```

```
ExtractionUtils.DoStage("Stage1", Model, Simulator, [tc,tcfct],
                        Optimizer, [p2,p3], verbose=True,
                        parallel=True)
```

As you are going through two layers of data transformation here (time-averaged smoothing and FoM extraction), there will be significant noise in the output response based on small changes in the input parameter values. This means a conventional gradient-descent-based optimization algorithm will struggle to converge to a good solution. To overcome this issue, you can use the derivative-free Bayesian optimization algorithm from the Mystic optimizer library.

This algorithm samples the parameter space between the upper and lower bounds, which you defined, and then builds a surrogate objective function based on these evaluated points. It quantifies the uncertainty of the surrogate using Gaussian process regression. The algorithm then resamples the parameter space based on an acquisition function defined by the surrogate to obtain more information about the objective in areas where the surrogate predicts the error will be low. This process is repeated until the algorithm has established a minimum error within the parameter space. At each iteration of sampling, the algorithm produces a variation of the plot shown in Figure 18.

*Figure 18 Two-dimensional space sampled during Bayesian optimization of TC and TCFCT*



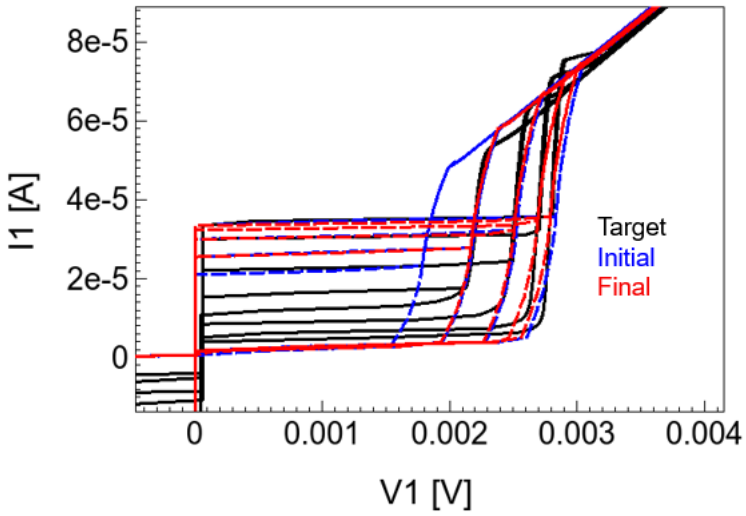

The plot shows the points that have been sampled in all the previous iterations, along with the current 2D representation of the current surrogate model (lines) and the uncertainty of the model (crosses) across the parameter space. If you have more than two parameters, then one 2D plot will be produced for each combination of two parameters. In this case, the 2D model represents a cutplane of the actual *N*-dimensional model going through the current optimal point.

## Data Output

The I–V data for Josephson junctions can be treated like any other target data object in the Mystic tool and can be written to PLT or CSV files using the `WritePLTFile()` and `WriteCSVFile()` methods, respectively. For details about these methods, see the Mystic API documentation.

Figure 19 shows an example of temperature-dependent I–V curves plotted externally in Sentaurus Visual, having been output from the Mystic tool. This data shows that the Mystic tool has extracted values for `tc` and `tcfct` that accurately capture the temperature dependence shown by the target data.

*Figure 19 Josephson junction I–V curves representing target data (black), initial fit data (blue), and final fit data (red) at various temperatures from 4 K to 7 K*



# 5

# Mystic Graphical User Interface

*This chapter describes the graphical user interface of the Mystic tool.*

You can use the Mystic graphical user interface (Mystic GUI) to run Mystic SPICE model extraction strategies interactively. It allows you to move through existing extraction strategies step by step, to analyze the outcomes of each individual command, and to generate plots to monitor the progress of the SPICE model fitting against the target data. The built-in IPython console also allows you to build up new strategies step by step, to explore parameter spaces, and to make manual adjustments to the values of SPICE model parameters.

This chapter provides an overview of the Mystic GUI and examples of how the Mystic GUI can be used to enhance the SPICE model extraction process. For more detailed information, consult the Mystic API documentation.

For instructions on how to launch the Mystic GUI, see Running the Mystic Tool as a Standalone Tool on page 10.

## Features of the Mystic GUI

The Mystic GUI comprises multiple dockable panels that can be configured to suit your preferences. Figure 20 shows the default layout of panels in the main window of the Mystic GUI.

The default layout splits the Mystic GUI into four main panels. Each of the elements in these panels is described in this section. The elements link together and interactive with each other to provide an environment for the interactive extraction of SPICE compact models.

All panels can be shown or hidden by using the toolbar on the left side of the main window.

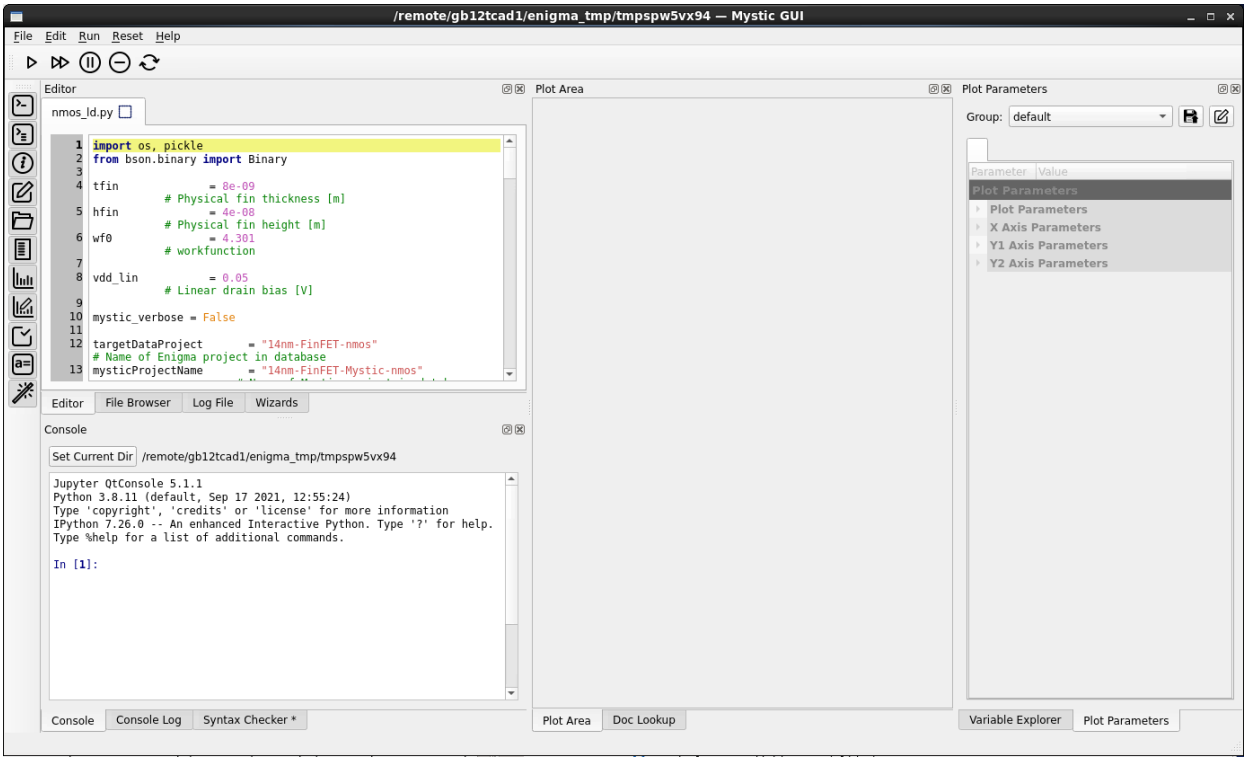
After you have arranged the layout of the main window to your preferences, you can save your layout as follows:

1. Choose **Edit** > **Preferences**.

   The Preferences dialog box opens.

2. In the left pane, click **Layout**.
3. In the **Save Current Layout As** field, enter a name for the layout.
4. Click **OK** or **Apply**.

*Figure 20 Default layout of panels in main window of Mystic GUI*




## Menus and Toolbars

Table 11 describes the available menus.

*Table 11 Menus of Mystic GUI*

| Menu | Description |
|---|---|
| File | Commands for opening and saving files, and exiting the Mystic GUI. Shortcuts for these operations are also provided. |
| Edit | Utilities for copying commands, finding patterns in the open file, and accessing the GUI preferences. |

*Table 11 Menus of Mystic GUI (Continued)*

| Menu | Description |
|---|---|
| Run | Commands for running the strategy file in the Console panel (see Console Panel on page 84). |
| Reset | Resets the Editor, Console, and Plot Area panels, or resets the entire session. |
| Help | Provides list of keyboard shortcuts and links to the Mystic API documentation. |

The Run toolbar, below the menu bar, controls the execution of the strategy (see Controlling the Execution of the Strategy on page 85).

Additional functionality is easily accessible from the toolbar on the left side of the main window. All these toolbar buttons have tool tips. To view tool tips, hover the cursor over a button.

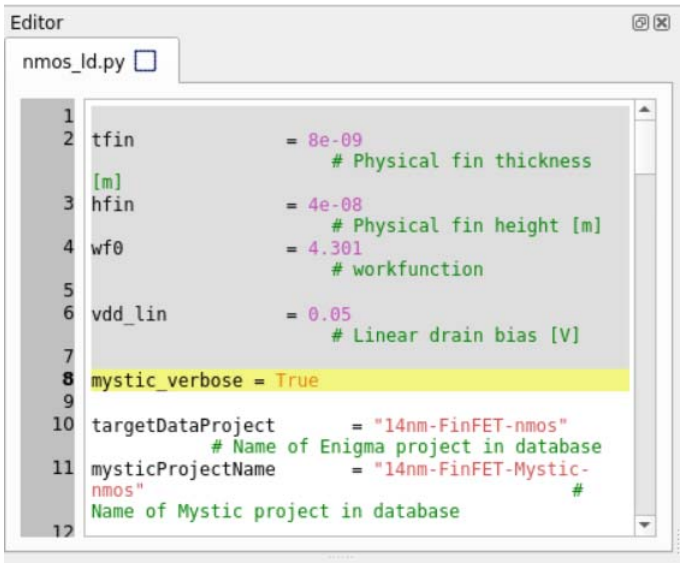
## Editor Panel

The strategy file itself opens in the Editor panel. When first opened, the entire strategy is editable. You can make alterations and save these to disk by using the **File** menu.

When an extraction is running, the contents of the file are progressively blocked and made read-only as each line is executed. The line highlighted by a yellow background is the next to be executed. For longer-running commands such as extraction stages, the line that is currently being executed has a green background. Executed lines have a gray background and become read-only (see Figure 21).

The Editor panel provides many useful features to help with strategy editing such as auto-indentation when there are line breaks, find and replace (press Ctrl+F), block commenting (press Ctrl+T), block tab indentation, and code completion.

*Figure 21 Strategy file: executed lines have gray background and are read-only*




## Inserting Breakpoints in Strategy File

You can insert breakpoints at any line below the one that is next to be executed.

To insert a breakpoint, either click the line number on the left side or move the cursor to the required line and then click the **Breakpoint** button on the Run toolbar (see Figure 25 on page 85).

After a breakpoint has been set, script execution stops at the breakpoint when you click the **Run/Continue** button on the Run toolbar (see Figure 25).

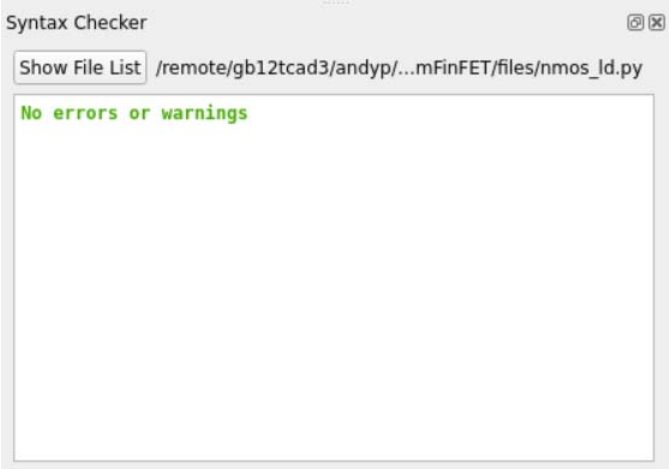
See also Controlling the Execution of the Strategy on page 85.

## Syntax Checker Panel

Prior to running a strategy, it is important to check for potential errors in the syntax. Identifying issues can save time, particularly, with long-running strategies.

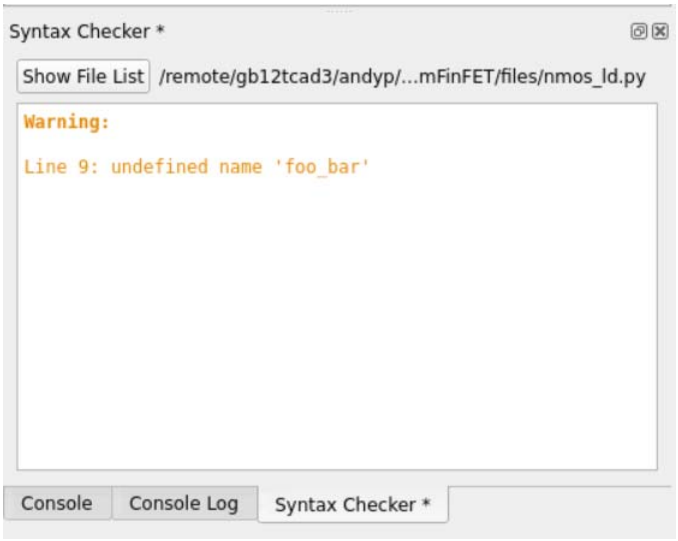
You can use the Syntax Checker panel to check for errors in the general Python syntax. If no errors are detected, then you will see the message as shown in Figure 22.

*Figure 22 Syntax Checker panel with message indicating no identified issues in the strategy*




If there are errors in the strategy file, for example with Python variable definitions, then an asterisk appears on the **Syntax Checker** tab as shown in Figure 23.

*Figure 23 Syntax Checker panel indicating an error by inclusion of asterisk next to its name*




Potential issues are split into two categories:

- *Errors* arise from problems that will definitely cause the extraction to fail, for example, Python syntax errors.
- *Warnings* arise from problems that might cause failures but might be harmless.

**Note:**

The Syntax Checker panel is designed to identify syntax-related problems only and cannot predict failures in the extraction procedure.

## Console Panel

The Mystic GUI runs commands by using a built-in IPython console built in to the Console panel. The Mystic namespace, described in Extraction Environment on page 12, is injected automatically into the Console panel when the Mystic GUI is initialized. The Console panel receives commands from the Editor panel and executes them in the context of the Mystic extraction environment.

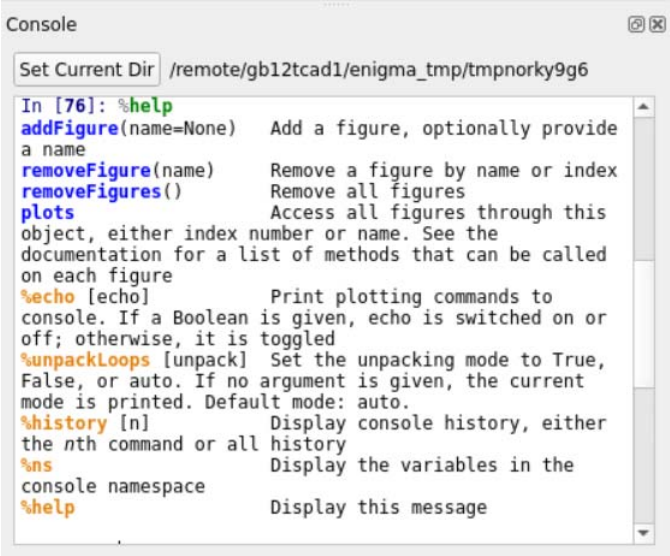
You can also enter commands into the Console panel directly. This can be useful for plotting results at the end of a strategy, making manual adjustments to the model, or debugging failures in the strategy.

As well as the Mystic namespace, the Console panel provides specific commands for creating, modifying, and updating graphics in the Plot Area panel. These commands are described further in Plot Area Panel on page 90. For a complete list of plotting commands, consult the Mystic API documentation.

The IPython console has a set of *magic* commands that provide extra functionality to the Console panel. Those commands specific to the Mystic GUI are described in the following table. IPython also provides a set of built-in magic commands [1].

| Command | Description |
|---|---|
| `%echo` | If this command is set, then making alterations to plots using the Mystic GUI will result in the equivalent command being printed to the Console panel. |
| `%help` | Prints a general help message, with an overview of some plotting commands and other commands (see Figure 24). |
| `%history` | Displays all the commands previously executed in the Console panel. |
| `%ns` | Lists the variables in the namespace. |
| `%unpackLoops` | When this command is set, *for* and *while* loops and *if* statements are unpacked when executing, so they can be stepped through line-by-line rather than being run as a single block. |

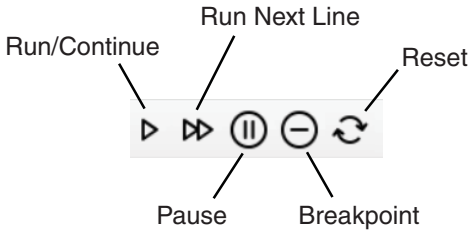
*Figure 24 Output of the %help command*




## Controlling the Execution of the Strategy

The Run toolbar controls the execution of the strategy in the Console panel.

*Figure 25 Run toolbar*




The **Run/Continue** button continuously executes commands until either a breakpoint or the end of the file is reached. This execution can be paused at any time by clicking the **Pause** button.

**Note:**

The line that is currently being executed will run until completion first.

By default, indented code blocks such as *if* statements and *for* and *while* loops are treated as a single block of Python code in continuous run mode. This means that the entire block is parsed and executed as a single entity.

The **Run Next Line** button executes the next line of the strategy that has a yellow background in the Console panel. If the next line is the start of an indented block, then the

block is unpacked, and the cursor and next line highlighting will move inside it. This means you can loop round a *for* or *while* loop multiple times, line by line, as you would in a debugger.

You can adjust the rules for unpacking blocks by using the `%unpackLoops` command.

Otherwise, to adjust the rules for unpacking blocks:

1. Choose **Edit** > **Preferences**.

   The Preferences dialog box opens.

2. In the left pane, click **Run**.

3. Under Loop Unpacking, select the required option.

4. Click **OK** or **Apply**.

The **Breakpoint** button inserts a breakpoint at the current line where the cursor is positioned in the Editor panel. A red dot next to the line number indicates that the line has a breakpoint. When breakpoints are set, the **Run/Continue** button will execute only as far as the next breakpoint. You can also create breakpoints by clicking the line number in the Editor panel.

The **Reset** button resets the entire console namespace, removing all plots and variables created or modified throughout the strategy, including the model. Click this button if the strategy is not converging as you expected, and you want to start again.

**Caution:**

Any information saved to the database is not reset, and content written to disk is not removed.

## Plot Log Panel

This panel lists all of the plotting commands that are executed in the background when you create or update plots by using the GUI.

This panel can be used to help you understand which commands to add to the strategy in the text editor to automate the creation or updating of plots during the strategy execution. You also can search the contents of the panel.

*Figure 26 Plot Log panel*



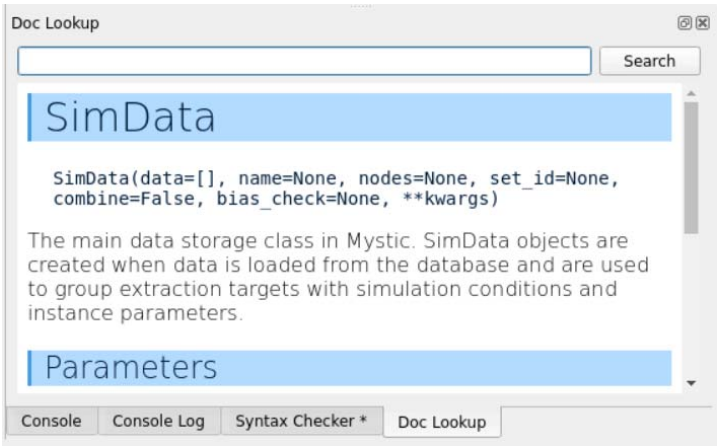

## Doc Lookup Panel

This panel shows the documentation for any Python variable. This includes all Python built-in functions and any functions, classes, modules, or variables available in the console namespace. For example, searching for *SimData* shows the documentation for the `SimData` class as shown in Figure 27.

You can also search for methods of classes using dot-separated syntax, for example, *SimData.WritePLTFile*.

You can press Ctrl+L to automatically search for the documentation related to the text directly under the cursor in either the Console or Editor panel.

*Figure 27 Documentation for SimData class*




## Variable Explorer Panel

You can use the Variable Explorer panel to browse the Python variables currently defined in the Console panel. The variable name, type, size, and value are displayed in a table (see Figure 28). The list of variables grows as the strategy runs and new variables are defined.

*Figure 28 Variable Explorer showing main table*

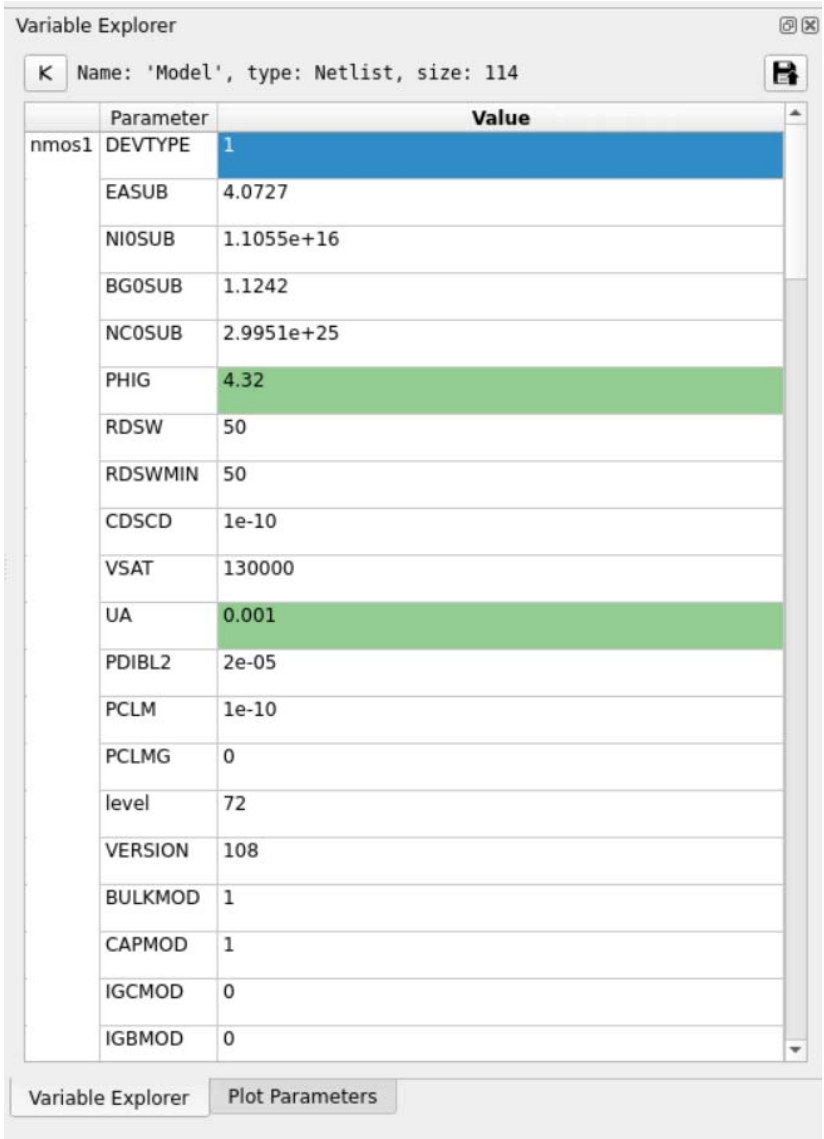
Variable Explorer

| Name | Type | Size | Value |
|---|---|---|---|
| Optimiser | Fitter | 1 | <common.optimisers.fitter.Fitter object at ... |
| Optimizer | Fitter | 1 | <common.optimisers.fitter.Fitter object at ... |
| **Model** | **Netlist** | **2** | **<common.CompactModels.model.Netlist object a...** |
| mdb | MysticDB | 1 | <mystic.Database.DB.MysticDB object at ... |
| args | NoneType | 1 | <NoneType object at 0x7ffff7d89280> |
| Simulator | HSPICESimulator | 1 | <common.SPICE.simulator.HSPICESimulator object at ... |
| **plots** | **NamedList** | **0** | **<qtcommon.utils.namedlist.NamedList object at ...** |
| a | int | 1 | 1 |
| tfin | float | 1 | 8e-09 |
| hfin | float | 1 | 4e-08 |
| wf0 | float | 1 | 4.301 |
| vdd_lin | float | 1 | 0.05 |
| mystic_verbose | bool | 1 | False |
| **targetDataProject** | **str** | **16** | **'14nm-FinFET-nmos'** |
| **mysticProjectName** | **str** | **23** | **'14nm-FinFET-Mystic-nmos'** |
| **drain_con** | **str** | **5** | **'drain'** |

Container-type variables highlighted in blue, such as *list*, *dict*, *np.array*, *pd.DataFrame*, *Data*, *SimData*, and *Netlist*, can be double-clicked to show a more detailed view of their contents.

For example, double-clicking the `Model` object (a `Netlist` instance) displays a table of all the SPICE model parameters and their current values. Parameters that have been modified during the extraction strategy are highlighted in green as shown in Figure 29. The `Model` can be saved to a file at any time by clicking the **Save** button in the upper-right corner.

*Figure 29 Netlist explorer view in Variable Explorer, where modified parameters are highlighted in green cells*



Double-clicking `SimData` objects shows the internal `Data` objects contained within the `SimData` object. The contents of the object can be plotted by clicking the button in the upper-right corner of the Variable Explorer panel.

This is one of many ways of creating plots in the Mystic GUI. For more information, see Plot Area Panel.

## Plot Area Panel

The Plot Area panel is an interactive canvas that displays the plots created during the execution of the strategy. There are different ways to generate plots in the Mystic GUI (such as using the Variable Explorer panel) and different types of plot can be created.

This section describes how to create simple line plots in the Console panel using the built-in plotting commands and Mystic `SimData` objects. Plots can also be generated automatically and updated by the Mystic tool as it runs an extraction (see Running a Strategy Interactively on page 101). For more information about different plot types, consult the Mystic GUI section in the Mystic API documentation.

You can create a plot by entering the `addFigure` command in the Console panel as follows:

```
fig = addFigure('IdVgld')
```

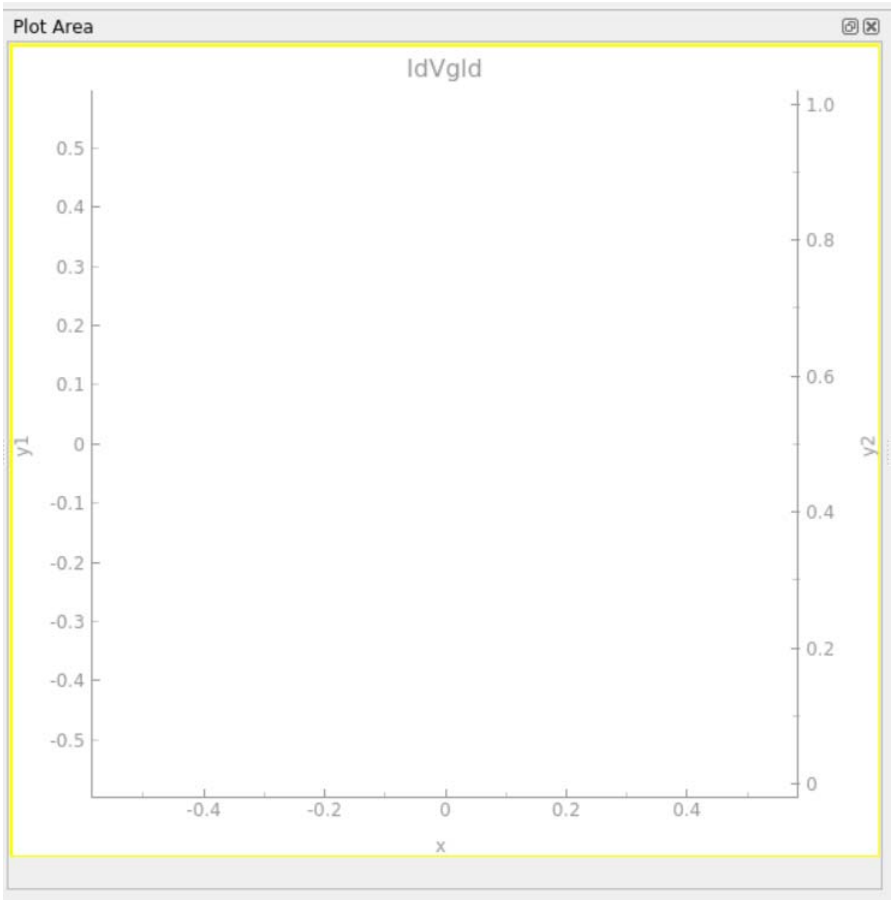
This command creates an empty plot in the Plot Area panel (see Figure 30) and returns a `FigureWidget` object that can be used to populate the plot with data.

The empty plot has a left and right y-axis allowing you to display two different data series on the same plot or to apply different scales to each axis, for example, a log10 scale. For more information about the axis properties, see Plot Parameters Panel on page 95.

Before adding data, first, you must decide which data you want to visualize. For this example, assume that a low-drain $I_d$–$V_g$ `SimData` object has been loaded into the environment already using the `mdb.Load` command. This data can be plotted directly using the `FigureWidget.plot` command as follows:

```
series = fig.plot(IdVgld)
```

*Figure 30 Plot Area panel showing empty plot*




The contents of the `SimData` object are plotted as shown in Figure 31.

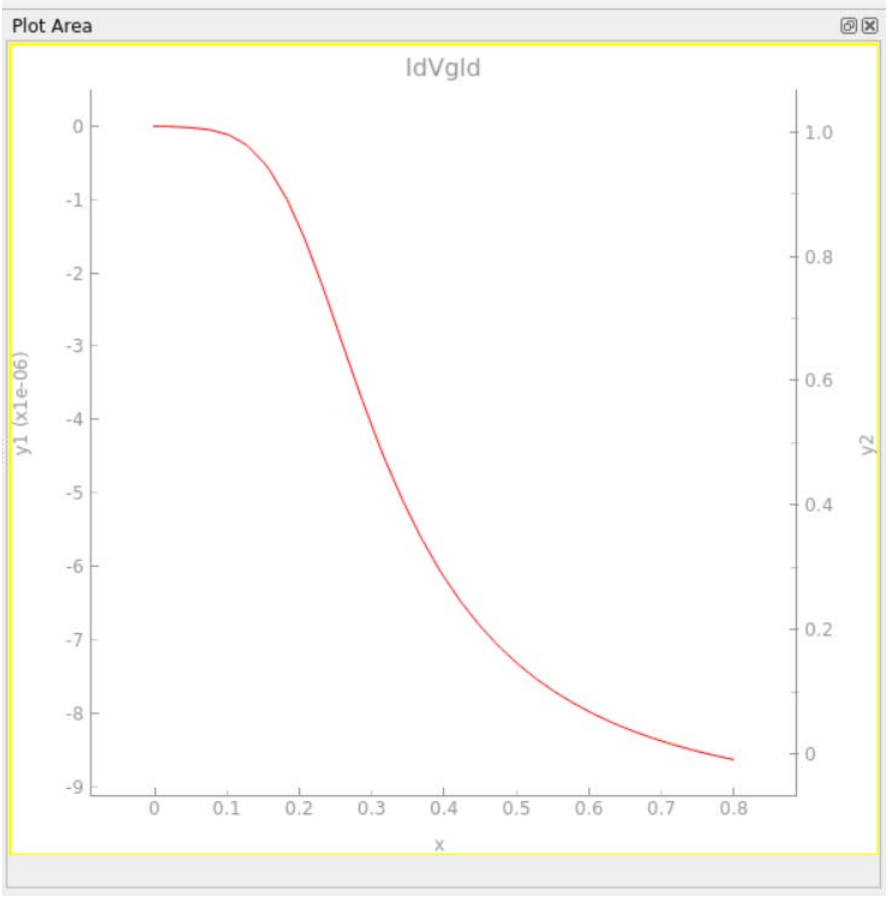
The `FigureWidget.plot` command returns a `PlotSeries` object, which can be used to update the curve properties. You can also do this by using the Plot Parameters panel (see Plot Parameters Panel on page 95).

`SimData` objects should contain both target data and fit data, and so two series, `IdVgId0` and `IdVgId1`, have been created in the Plot Parameters panel, but only one curve has been displayed on the plot. In this example, the `IdVgId` object has been freshly loaded from the database, so no fit data is present. You can calculate an initial error and update the data on the plot as follows:

```
ExtractionUtils.PrintErrors("IdVgId", IdVgId, Simulator,
                            Model, ErrorMethod)

series.updateData(IdVgId)
```

*Figure 31 Simple low-drain $I_d$–$V_g$ line plot*




The fit data should now appear on the plot as shown in Figure 32.

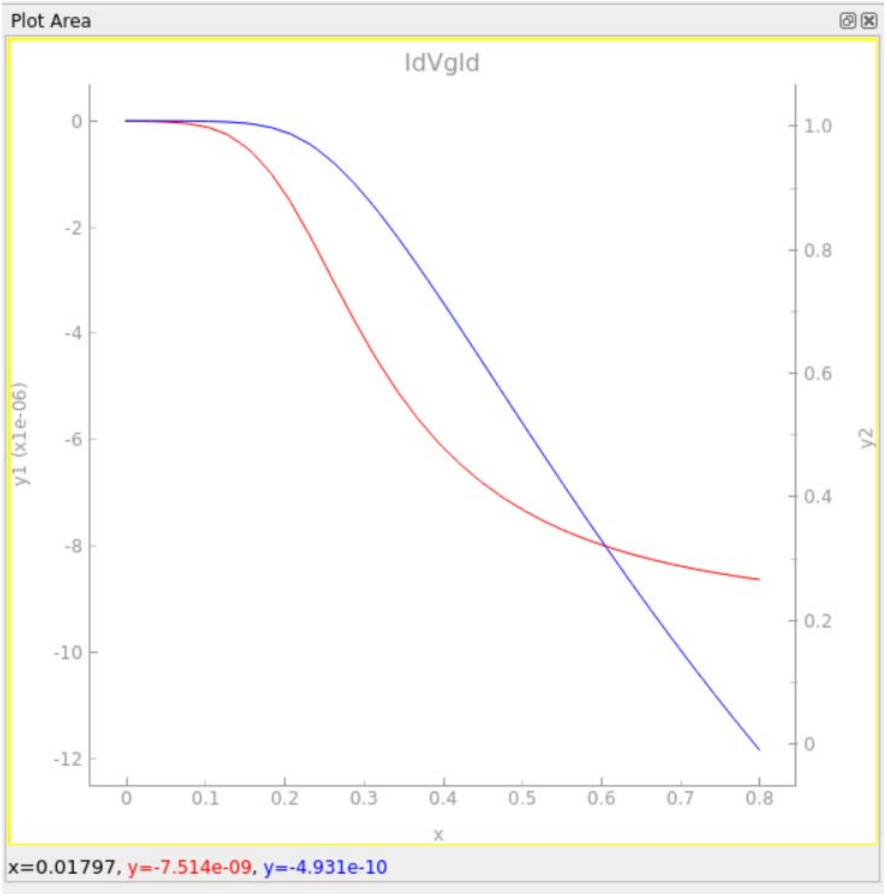
You can select a series on the plot by clicking it, which highlights both the series and its associated menu in the Plot Parameters panel.

There are several zoom and pan operations you can apply to a plot:

- To uniformly zoom in to or out of a plot, use the mouse wheel.
- For a free zoom that affects each axis independently, right-click and drag in any direction.
- Moving left and right zooms in to the x-axis, and moving up and down zooms in to the y-axes.
- To zoom only one axis, move the cursor on to that axis, and then hold the right mouse button and drag along the direction of the axis.
- To pan around the data freely, hold the left mouse button and drag on the plot.

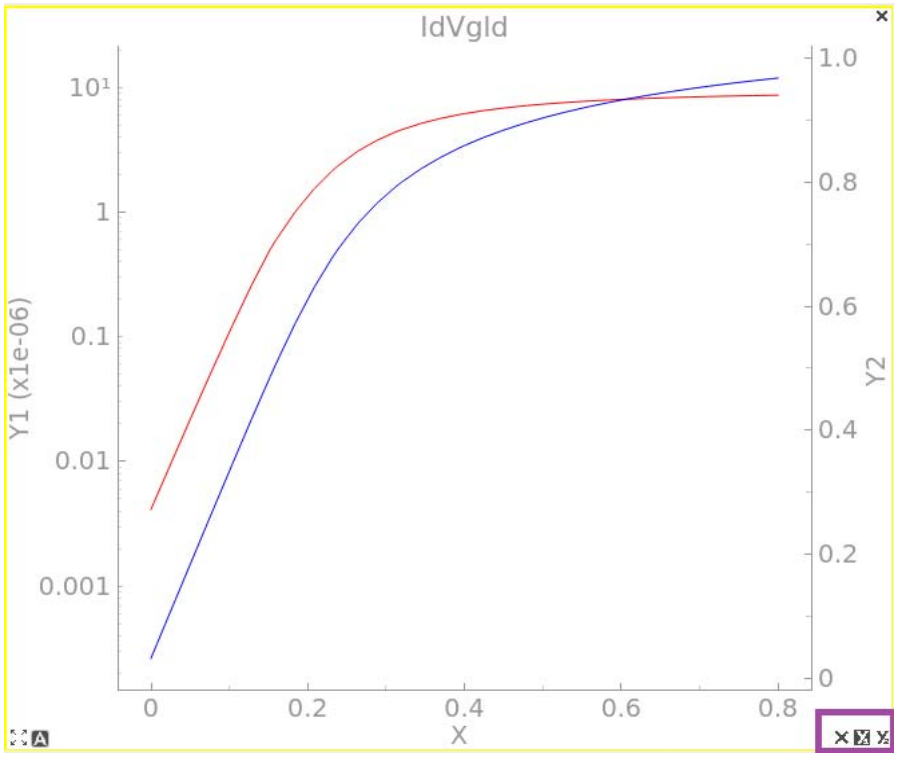
- To pan a single axis, hold the left mouse button on the axis.
- At any point, you can auto-scale the plot and return to the initial zoom by clicking the **A** button in the lower-left corner of the plot.

*Figure 32 Line plot for low-drain $I_d$–$V_g$ data showing both target data and fit data*




Plot axes can be log transformed directly from the plot using the three buttons in the lower right-hand corner of the plot as shown in Figure 34.

*Figure 33 Line plot $I_d$–$V_g$ data with a log 10 transform applied to the left-hand y-axis.*




These transforms can also be applied from the Plot Parameters panel as shown in the next section.

To remove a series from a plot, use the `FigureWidget.removeSeries` command. Plots can be removed from the Plot Area panel by using the `removeFigure` or `removeFigures` commands as follows:

```
removeFigure('IdVgld')
```

For a more comprehensive description of all the plotting commands, consult the Mystic GUI section in the Mystic API documentation.

When multiple plots are added to the Plot Area panel, they are arranged in a grid fashion, which you can scroll through. By default, a 2×2 grid of plots is visible. If more plots are added, then a scroll bar appears, allowing you to move between plots.

To adjust the number of plots that are shown without scrolling:

1. Choose **Edit** > **Preferences**.

   The Preferences dialog box opens.

2. In the left pane, click **Plots**.

3. Under Plot Grid, select **Auto Adjust**.

   With this option selected, the number of visible plots changes automatically as the size of the main window changes, in order to show the plots with the best aspect ratio.

4. Click **OK** or **Apply**.

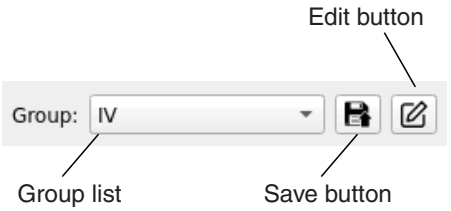
You can make a plot full screened by clicking the four arrows in the lower-left corner of the Plot Area panel. When the Mystic GUI is full screened, you can cycle through the plots using the tabs on the Plot Parameters panel.

When adding a new figure, an optional `plotGroup` parameter can be passed. This creates a new tab in the Plot Area panel with the given name. All subsequent figures created with the same `plotGroup` are added to that tab.

You can view groups by switching between tabs in the Plot Area panel or by using the **Group** list in the Plot Parameters panel as shown in Figure 34. You can edit groups by clicking the **Edit** button. Here, you can drag plots between groups. The **Add** and **Remove** buttons create a new group or remove the currently selected group, respectively. If a nonempty group is removed, then its plots are moved to the default group. When you are finished editing, click the **Back** button to return to the Plot Parameters panel.

The **Save** button next to the **Group** list exports all plots in the current group. A dialog box opens and all the plots in the group are exported to the selected directory as PNG files.

*Figure 34 Viewing groups using the Group list in the Plot Parameters panel*

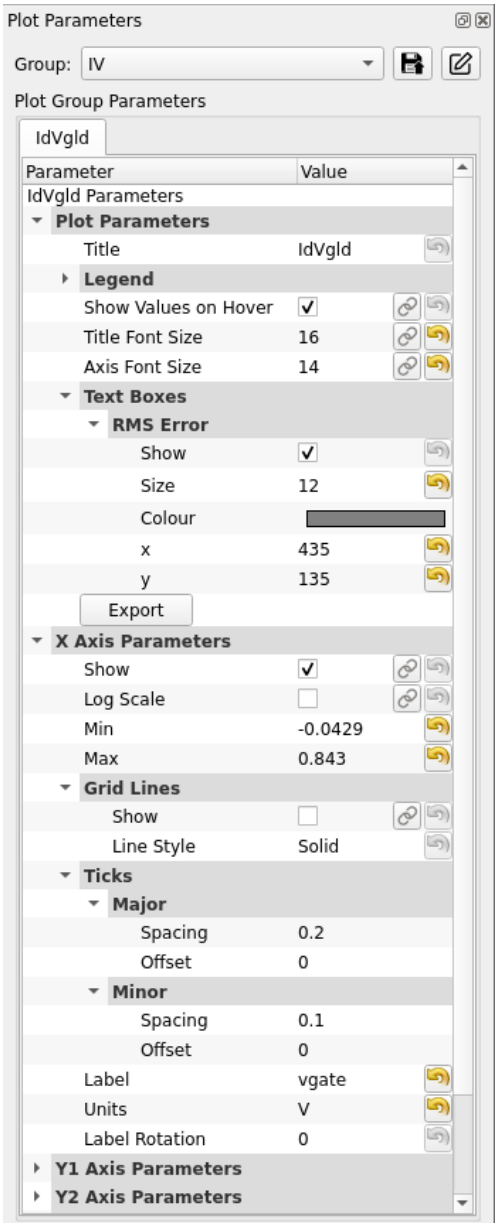



## Plot Parameters Panel

The Plot Parameters panel allows you to update the properties of all plots in the Plot Area panel. One tab on the Plot Parameters panel is created per plot.

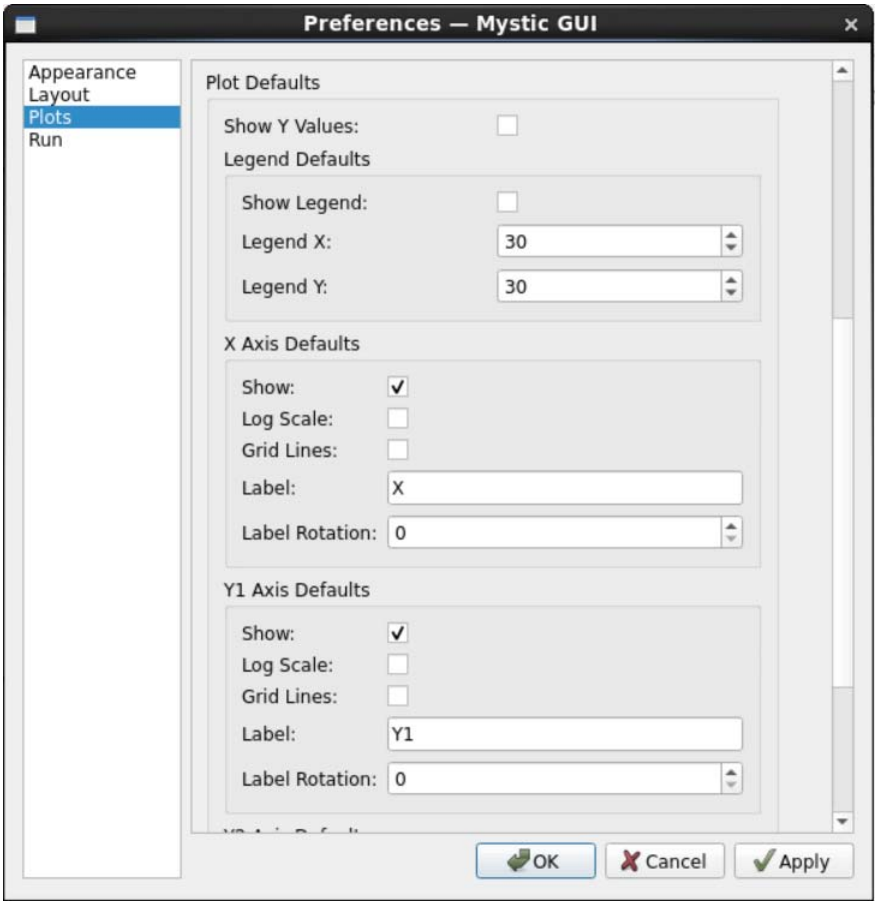
When a plot is first created, a new tab is created in the Plot Parameters panel titled with the name of the plot. Before a curve is created, the tab contains menus for plot parameters and axis parameters as shown in Figure 35.

*Figure 35 Plot Parameters panel showing a tab for an empty plot*




The default values of these parameters can be configured in preferences. To open the Preferences dialog box, choose **Edit** > **Preferences**. Then, click **Plots** in the left pane of the dialog box (see Figure 36).

*Figure 36 Default values for plot and axis parameters*



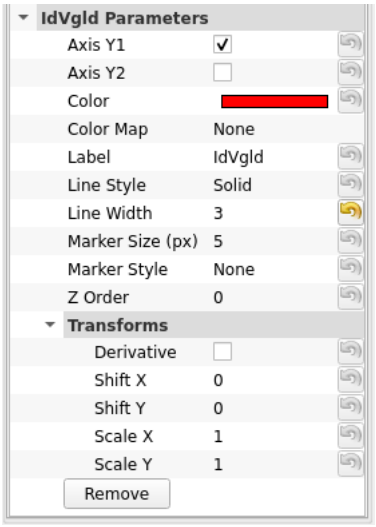

Some parameters have a link icon on their right. This icon can be used to link the state of the parameter across all plots in a group. For example, if you have a group featuring multiple I–V curves, then under Y1 Axis Parameters, you can select **Log Scale** in one plot to link it across all the plots. After you do this, any change to this parameter in any of the plots will propagate to all the other I–V plots in the group.

When the tab for a plot is selected, the corresponding plot in the Plot Area panel has a yellow border as shown in Figure 30, Figure 31, and Figure 32.

When a series is added to the plot, a new menu is created for that series. The contents of the menu depend of the type of series that has been plotted. Figure 37 shows the parameter structure for a standard scatter/line series.

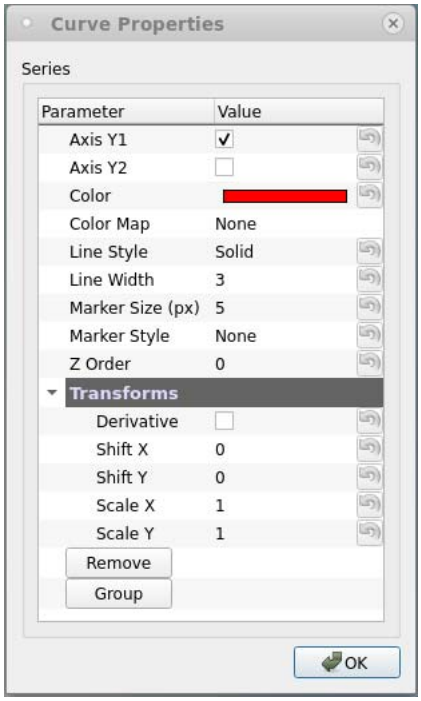
Changing any of these fields updates the series on the plot immediately. The menu for a series opens automatically when you click a series on the plot. The Transforms drop-down menu contains various visual transforms that can be applied to the series. If multiple transforms are applied simultaneously, the order in which they are applied is retained.

*Figure 37 Plot Parameters panel showing menu for a scatter/line series*




You can edit the properties of multiple curves simultaneously by highlighting multiple menus, and then right-clicking and choosing **Curve Properties**. Any properties changed in the Curve Properties dialog box apply to all of the selected curves (see Figure 38).

*Figure 38 Curve Properties dialog box for editing the properties of multiple curves simultaneously*

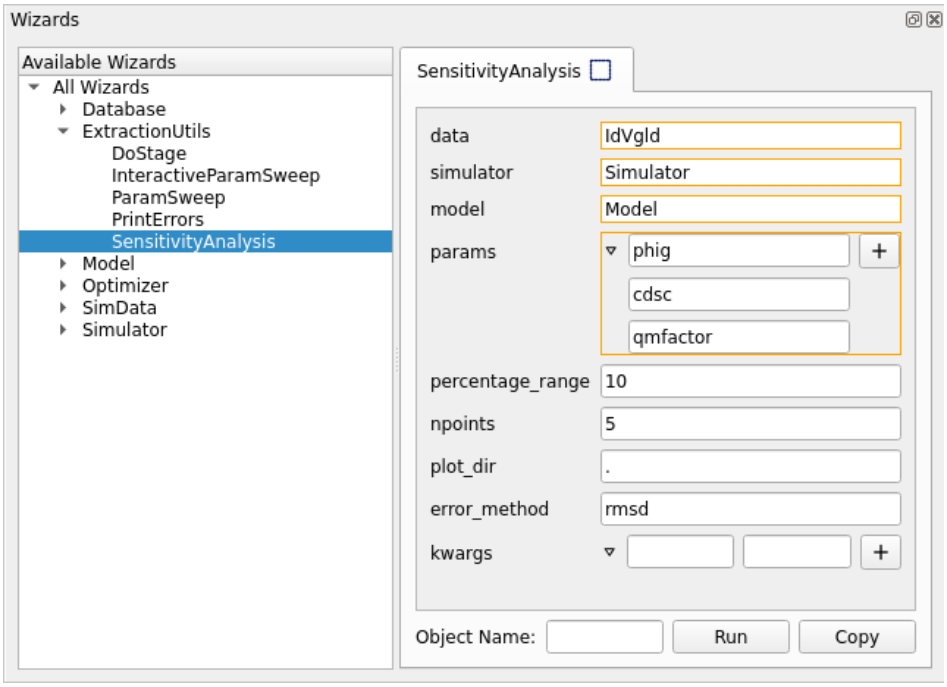


## Wizards Panel

The Wizards panel facilitates many common Mystic operations by either adding commands to the Editor panel or executing commands directly in the Console panel. The left pane of the panel shows a categorized tree view of the available wizards.

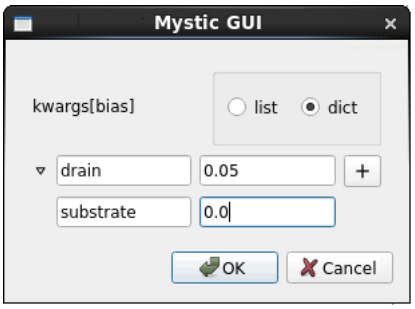
When a wizard is opened (by clicking its name in the left pane), it is added to a tab on the right side of the panel. Figure 39 shows the Wizards panel with the Sensitivity Analysis wizard opened.

*Figure 39 Wizards panel displaying available Mystic wizards in left pane; Sensitivity Analysis wizard is opened and populated with arguments*




Each wizard has the same general structure: a table of argument names and input forms that you can fill out. Some of these are self-explanatory, for example, a text box or Boolean option. List and dictionary argument types have **+** buttons next to their input boxes to add a new entry to the list or dictionary. These can then be collapsed or expanded using the arrow on the right-hand side. Lists and dictionaries can also be nested by right-clicking the input box and choosing **Create list or dict**. This opens a dialog box where you can enter new data as shown in Figure 40.

*Figure 40 Nested dictionary dialog box*



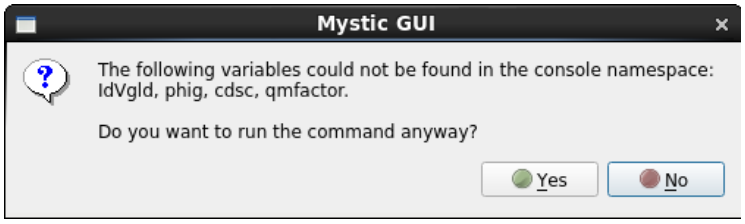

Some arguments can also be references to other Python variables defined in the Console panel, for example, the `DoStage` utility function requires a `Model` and `Simulator` object. In this case, the text entered in the box should be the name of the variable as defined in the Console panel.

When the arguments have been defined, you can copy the resultant command (click **Copy**) and then paste it into the Editor panel (press Ctrl+V), or you can executed the command directly in the Console panel by clicking the **Run** button. If the function to be executed has a return, then you can use the **Object Name** box to specify the name of the returned variable.

When you click the **Run** button, the arguments are cross-referenced with the console to ensure all Python variables defined in the argument table exist in the console namespace.

If any variables are not defined, then a dialog box opens and you are prompted to confirm whether to proceed, as shown in Figure 41.

*Figure 41 Dialog box listing missing variables: in the Sensitivity Analysis example in Figure 39, phig, cdsc, qmfactor, and IdVgld are missing*




## Usage Examples

This section describes some common uses of the Mystic GUI in realistic Mystic examples, which focus specifically on plotting rather than the strategy itself. For more information and access to demonstration projects, contact your local TCAD support team.

## Running a Strategy Interactively

When running a strategy in the Mystic GUI, you can monitor the quality of the fit of the model against the target data in real time as the strategy progresses. You can do this by making some minor updates to an existing strategy file.

First, you must identify the target data you want to monitor throughout the strategy. This is usually the target against which the best overall fit is required by the end of the strategy. The identified target or targets can then be tagged as interactive as follows:

```
IdVg = mdb.Load(ivar="vgate", dvar="idrain",
                project=targetProjectName)
IdVgld = IdVg.FilterData('eq', 0.05, 'bias.drain')

IdVgld.interactive = True
IdVgld.name = 'IdVgld'
IdVgld.plotGroup = 'IV'
```

One plot is created in the Plot Area panel for each piece of interactive data, using the given name containing both the target data and fitted data. For example, if $I_d$–$V_g$ and $I_d$–$V_d$ data are both present and tagged as interactive, then two separate plots are created that are both updated in real time. In this case, you are working on a low-drain $I_d$–$V_g$ strategy, so you have only one target.

As mentioned in Plot Area Panel, figures can be divided into different plot groups. This feature can also be used when running in interactive mode by setting the `plotGroup` attribute of the `SimData` object. This can be used to split interactive data into different categories, for example, separating I–V and C–V data.

As the strategy runs, the interactive targets are updated at the end of every optimization stage (`ExtractionUtils.DoStage`), even if the interactive object is not the target for that stage.

A snapshot of an interactive target can be taken at any point throughout the extraction by calling `ExtractionUtils.PrintErrors` with that target. The label passed to `ExtractionUtils.PrintErrors` dictates how the snapshot is displayed. If the label passed *matches* the name of the object, that is:

```
ExtractionUtils.PrintErrors("IdVgld", IdVgld, Simulator,
                            Model, ErrorMethod)
```

then the existing fit data is updated. The next time that `ExtractionUtils.DoStage` is called, this snapshot is overwritten by new results.

If the label *does not match* the name of the object, that is:

```
ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator,
                            Model, ErrorMethod)
```

then a new series is created on the plot with the specified label. This series remains unchanged for the duration of the extraction unless it is updated again using `ExtractionUtils.PrintErrors`. This can be used to create snapshots of certain conditions, such as the initial fit or the best overall fit that has been achieved so far.

Providing a visual reference point for the remainder of the extraction, consider the following low-drain $I_d$–$V_g$ example strategy:

```
ExtractionUtils.PrintErrors("Initial", IdVgld, Simulator,
                            Model, ErrorMethod)

fit_error=1000
x=0
while (x < 3 and fit_error>2.0):
   ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                           Stage1Parameters, Optimizer,
                           [IsVgld_BelowVt])
   ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                           Stage2Parameters, Optimizer,
                           [IsVgld_AboveVt])
   ExtractionUtils.DoStage(mysticProjectName, Model, Simulator,
                           Stage3Parameters, Optimizer,
                           [IsVgld])
   fit_error=ExtractionUtils.PrintErrors("IsVgld", IsVgld, Simulator,
                                         Model, ErrorMethod)
   x+=1
```

In this example, an initial snapshot labeled `"Initial"` is created to retain a visual reference to the starting point of the extraction. Going through the example, line by line, you can see what is produced. Figure 42 shows the plot after the first call to `ExtractionUtils.PrintErrors`.

The initial conditions are represented by the blue curve. There is also an RMS error value between the target and fitted curves. This can be activated or deactivated from the Plot Parameter panel. Moving passed the first call to `ExtractionUtils.DoStage`, you see the plot shown in Figure 43.

*Figure 42 Snapshot of initial conditions of the SPICE model fitting*

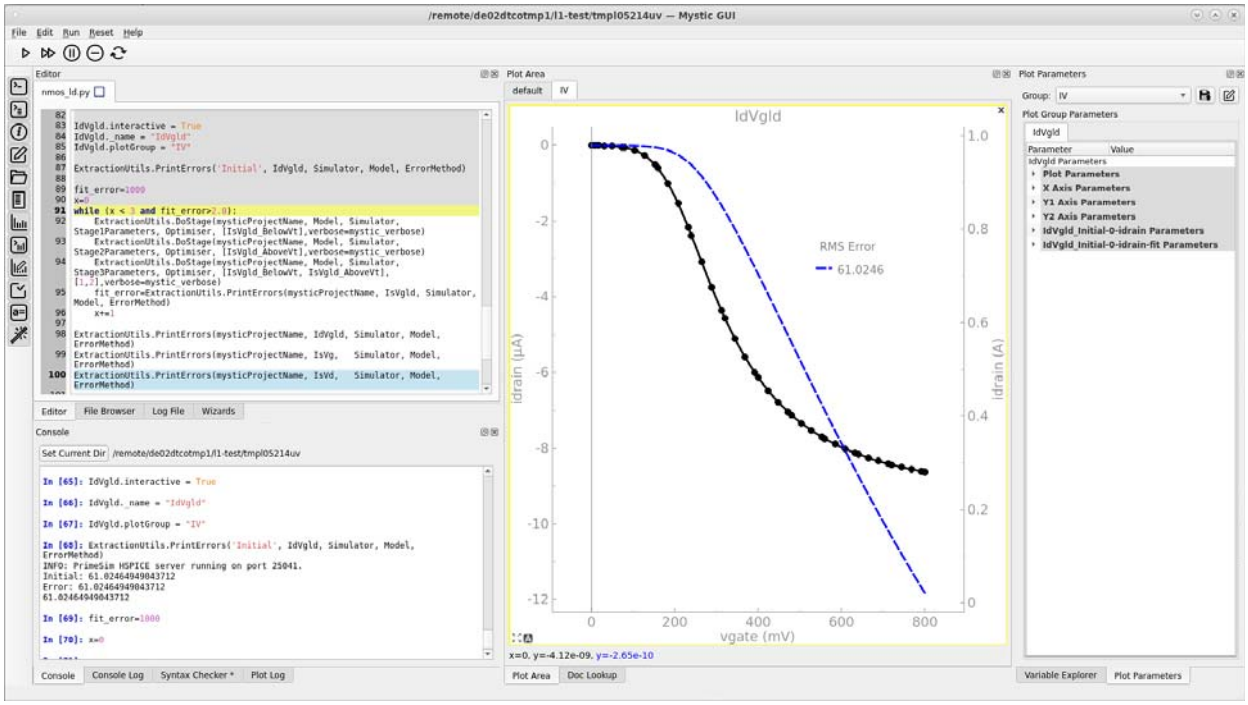
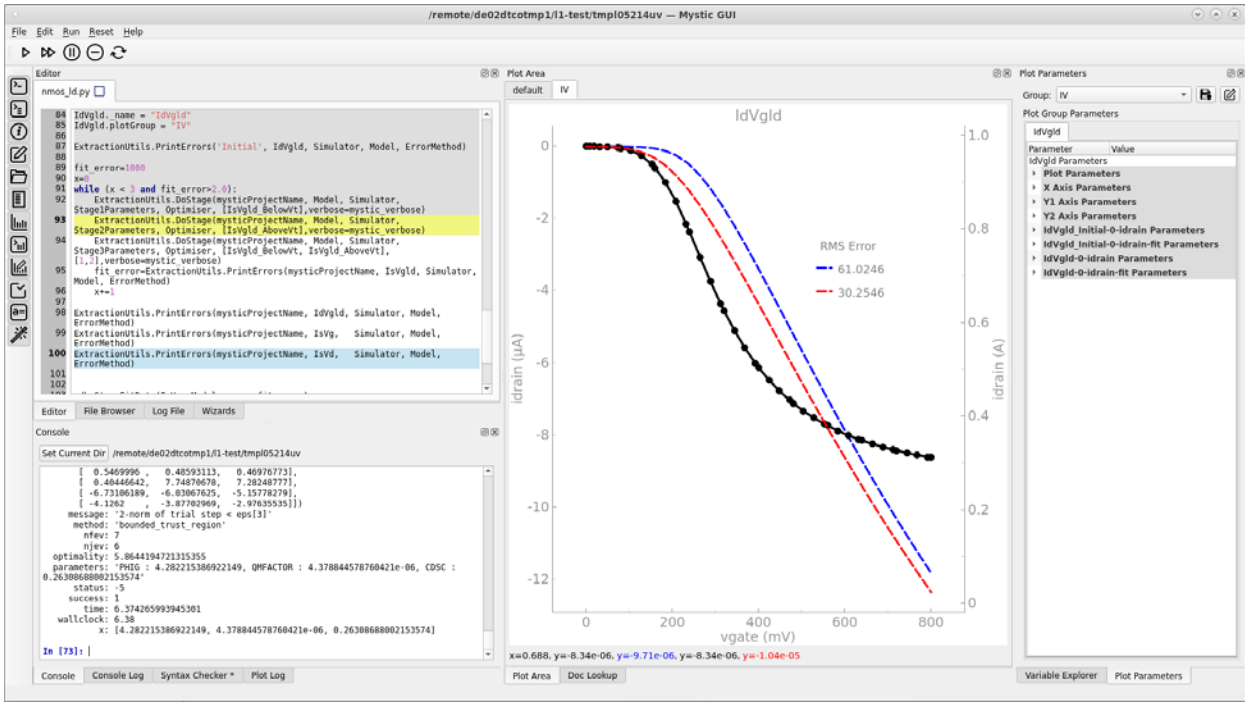



*Figure 43 Real-time fit of the model after the first call to ExtractionUtils.DoStage; initial conditions are shown by blue curve, and current fit is shown by red curve*

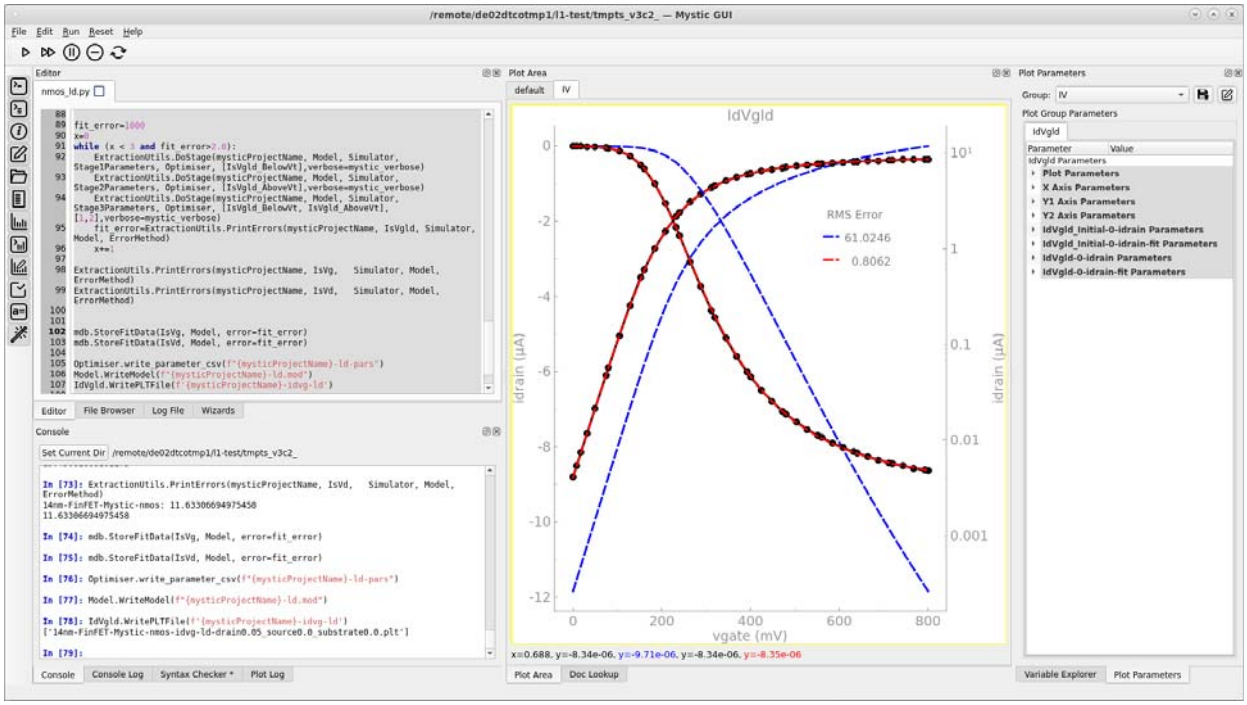


Now a red curve has been created showing the current status of the fit. The blue curve of the initial conditions remains on the plot. Now, you can run to the end and watch the extraction progress. Figure 44 shows the final result of the strategy.

You can see that the red fit curve has been updated to reflect the current fit of the model.

Now that the strategy has completed, the IPython console remains open for you to further analyze the results, rerun stages, or manually adjust SPICE model parameter values.

*Figure 44 Final result of low-drain $I_d$–$V_g$ strategy*




## Exploring Parameters Interactively

When developing a strategy from the beginning, you can use the Mystic GUI to investigate parameter sensitivity interactively. \

The `ExtractionUtils.InteractiveParamSweep` utility takes a single `SimData` target and a list of SPICE model parameters and creates an interactive plot where you can adjust parameter values using sliders and watch the response of the model in real time on the plot.

The following example shows an interactive parameter sweep for low-drain $I_d$–$V_g$ data:

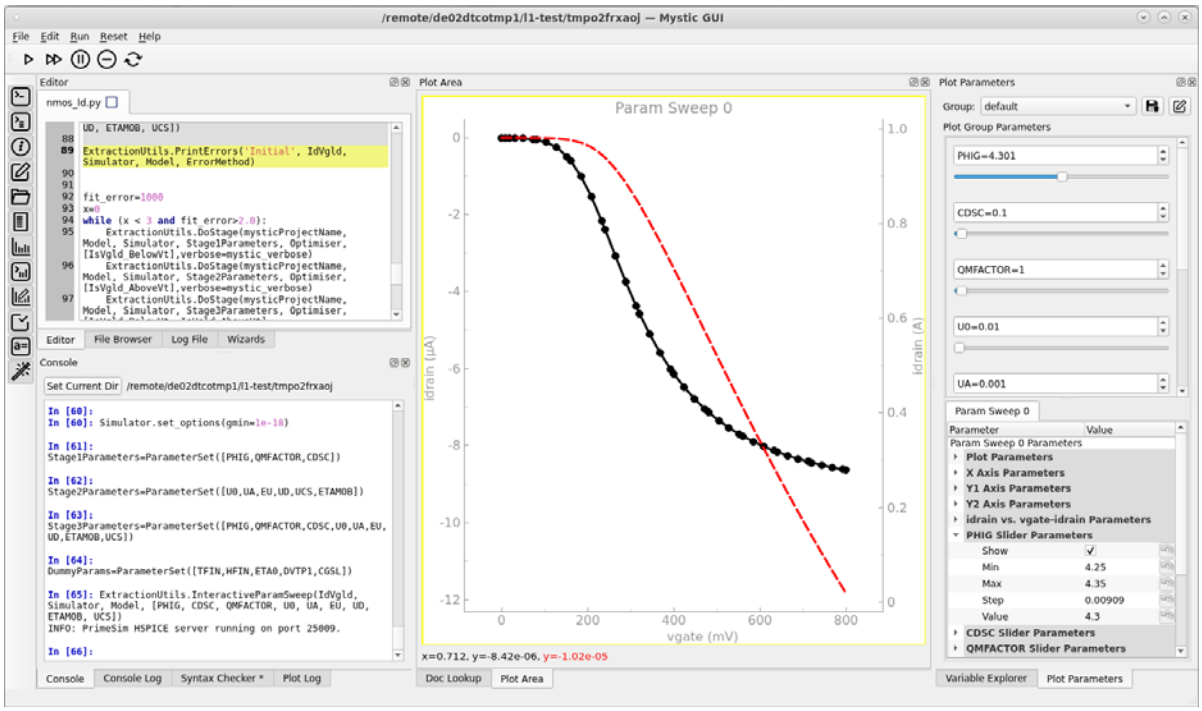
```
pmax = {"QMFACTOR": 1.0,
        "CDSC": 0.2,
        "U0": 0.1,
```

```
            "UA": 10.0,
            "UD": 0.2}

ExtractionUtils.InteractiveParamSweep(IdVgld, Simulator, Model,
                [PHIG,QMFACTOR,CDSC,U0,UA,EU,UD,ETAMOB,UCS],
                pmax=pmax, num=101)
```

The optional `pmax` dictionary provides the upper limit of the slider for the parameters. If this is not provided, or certain parameter names do not exist in the dictionary, then the maximum bound supplied in the `OptParam` definition is used. Minimum bounds can also be supplied using an optional `pmin` dictionary. The number of levels between the minimum and maximum bounds of the parameters is determined by the `num` argument. Figure 45 shows the interactive plot produced by `ExtractionUtils.InteractiveParamSweep`.

*Figure 45 Interactive parameter plot for low-drain $I_d$–$V_g$ data*




When you move each slider, the Mystic tool runs a backend simulation to evaluate the new set of parameters for the given target conditions and updates the plot accordingly. The interactive parameter plot can be used to manually fit a set of parameters for the model or to provide guidance in assembling an extraction strategy for the model based on the sensitivity of parameters to certain sections of the target data.

As can be seen in the right-hand side of the plot in Figure 45, each slider has adjustable boxes for current, minimum, and maximum values. The value box can be used to set values for the parameter that are not included in the step resolution of the slider, while the minimum

and maximum boxes can be used to adjust the slider range. When the range is changed, the step of the slider is automatically adjusted to one-hundredth of the range. The step can also be adjusted manually in the drop-down list for the slider in the Plot Parameters panel below. Sliders can be added by entering the name of an existing `OptParam` to the Add Slider box and removed by clicking the minus symbol next to the slider to be removed.

When you are satisfied with the quality of the fit, or you want to take a snapshot of the current model parameter values, you can use the model explorer in the Variable Explorer panel to save the model to disk.

## Parallel Coordinates Plots

When using the Mystic tool to extract geometry- or temperature-dependent models, the number of data points you are fitting across increases significantly, making it difficult to assess the quality of the model fit across all conditions using standard line plots. You can use parallel coordinates plots to effectively visualize and compare higher-dimensional datasets.

This example demonstrates how to use the Mystic GUI to visualize a SPICE model fitted for a 14 nm FinFET device across multiple gate lengths and fin widths. It assumes the model has been prefitted to a good standard and the example focuses on plotting.

To learn more about fitting across multiple geometries or temperatures using the Mystic tool, contact your local TCAD support team.

The starting point of this example is two `SimData` objects, `IdVgld` and `IdVghd`, representing low-drain and high-drain geometry-dependent data, respectively, for the 14 nm FinFET device. As the model is prefitted, both target data and fit data are already present.

A parallel coordinates plot requires a set of features that can be represented as single points for each geometry. For this FinFET device, you want to evaluate the fit at these metrics: Ioff-lin, Vt-lin, Ion-lin, Ioff-sat, Vt-sat, and Ion-sat. To do this, you must first calculate them using a figure of merit (FoM) transform as follows:

```
ld_foms = IdVgld.MakeFoMTarget(['ioff','vt','ion'])
hd_foms = IdVghd.MakeFoMTarget(['ioff','vt','ion'])
```

Now, you can create the plot and pass the FoM data to the parallel coordinates plotting function:

```
fig = addFigure('NMOS FoMs')

series = fig.plotParallelCoordinates([ld_foms, hd_foms],
                                     suffix=['_lin','_sat'])
```

Figure 46 shows the resulting parallel coordinates plot.

One series is created for the target data (black) and one is created for the fit data (red). You can see from Figure 46 that the model fits well across most metrics.

One of the main strengths of parallel coordinates plots is their ability to relate trends in the output metrics to changes in the inputs. When a large amount of data is being analyzed on the plot, it can quickly become difficult to track these patterns in the data. In the Mystic GUI, the visible data on the plot can be filtered, based on selection criteria on each axis.

To add a selection, hold the Shift key and drag one point of an axis to a new point. This filters out the data outside of the selection region.

*Figure 46 Parallel coordinates plot of geometry-dependent fitting for a 14 nm FinFET device: black=target data and red=fit data*

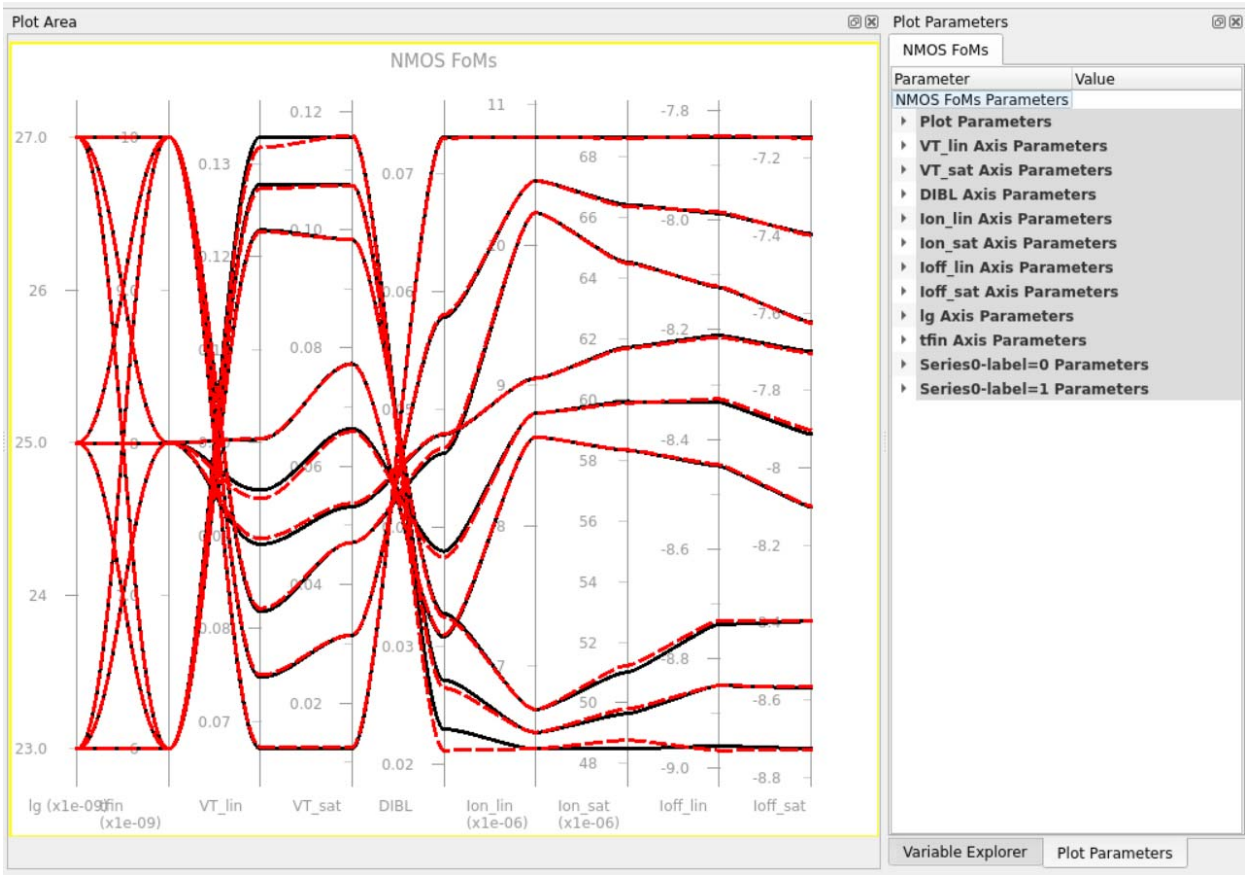
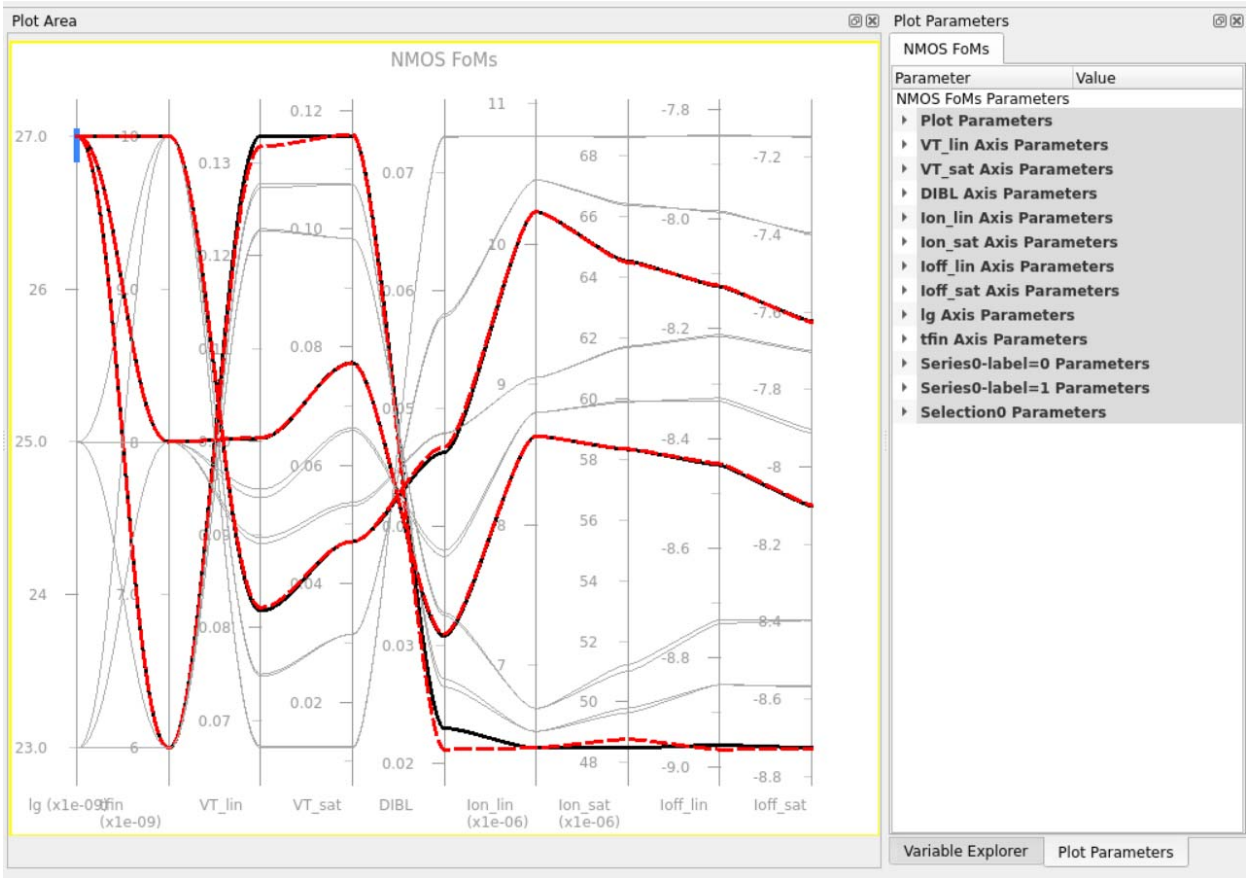



Figure 47 shows a selection filter placed at 27 nm on the Lg axis. The data at all other gate lengths is gray. The selection can be extended or moved at any time by holding the Shift key while dragging the selection line or by using the selection menu that has been added to the Plot Parameters panel.

Multiple selections can be applied to any of the axes to further filter the data based on as many selection criteria as you want.

*Figure 47 Parallel coordinates plot with a selection added at Lg=27 nm*




## RF Smith Charts and Polar Charts

As described in RF Extraction, RF devices require the scattering parameters (S-parameters) to be captured accurately. S-parameters come in the form of a 2×2 matrix with both real and imaginary parts and are most commonly plotted on Smith and polar charts.

You can create Smith and polar charts to compare target and fit S-parameter data for an RF device extraction. The reflection coefficients (S11 and S22) are usually viewed on a Smith chart, and the forward and reverse gains (S12 and S21) are usually viewed on a polar chart. This example focuses on how to plot S-parameter data that has been prefitted. For a full demonstration flow of an RF SPICE model extraction, contact your local TCAD support team.

The starting point for this example is a `SimData` object named `S_data` that contains a set of S-parameters at frequencies ranging from 27.7 GHz to 28.2 GHz as shown in Figure 48.

*Figure 48 S-parameter data for the example RF device*

| | s(11)re | s(11)i | ... | s(22)i-fit | errors |
|---|---|---|---|---|---|
| frequency | | | ... | | |
| 2.772000e+10 | 4.686165e-01 | -7.190487e-01 | ... | -3.525271e-01 | 2.539432e-02 |
| 2.772600e+10 | 4.672343e-01 | -7.194720e-01 | ... | -3.527785e-01 | 2.540475e-02 |
| 2.783200e+10 | 4.658533e-01 | -7.198920e-01 | ... | -3.530283e-01 | 2.541532e-02 |
| 2.788800e+10 | 4.644735e-01 | -7.203088e-01 | ... | -3.532768e-01 | 2.542601e-02 |
| 2.794400e+10 | 4.630949e-01 | -7.207224e-01 | ... | -3.535237e-01 | 2.543684e-02 |
| 2.800000e+10 | 4.617175e-01 | -7.211327e-01 | ... | -3.537692e-01 | 2.544779e-02 |
| 2.805600e+10 | 4.603414e-01 | -7.215398e-01 | ... | -3.540132e-01 | 2.545888e-02 |
| 2.811200e+10 | 4.589664e-01 | -7.219437e-01 | ... | -3.542558e-01 | 2.547011e-02 |
| 2.816800e+10 | 4.575927e-01 | -7.223444e-01 | ... | -3.544970e-01 | 2.548147e-02 |
| 2.822400e+10 | 4.532203e-01 | -7.227419e-01 | ... | -3.547367e-01 | 2.549297e-02 |
| 2.828000e+10 | 4.548490e-01 | -7.231362e-01 | ... | -3.549750e-01 | 2.550460e-02 |

You can create a Smith chart from this data by using the following commands:

```
fig = addFigure('Smith Chart')
target = fig.plot(S_data,
                  column=['s(11)re', 's(11)i','s(22)re', 's(22)i'],
                  mode="smith", colour="#000000", markerStyle='o')

fit = fig.plot(S_data,
               column=['s(11)re-fit', 's(11)i-fit', 's(22)re-fit',
               's(22)i-fit'],
               mode="smith", colour="#FF0000", markerStyle='Triangle')
```

Polar charts can be created from the same data by using the following commands:

```
fig = addFigure('Polar Chart')
target = fig.plot(S_data,
                  column=['s(12)re', 's(12)i', 's(21)re', 's(21)i'],
                  mode="polar", colour="#000000", markerStyle='o')

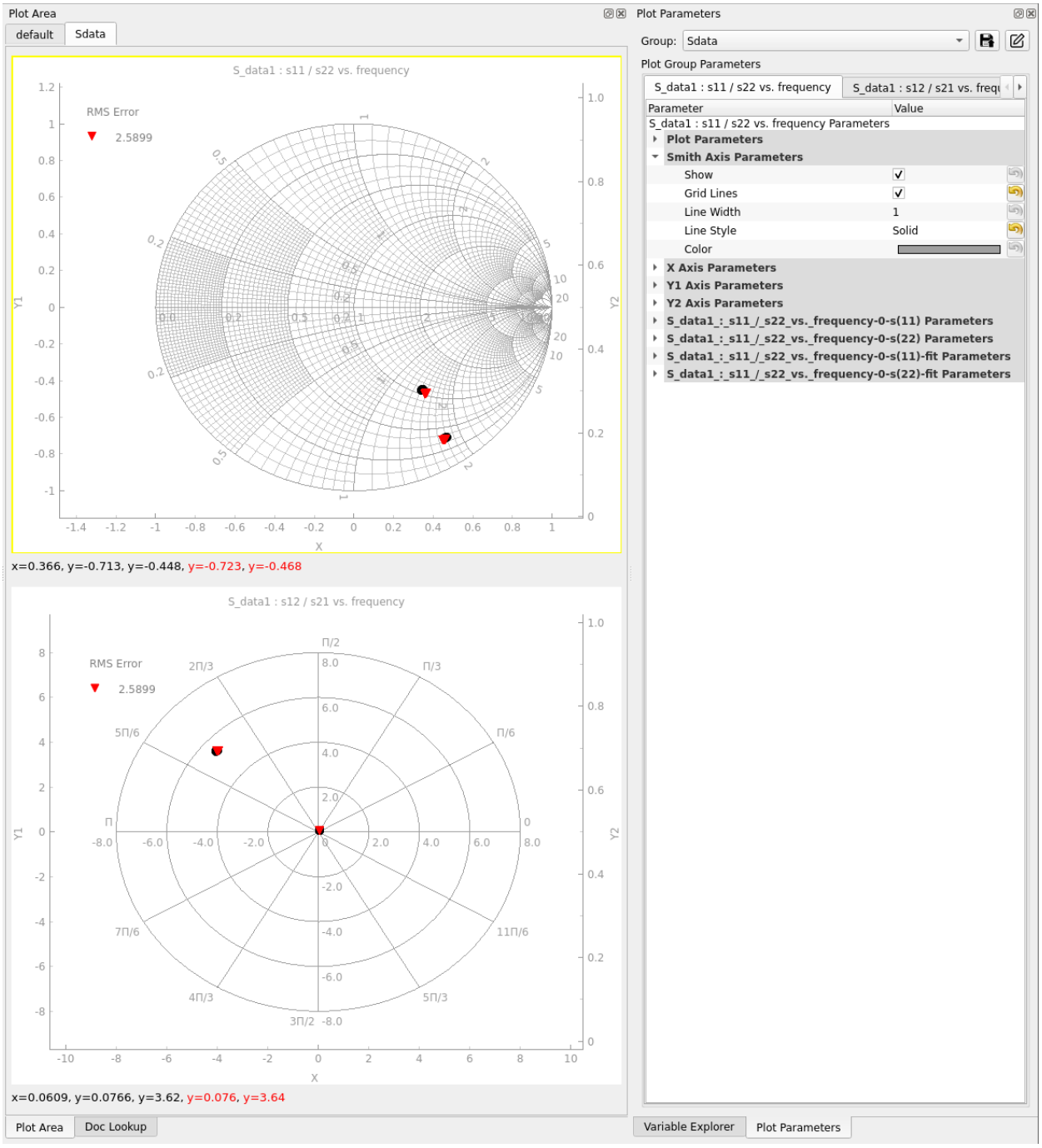
fit = fig.plot(S_data,
               column=['s(12)re-fit', 's(12)i-fit', 's(21)re-fit',
               's(21)i-fit'],
               mode="polar", colour="#FF0000", markerStyle='Triangle')
```

For Smith charts, an extra Smith axis parameter menu is added to the Plot Parameters panel. This menu controls the properties of the circular Smith axis.

Figure 49 shows the generated plots.

For polar charts, a polar axis parameter is added for the same purpose. S-Parameter data can also be tagged as interactive in the same way as I-V data shown in Running a Strategy Interactively on page 101. In this case, the Smith and Polar charts are created automatically and updated with the current state of `fit` upon every call to `ExtractionUtils.DoStage`.

*Figure 49 Smith chart and polar chart for example RF device showing target data (black) and fit data (red)*



# References

[1] For more information, go to https://ipython.readthedocs.io/en/stable/interactive/magics.html.

# 6

# Mystic Examples

*This chapter presents a step by step guide through an example packaged with the Mystic tool.*

## Location of Examples

You can find all of the examples in the following directory, in your installation directory:

```
$STROOT/tcad/$STRELEASE/manuals/tcad_spice/mystic_api/examples
```

**Note:**

Before running any example, ensure that both the Mystic tool and the PrimeSim HSPICE tool are available in your environment and licenses for both tools are available through the `SNPSLMD_LICENSE_FILE` environment variable.

To check whether both tools are available and that you have the required licences, enter the following on the command line:

```
:$ Mystic --version
=============================================================================
                          TCAD to SPICE - Mystic

                      Version V-2023.12 for linux64

                 Copyright (c) 2010-2023 Synopsys, Inc.
       This Synopsys software and all associated documentation are
       proprietary to Synopsys, Inc. and may only be used pursuant
     to the terms and conditions of a written license agreement with
       Synopsys, Inc. All other use, reproduction, modification, or
         distribution of the Synopsys software or the associated
                    documentation is strictly prohibited.
=============================================================================
Mystic V-2023.12 (9c5934d)
```

For these examples, you will run the Mystic tool on the command line and then use Sentaurus Visual to view the results.

These examples assume you have some familiarity with the Mystic extraction environment and how to structure an extraction strategy. For more information, see Full Strategy on page 52.

## 14 nm FinFET Example

This example is located in your installation directory as follows:

```
$STROOT/tcad/$STRELEASE/manuals/tcad_spice/mystic_api/examples/
14nmFinFET.zip
```

When you unzip the file in a new directory, you should have the following files and directories:

- `models/nmos.mod`: The initial model card. This is a BSIM-CMG subcircuit model set up to represent a 14 nm FinFET technology. Some default parameter values have been set. For details about the default parameters, refer to the TCAD to SPICE module of the TCAD Sentaurus Tutorial (see TCAD Sentaurus Tutorial: Simulation Projects on page 10).
- `results`: An empty directory where SPICE model extraction results will be written.
- `strategies`: A directory containing three extraction strategies to be run sequentially: a low-drain $I_d$–$V_g$ strategy, a high-drain $I_d$–$V_g$ and $I_d$–$V_d$ strategy, and a C–V strategy. It also contains Sentaurus Visual plotting scripts.
- `targets`: A directory containing the I–V and C–V target data for the extraction.

The extraction is separated into three separate stages: low-drain $I_d$–$V_g$, high-drain $I_d$–$V_g$ and $I_d$–$V_d$, and C–V. Splitting up the extraction helps to check the strategy and retain partial results while working on downstream stages.

### First Stage: Low-Drain Extraction

The first extraction stage fits the low-drain $I_d$–$V_g$ target data. To execute this stage, navigate to the unzipped `14nmFinFET` file and run:

```
Mystic strategies/ld_extraction.py --model=models/nmos.mod
```

Mystic should start executing in the terminal as follows:

```
=============================================================================
                          TCAD to SPICE - Mystic

                       Version V-2023.12 for linux64

                  Copyright (c) 2010-2023 Synopsys, Inc.
       This Synopsys software and all associated documentation are
```

```
      proprietary to Synopsys, Inc. and may only be used pursuant to
       the terms and conditions of a written license agreement with
       Synopsys, Inc. All other use, reproduction, modification, or
         distribution of the Synopsys software or the associated
                   documentation is strictly prohibited.
=============================================================================
  Loading:
          Extraction Strategy:     /<path>/strategies/ld_extraction.py
          Initial Model:           /<path>/models/nmos.mod
          Database:                No database loaded.
          SPICE Backend:           hspice
          Optimizer:               BOUNDED_TRUST_REGION

  INFO: HSPICE server running on port 25001.
  Initial: 60.37694266243185

  Fitting Result Data
  -------------------
  parameters = mosmod.PHIG : 4.228635892175941, mosmod.CDSC :
               0.1191443780274296
  iterations = 11
  fit error  = 0.020096712548626192
  converged  = 1
  method     = BOUNDED_TRUST_REGION
  wallclock  = 9.02s
```

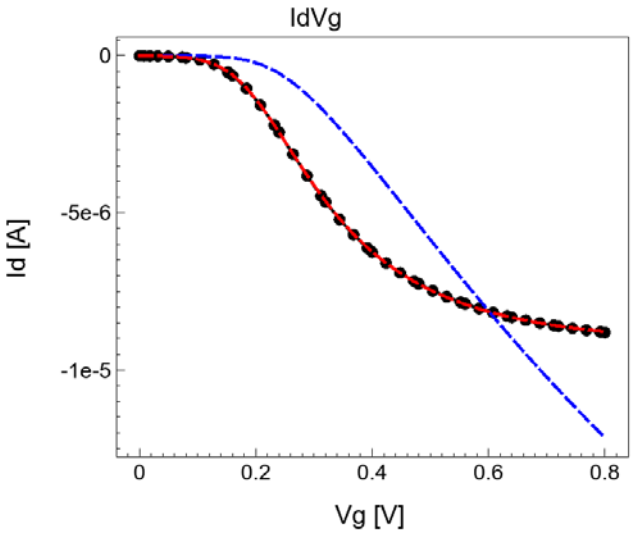
When the strategy has been completed, the results of the low-drain $I_d$–$V_g$ extraction is written to the `results` directory. The results can be visualized using Sentaurus Visual with the low-drain plotting script as follows:

```
svisual strategies/plot_ld.tcl
```

The final model card is also written to the `results` directory. This will be used as the starting point for the high-drain $I_d$–$V_g$ and $I_d$–$V_d$ extraction strategy (see Second Stage: High-Drain and $I_{d–Vd}$ Extraction on page 115).

Figure 50 shows the results in Sentaurus Visual.

*Figure 50 Low-drain $I_d$–$V_g$ extraction strategy results in Sentaurus Visual*




## Second Stage: High-Drain and $I_d$–$V_d$ Extraction

The high-drain $I_d$–$V_g$ and $I_d$–$V_d$ extraction strategy uses the output model card from the low-drain $I_d$–$V_g$ extraction strategy as a starting point. To run this strategy, use the following command:

```
Mystic strategies/hd_extraction.py --model=models/nmos-ld.mod
```

Again, the Mystic tool will start running in the terminal. This extraction strategy is much more complex than the low-drain $I_d$–$V_g$ extraction strategy, fitting a larger set of parameters to the full set of I–V curves. It will take longer to execute.

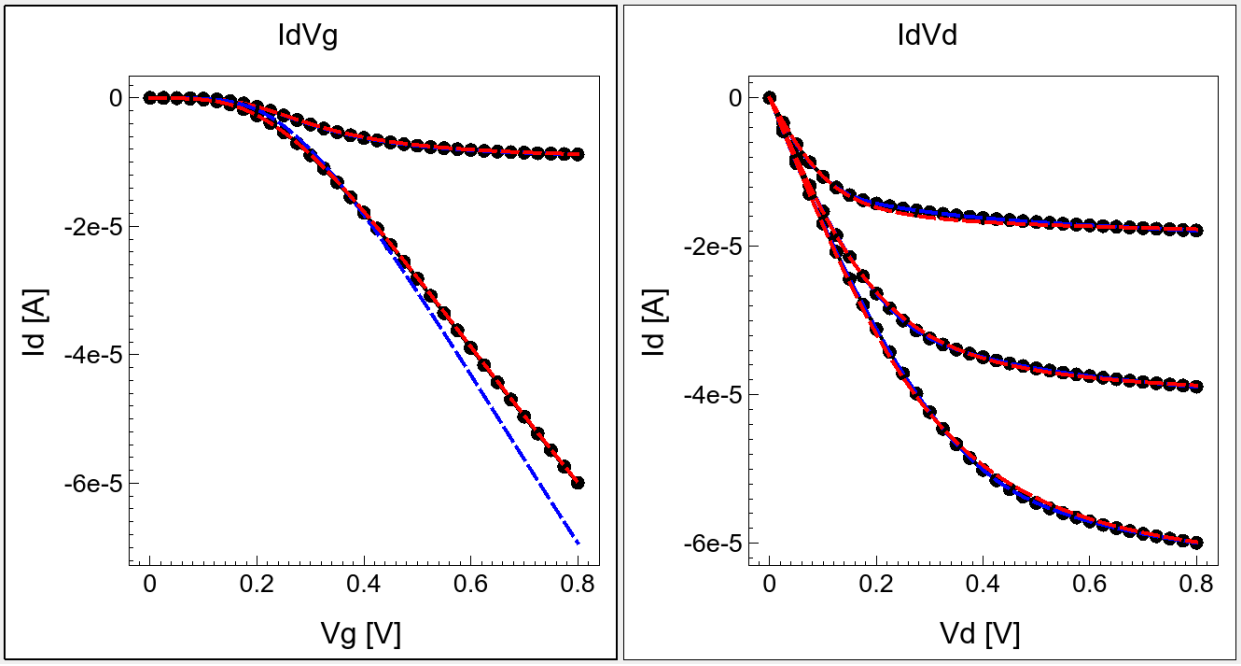
When the extraction has completed, you can visualize the results in Sentaurus Visual using the high-drain plotting script as follows:

```
svisual strategies/plot_hd.tcl
```

There will be two plots, for $I_d$–$V_g$ and $I_d$–$V_d$ data, as shown in Figure 51.

You can see from the initial conditions that the low-drain $I_d$–$V_g$ data is well fitted from the low-drain extraction strategy. The results of this extraction stage are again stored in the `results` directory. The fitted model will supply the initial conditions for the third and final extraction stage (see Third Stage: C–V Extraction on page 116).

*Figure 51 Initial conditions of high-drain strategy in Sentaurus Visual*




## Third Stage: C–V Extraction

The C–V extraction strategy uses the output model card from the high-drain $I_d$–$V_g$ and $I_d$–$V_d$ strategy as a starting point.

To run this strategy, use the following command:

```
Mystic strategies/cv_extraction.py --model=models/nmos-hd.mod
```

The C–V extraction strategy has the same basic structure as the two I–V strategies but operates on gate capacitance target data. The C–V targets require no special treatment, from a strategy perspective, as the Mystic tool inherently knows how to handle them on the backend.

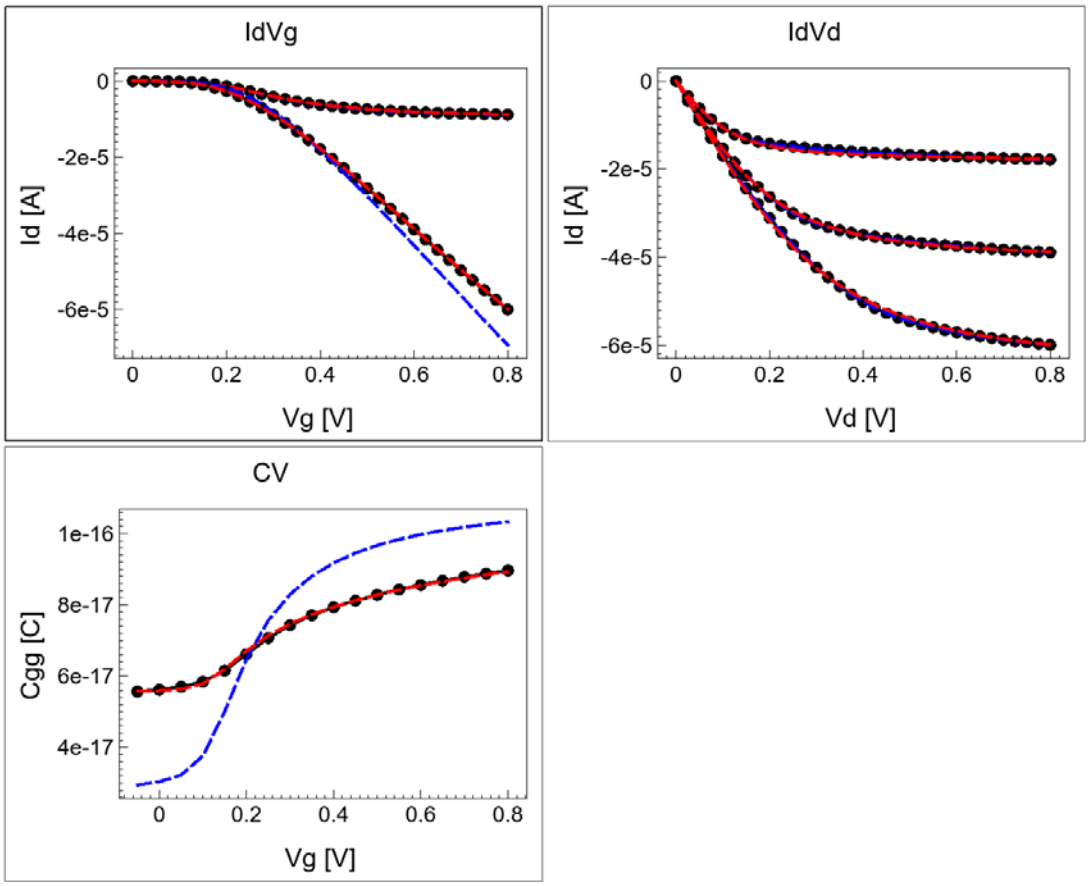
When the strategy has completed, you can visualize the results as follows:

```
svisual strategies/plot_cv.tcl
```

Now, all three targets used throughout the extraction procedure are displayed in Sentaurus Visual as shown in Figure 52.

The final fitted NMOS model card has been written to `models/nmos-final.mod`. This can now be used in any PrimeSim HSPICE circuit simulation.

*Figure 52 Results of capacitance extraction strategy in Sentaurus Visual*



# Glossary

**DoE**
Design-of-experiments.

**FoM**
Figure of merit.

**LUT**
Lookup table.

**MOSRA**
MOSFET reliability analysis.

**PCA**
Principal component analysis.

**PWL**
Piecewise linear.

**RMS**
Root mean square.

**RMSD**
Root mean square deviation.

# Index


## A

add_extraction_step method 52
agauss statement 63, 65

## B

Bayesian optimization algorithm 43, 45
Beta factor 35
breakpoints
  inserting 82, 86
BSIM-CMG model extraction 54

## C

checkpoints
  automated 36
command-line options 11
Console panel 84
CorrelatedParameters class 63, 67
CorrelatedParameters.set_correlation method 65
CorrelatedParameters.set_sigma method 65
CorrelatedParameters.update_model method 64, 65
CreateMOSRATarget method 61

## D

Data class 15
data field 21
data mask 31, 53
data structures
  combining datasets 25
database 20
differential evolution algorithm 43, 57
Doc Lookup panel 87

## E

Editor panel 81
Engine module 51
extract method 52
extraction
  Josephson junction 75
extraction environment 12
extraction strategy 52
ExtractionUtils.DoStage utility 72
ExtractionUtils.InteractiveParamSweep utility 104
ExtractionUtils.ParamSweep utility 42
ExtractionUtils.PrintErrors utility 66, 72, 75
ExtractionUtils.SensitivityAnalysis utility 42

## F

figure of merit transforms 27
figures of merit calculation 27
filter data 26
fringing capacitance extraction 72

## G

get_checkpoint method 36

## I

.INCLUDE command 34
Intel MKL bounded trust region 43
Intel MKL trust region 43
IPython console 84

## J

Josephson effect 75
Josephson junction 28, 29, 75
  methods 29

## L

line plots
  creating 90
lookup table models 73

## M

mathematical operators 26
mdb object 20

mdb.GetFitData method 22, 54
mdb.GetFitModel method 21
mdb.Load method 22, 23, 25, 54, 66, 74
mdb.StoreFitData method 22, 35
mdb.StoreFitParameters method 35
mean aggregation transform 30
.MEASURE command 32
metadata 16
metadata field 21
model
  checkpointing 35
Model.write method 35
MOSFET reliability (MOSRA) extraction 60
Mystic GUI 79
  Console panel 84
  Doc Lookup panel 87
  Editor panel 81
  inserting breakpoints 82, 86
  line plots 90
  Plot Area panel 90
  Plot Parameters panel 95
  Run toolbar 85
  Syntax Checker panel 82
  Wizards panel 99

## N

normal mode 37

## O

optimization algorithm
  Bayesian 45
  specifying 11
Optimizer.SetOptimizationParameter method 44
.OPTION command 37, 53

## P

parallel coordinates plot 106
parallel simulations 38
parameter space 41
parameter sweep 42
parasitic networks 68
Plot Area panel 90
Plot Parameters panel 95
pointwise error method 24
polar chart 108
principal component analysis 62
Python modules 13

## R

Run toolbar 85

## S

scale factor 44
sensitivity analysis 42
server mode 37, 53
set_checkpoint method 35
SimData.ApplyMeasure method 32
SimData.ExtendBias method 33
SimData.FilterData method 26, 31
SimData.FromPLT method 55
SimData.GenerateData method 37
SimData.MakeFoMTarget method 30
SimData.resampleNd method 74
SimData.SetFoMCriteria method 30
SimData.SetOptions method 53
SimData.WriteCSVFile method 33
SimData.WritePLTFile method 33
Simulator.executable 75
Simulator.set_options method 37
Simulator.write_LUT method 74
Smith chart 108
S-parameters 108
strategy file
  editing 81
supercurrent 75
SwitchMOSRAEquation method 62
Syntax Checker panel 82

## T

target data 23
TestMeasure method 33
transient simulation 58

## V

variability
  principal component analysis 62

## W

weighting factors 72
Wizards panel 99
WriteCSVFile method 73
WriteMOSRAFile method 62
WritePLTFile method 73